

C# và ASP.NET Core — Tập 2

Lập trình web hiện đại với ASP.NET Core, Entity Framework
Core và Blazor — giả định đã nắm Tập 1

https://github.com/phamtuanchip/cshap_book

Mục lục

Phần 0 — Nền tảng web và môi trường

1. Web hoạt động thế nào: HTTP và REST
2. Dự án ASP.NET Core đầu tiên

Phần 1 — Cốt lõi ASP.NET Core

3. Middleware và request pipeline
4. Routing và Minimal API
5. Dependency Injection
6. Cấu hình, Options và Logging

Phần 2 — Web API

7. Web API với Controller
8. DTO và Validation
9. Xử lý lỗi, versioning và OpenAPI
10. Kiểm thử API

Phần 3 — Dữ liệu với EF Core

11. SQL cơ bản và SQLite
12. Entity Framework Core cơ bản
13. Quan hệ, truy vấn nâng cao và hiệu năng
14. Web API hoàn chỉnh với EF Core

Phần 4 — Giao diện web

15. ASP.NET Core MVC
16. Razor Pages
17. Form, validation phía server và chống CSRF
18. Blazor
19. Gọi API ngoài với HttpClient

Phần 5 — Bảo mật cơ bản

20. Xác thực và phân quyền

21. Bảo mật ứng dụng web

Phần 6 — Dự án tổng hợp

22. Dự án tổng hợp Tập 2: Web quản lý kho

Phụ lục

23. Phụ lục A — Lỗi thường gặp ASP.NET Core

24. Phụ lục B — Tài liệu tham khảo

25. Phụ lục C — Cheat sheet ASP.NET Core

Chương 1 — Web hoạt động thế nào: HTTP và REST

Mục tiêu học

Sau chương này, bạn sẽ:

- Hiểu mô hình **client – server** và một **request/response HTTP** gồm những phần nào.
- Đọc hiểu **phương thức, mã trạng thái, header, body** và cấu trúc một **URL**.
- Nắm các nguyên tắc **REST** và cách thiết kế URL cho một API.
- Dùng công cụ (`curl` , trình duyệt, file `.http`) để "nói chuyện" trực tiếp với một server.

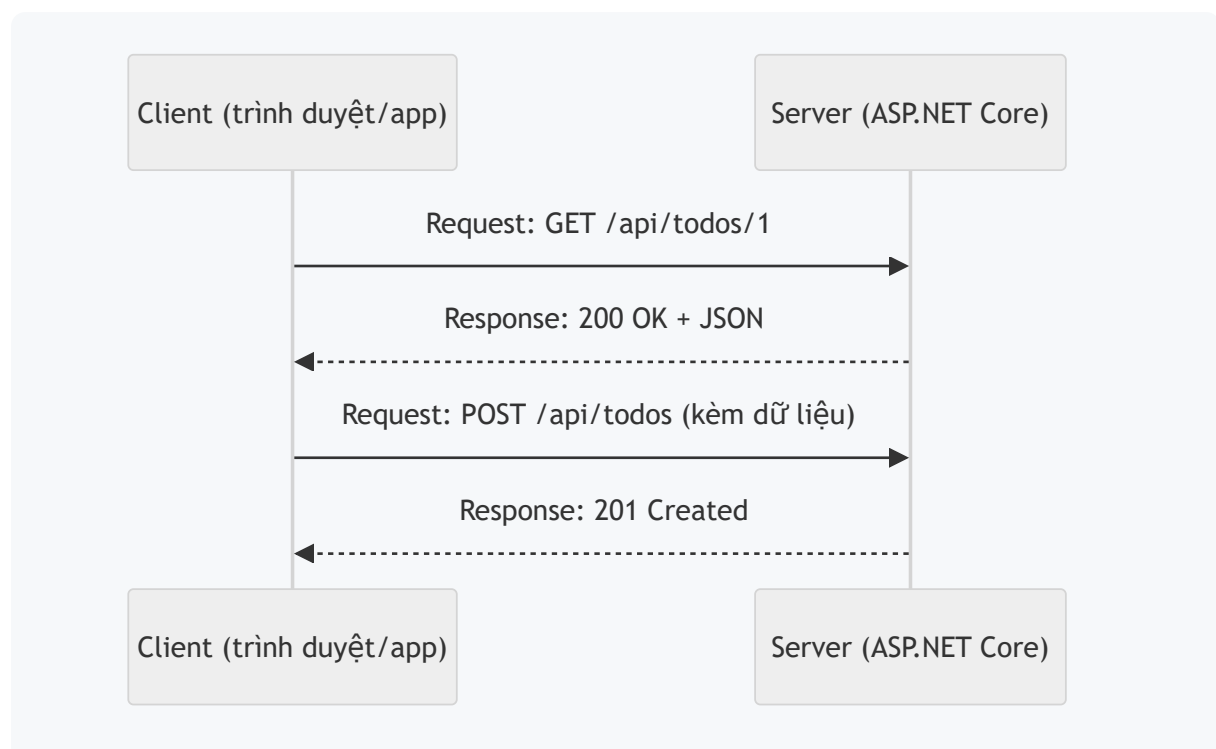
Code mẫu: [code/ch01-http-co-ban/](#) — chương trình console tự dựng một máy chủ HTTP thô để bạn **nhìn thấy** nguyên văn request và response.

Tập 2 giả định bạn đã đọc Tập 1 (C#, OOP, LINQ, `async/await` , JSON, DI qua constructor, unit test). Khi cần ôn, sách ghi rõ "*Tập 1, Chương N*".

Mô hình client – server

Mọi ứng dụng web đều là cuộc **hỏi – đáp** giữa hai bên:

- **Client** (trình duyệt, ứng dụng di động, chương trình khác) **gửi request** để hỏi/nhờ.
- **Server** ([ASP.NET](#) Core chạy trên máy chủ) **trả response**.



HTTP (HyperText Transfer Protocol) là "ngôn ngữ" quy định cách hai bên viết request và response. Nó chạy trên TCP (và **HTTPS** = HTTP + mã hoá TLS). Đặc điểm cốt lõi: **phi trạng thái (stateless)** — mỗi request độc lập, server không tự nhớ request trước (muốn nhớ phải dùng cookie, token, session).

Giải phẫu một request

Chạy code mẫu, bạn sẽ thấy đúng nguyên văn bản mà `HttpClient` gửi đi:

```
GET /api/chao?ten=An&lang=vi HTTP/1.1
Host: 127.0.0.1:64457
Accept: application/json
X-Yeu-Cau: thu-nghiem
```

Phần	Ví dụ	Ý nghĩa
Request line	GET /api/chao?ten=An HTTP/1.1	phương thức + đường dẫn (kèm query) + phiên bản
Headers	Host, Accept, Authorization, Content-Type	siêu dữ liệu về request
Dòng trống		ngăn cách header và body
Body (tùy chọn)	{"title":"Hoc C#"}	dữ liệu gửi kèm (POST/PUT/PATCH)

URL

```
https://api.cuahang.vn:8443/api/san-pham/42?nhom=phu-kien&sapXep=gia#chi-tiet
|_____|_____|_____|_____|_____|_____|
scheme  host  port  path  query  fragment
```

fragment (#...) **không** gửi lên server — chỉ dùng phía client. query (?a=1&b=2) là các cặp tên-giá trị. Ký tự đặc biệt phải được mã hoá phần trăm (%20 cho dấu cách) — `Uri` / `HttpClient` lo giúp.

Phương thức HTTP

Phương thức	Dùng để	An toàn*	Idempotent**
GET	đọc tài nguyên	✓	✓
POST	tạo mới / thực hiện hành động	✗	✗

Phương thức	Dùng để	An toàn*	Idempotent**
PUT	thay thế toàn bộ tài nguyên	X	✓
PATCH	sửa một phần	X	thường không
DELETE	xoá	X	✓
HEAD/OPTIONS	lấy header / hỏi khả năng (CORS)	✓	✓

* *An toàn*: không làm thay đổi dữ liệu trên server. ** *Idempotent*: gọi nhiều lần cho kết quả cuối giống gọi một lần (xoá hai lần vẫn là "đã xoá"). Tính chất này quyết định việc có thể **tự động gửi lại** khi mạng lỗi hay không — không bao giờ để GET làm thay đổi dữ liệu.

Mã trạng thái (status code)

Hàng trăm đầu tiên cho biết **nhóm** kết quả:

Nhóm	Ý nghĩa	Thường gặp
1xx	thông tin	ít gặp
2xx	thành công	200 OK, 201 Created, 204 No Content
3xx	chuyển hướng	301/302 (redirect), 304 Not Modified
4xx	lỗi do client	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 422, 429 Too Many Requests
5xx	lỗi do server	500 Internal Server Error, 502/503/504

Phân biệt hay nhầm: **401** = chưa xác thực (chưa đăng nhập / token sai); **403** = đã biết bạn là ai nhưng **không đủ quyền**. Chọn mã đúng là phần quan trọng của thiết kế API: POST tạo mới nên trả 201 Created kèm header Location; xoá thành công thường 204.

Header và body

Header thường gặp:

Header	Hướng	Ý nghĩa
Content-Type	cả hai	định dạng body (application/json, text/html, multipart/form-data)
Accept	request	định dạng client muốn nhận

Header	Hướng	Ý nghĩa
Authorization	request	thông tin xác thực (Bearer <token>)
Cookie / Set-Cookie	request / response	duy trì trạng thái
Location	response	URL tài nguyên mới tạo/chuyển hướng
Cache-Control, ETag	response	điều khiển bộ nhớ đệm

Body JSON (Tập 1, Chương 36) là định dạng chính của API hiện đại. Form HTML gửi `application/x-www-form-urlencoded` hoặc `multipart/form-data` (tải file).

REST — thiết kế API theo tài nguyên

REST là phong cách thiết kế API xoay quanh **tài nguyên** (danh từ) được định danh bằng URL, và thao tác bằng **phương thức HTTP** (động từ):

Việc cần làm	Request	Kết quả
Lấy danh sách	GET /api/todos	200 + mảng
Lấy một	GET /api/todos/5	200 hoặc 404
Tạo mới	POST /api/todos	201 + Location: /api/todos/6
Thay thế	PUT /api/todos/5	204 hoặc 404
Sửa một phần	PATCH /api/todos/5	200/204
Xoá	DELETE /api/todos/5	204
Quan hệ	GET /api/khach-hang/3/don-hang	các đơn của khách 3

Quy tắc đặt URL: dùng **danh từ số nhiều**, chữ thường, phân cách bằng `-`, không đặt động từ vào URL (/api/todos , không phải /api/layDanhSachTodo); lọc/sắp xếp/phân trang qua query (?done=true&sort=ten&page=2&pageSize=20); phiên bản qua đường dẫn hoặc header (/api/v1/...).

Một API REST **không** phụ thuộc trạng thái phiên: mỗi request mang đủ thông tin (token) để xử lý. Đó là lý do nó mở rộng ngang dễ dàng (thêm server sau bộ cân bằng tải).

Công cụ để thử request

- **Trình duyệt**: gõ URL (chỉ GET); **DevTools** → **Network** xem mọi request/response, header, thời gian.

- **curl** (có sẵn Windows 10+, macOS, Linux):

```
curl -i http://localhost:5000/api/todos
curl -i -X POST http://localhost:5000/api/todos -H "Content-Type: application/json" -d '{"
```

`-i` in cả header response; `-X` chọn phương thức; `-H` thêm header; `-d` gửi body.

- **File .http** (Visual Studio 2022+ và VS Code với REST Client): request lưu ngay trong repo, chạy được từ IDE:

```
### Lay danh sach
GET {{host}}/api/todos
Accept: application/json

### Tao moi
POST {{host}}/api/todos
Content-Type: application/json

{ "title": "Hoc HTTP" }
```

- **Postman/Insomnia/Bruno**: giao diện đồ hoạ, lưu bộ sưu tập request.
- **Swagger UI / Scalar**: giao diện thử API sinh từ tài liệu OpenAPI (Chương 9).

Từ code C# nhìn HTTP

Trong code mẫu, `HttpClient.GetAsync(url)` (Tập 1, Chương 33 — async) gửi request; server thô đọc và in ra nguyên văn, rồi trả:

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Length: 58
X-May-Chu: demo
Connection: close

{"loiChao":"Xin chao tu may chu tu viet","thanhCong":true}
```

[ASP.NET](#) Core làm đúng việc đó — nhưng lo giúp bạn: phân tích request, định tuyến, chuyển JSON ↔ đối tượng C#, header, mã hoá, đa luồng, keep-alive, HTTP/2... Bạn viết nghiệp vụ, framework lo giao thức.

Lỗi thường gặp

- Dùng `GET` để **thay đổi** dữ liệu (mất tính an toàn; trình duyệt/crawler có thể gọi lại).

- Trả `200 OK` kèm nội dung lỗi thay vì mã `4xx/5xx` đúng nghĩa.
- Nhầm `401` với `403`.
- Đặt động từ trong URL, dùng danh từ số ít/số nhiều lộn xộn.
- Quên `Content-Type: application/json` khi gửi body JSON → server không đọc được.
- Nhét dữ liệu nhạy cảm (mật khẩu, token) vào **URL/query** — URL bị ghi vào log và lịch sử trình duyệt; đặt vào header/body qua HTTPS.

Bài tập

1. Chạy code mẫu; giải thích từng dòng của request in ra. Đổi header `Accept` rồi quan sát.
2. Dùng DevTools mở một trang web bất kỳ, tìm request `GET` đầu tiên, ghi lại mã trạng thái và 3 header của response.
3. Thiết kế (chỉ viết ra giấy) bộ URL REST cho hệ thống "thư viện": sách, đọc giả, phiếu mượn (kể cả "trả sách").
4. Dùng `curl -i https://example.com` và giải thích mọi dòng header.
5. Viết chương trình dùng `HttpClient` gọi `https://httpbin.org/status/404` và in mã trạng thái (cần Internet); xử lý bằng `response.IsSuccessStatusCode`.

Chương 2 — Dự án [ASP.NET](#) Core đầu tiên

Mục tiêu học

Sau chương này, bạn sẽ:

- Tạo, chạy và **hot reload** một dự án [ASP.NET](#) Core bằng `dotnet` CLI.
- Đọc hiểu file `Program.cs` theo bốn bước: **builder** → **services** → **middleware/endpoint** → **run**.
- Biết các mẫu dự án (`web` , `webapi` , `mvc` , `razor` , `blazor`) và cấu trúc thư mục.
- Hiểu **Kestrel**, `launchSettings.json` , môi trường `Development` / `Production` , HTTPS dev certificate.

Code mẫu: [code/ch02-du-an-dau-tien/](#) .

[ASP.NET](#) Core là gì?

[ASP.NET](#) Core là framework mã nguồn mở, đa nền tảng của Microsoft để xây dựng ứng dụng web, API, dịch vụ nền, ứng dụng thời gian thực... trên .NET. Chạy trên Windows, macOS, Linux, container; hiệu năng thuộc nhóm hàng đầu. Nó là **một** framework thống nhất với nhiều "kiểu ứng dụng" xây trên cùng nền:

Kiểu	Dùng cho	Chương
Minimal API / Web API (controller)	API JSON cho web/mobile/dịch vụ khác	4, 7
MVC	web nhiều trang phía server (HTML)	15
Razor Pages	web trang-là-đơn-vị, đơn giản hơn MVC	16
Blazor	giao diện tương tác bằng C# (thay JavaScript)	18
SignalR, gRPC, Worker Service	thời gian thực, RPC, dịch vụ nền	Tập 3

Tất cả dùng chung: middleware pipeline, dependency injection, cấu hình, logging, xác thực — học một lần dùng mọi nơi.

Tạo dự án

```
dotnet new list web           # xem các template web
dotnet new web -n WebDauTien  # ứng dụng rỗng tối thiểu
cd WebDauTien
dotnet run
```

Terminal hiển thị:

```
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5102
```

Mở địa chỉ đó bằng trình duyệt: thấy `Hello World!` . Nhấn `Ctrl+C` để dừng.

Template	Lệnh	Gồm
Rỗng	<code>dotnet new web</code>	chỉ <code>Program.cs</code> — nền tảng để học (dùng trong chương này)
Web API	<code>dotnet new webapi</code>	API mẫu + OpenAPI
MVC	<code>dotnet new mvc</code>	Controllers/Views/Models
Razor Pages	<code>dotnet new razor</code>	thư mục Pages
Blazor Web App	<code>dotnet new blazor</code>	components Razor

Trong Visual Studio: **File** → **New** → **Project** → [ASP.NET Core Empty](#). Chọn "*Configure for HTTPS*" và đừng bật "*Enable container support*" ở giai đoạn này.

Program.cs — bốn bước

```
var builder = WebApplication.CreateBuilder(args); // (1) Cấu hình

// (2) Đăng ký dịch vụ vào DI container: builder.Services.AddXxx(...)

var app = builder.Build(); // Dựn  ứng dụng

// (3) Cấu hình middleware pipeline + endpoint: app.UseXxx(), app.MapGet(...)
app.MapGet("/", () => "Xin chao ASP.NET Core!");
app.MapGet("/chao/{ten}", (string ten) => $"Xin chao, {ten}!");
app.MapGet("/cong", (int a, int b) => new { a, b, tong = a + b });

app.Run(); // (4) Chạy máy chủ, chặn tới kh
```

- `WebApplication.CreateBuilder(args)` : tạo builder đã cấu hình sẵn: nạp `appsettings.json` , biến môi trường, tham số dòng lệnh, logging, Kestrel, DI. Bạn hầu như không cần tự cấu hình những thứ này.
- `builder.Services` : đăng ký dịch vụ (Chương 5) — *trước* `Build()` ; sau `Build()` không đăng ký thêm được.

3. `app.Use.../Map...` : xếp middleware theo thứ tự (Chương 3) và khai báo endpoint (Chương 4).
4. `app.Run()` : khởi động và **chặn** luồng hiện tại cho tới khi nhận tín hiệu tắt (Ctrl+C, SIGTERM).

Đây là **minimal hosting model**: không còn `Startup.cs` như [ASP.NET](#) Core 3.x/5 — một file duy nhất, nhờ *top-level statements* (Tập 1, Chương 3). Lambda `() => ...` là **route handler**; giá trị trả về được tự chuyển thành response: `string` → `text/plain`, `object` → **JSON**, `ActionResult` → tùy ý.

Kiểm tra các endpoint:

```
curl http://localhost:5102/
curl http://localhost:5102/chao/An
curl "http://localhost:5102/cong?a=2&b=3"      # {"a":2,"b":3,"tong":5}
```

Tham số handler được **tự động gắn (bind)**: `ten` lấy từ đường dẫn `{ten}`, `a`, `b` từ query string, kiểu `int` được chuyển đổi và kiểm tra (gửi `?a=abc` sẽ nhận `400`). Chương 4 sẽ nói kỹ.

File .csproj và cấu trúc thư mục

```
<Project Sdk="Microsoft.NET.Sdk.Web">
  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>
</Project>
```

Khác Tập 1: SDK là **Microsoft.NET.Sdk.Web** (thêm framework [ASP.NET](#) Core; `ImplicitUsings` thêm `Microsoft.AspNetCore.*`). Thư mục điển hình:

```
WebDauTien/
├─ WebDauTien.csproj
├─ Program.cs           # điểm vào
├─ appsettings.json     # cấu hình chung
├─ appsettings.Development.json # cấu hình riêng cho môi trường Development
├─ Properties/launchSettings.json # cấu hình chạy khi phát triển (URL, biến môi trường)
├─ wwwroot/             # file tĩnh (css, js, ảnh) – nếu có UseStaticFiles
└─ bin/ obj/            # sinh ra khi build (không đưa vào Git)
```

launchSettings.json, môi trường và HTTPS

```
{
  "profiles": {
    "http": {
      "commandName": "Project",
      "launchBrowser": false,
      "applicationUrl": "http://localhost:5102",
      "environmentVariables": { "ASPNETCORE_ENVIRONMENT": "Development" }
    }
  }
}
```

- File này **chỉ dùng khi phát triển** (`dotnet run` , F5) — không được đóng gói khi publish.
- `ASPNETCORE_ENVIRONMENT` : `Development` , `Staging` , `Production` (mặc định `Production` nếu không đặt). Quyết định file `appsettings.{Env}.json` nào được nạp và hành vi như trang lỗi chi tiết:

```
if (app.Environment.IsDevelopment()) { /* công cụ chỉ cho dev */ }
```

- **HTTPS khi phát triển**: `dotnet dev-certs https --trust` (một lần) để trình duyệt tin chúng chỉ cục bộ; thêm profile `https` với `applicationUrl`: `"https://localhost:7102;http://localhost:5102"` . `Production` luôn dùng HTTPS thật (Chương 22 và Tập 3).
- Đổi URL không cần sửa file: `dotnet run --urls http://localhost:6000` hoặc biến `ASPNETCORE_URLS` .

Kestrel

Kestrel là máy chủ web tích hợp sẵn, đa nền tảng, rất nhanh; nhận kết nối TCP, phân tích HTTP/1.1, HTTP/2, HTTP/3 rồi chuyển vào pipeline của bạn. Khi triển khai thật thường đặt **sau một reverse proxy** (Nginx, IIS, YARP, load balancer đám mây, ingress Kubernetes) để lo TLS, nén, cân bằng tải (Tập 3). Không cần cài IIS/Apache khi phát triển.

Hot reload — sửa code không cần khởi động lại

```
dotnet watch run
```

Lưu file → ứng dụng tự nạp lại (nhiều thay đổi được áp dụng **ngay khi đang chạy** — hot reload; thay đổi lớn thì tự khởi động lại). Trong Visual Studio dùng **Hot Reload** (🔥) khi F5. Tiết kiệm rất nhiều thời gian thử-sửa.

Một endpoint thử nghiệm hữu ích

```
app.MapGet("/moi-truong", (IWebHostEnvironment env) => new
{
    env.EnvironmentName,
    env.ApplicationName,
    Cwd = Environment.CurrentDirectory,
});
```

Tham số `IWebHostEnvironment` được **tiêm** (Dependency Injection) vào handler — bạn khai báo cần gì, framework đưa cho. Đây là ý tưởng trung tâm của [ASP.NET](#) Core, Chương 5 sẽ đi sâu.

Lỗi thường gặp

- `Address already in use` — cổng đang bị chương trình khác/tiến trình cũ giữ; đổi `--urls` hoặc tắt tiến trình.
- Trình duyệt báo chứng chỉ không tin cậy: chạy `dotnet dev-certs https --trust`.
- Sửa code mà kết quả không đổi: đang chạy bản cũ; dừng rồi chạy lại hoặc dùng `dotnet watch`.
- `System.InvalidOperationException: Cannot modify ServiceCollection after application started` — gọi `builder.Services.Add...` sau `Build()`.
- Nhầm Development/Production: trang lỗi chỉ tiết chỉ hiện ở Development (an toàn — không lộ stack trace ra ngoài).
- Commit thư mục `bin/`, `obj/`, hoặc `launchSettings.json` chứa bí mật.

Bài tập

1. Tạo dự án `dotnet new web`, thêm ba endpoint: `/gio`, `/nhan-doi/{so:int}`, `/chao?ten=...`. Thử bằng `curl`.
2. Thêm endpoint `/phien-ban` trả về `Environment.Version` và `RuntimeInformation.OSDescription`.
3. Chạy bằng `dotnet run --urls http://localhost:7000` rồi đặt `ASPNETCORE_ENVIRONMENT=Production` và so sánh `/moi-truong`.
4. Bật `dotnet watch`, sửa chuỗi trả về và quan sát ứng dụng tự cập nhật.
5. Tạo thêm dự án `dotnet new webapi` và so sánh `Program.cs` của nó với dự án rỗng: template đã thêm những gì?

Chương 3 — Middleware và request pipeline

Mục tiêu học

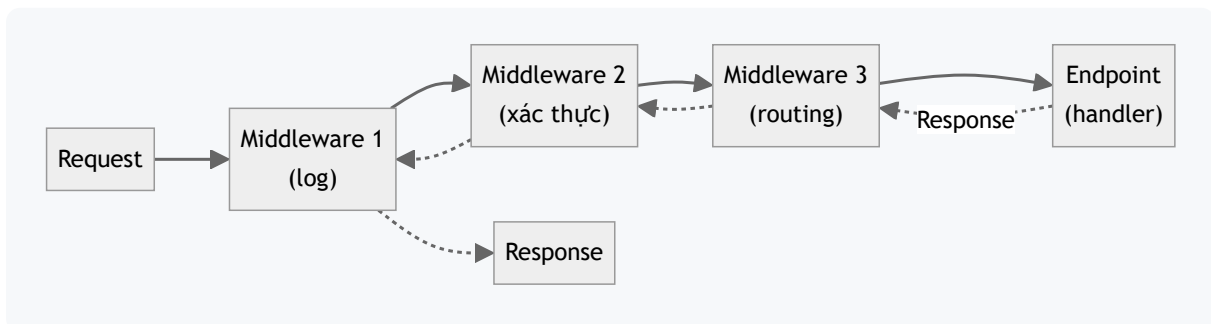
Sau chương này, bạn sẽ:

- Hiểu **request pipeline** là chuỗi middleware xử lý request/response theo dạng "củ hành".
- Viết middleware inline (`Use`), dạng class, nhánh (`Map` , `MapWhen`) và middleware **ngắn mạch**.
- Nắm **thứ tự** middleware chuẩn của [ASP.NET Core](#) và vì sao nó quan trọng.
- Biết chỗ đặt xử lý lỗi, HTTPS redirection, CORS, xác thực, static files.

Code mẫu: [code/ch03-middleware/](#) .

Pipeline: chuỗi củ hành

Mỗi request đi **vào** qua một dãy **middleware** (mỗi cái làm một việc), tới endpoint, rồi response đi **ngược ra** qua chính các middleware đó:



Mỗi middleware có thể: (1) làm gì đó **trước** khi gọi cái kế tiếp, (2) gọi `next` để chuyển tiếp, (3) làm gì đó **sau** khi các middleware phía sau hoàn tất, hoặc (4) **không** gọi `next` để kết thúc sớm (**ngắn mạch**). Đây là mẫu **Decorator/Chain of Responsibility** (Tập 1, Chương 41). Logging, xác thực, nén, CORS, xử lý lỗi... đều là middleware.

Middleware inline: `app.Use`

```
app.Use(async (ctx, next) =>
{
    app.Logger.LogInformation("[1] vào {Method} {Path}", ctx.Request.Method, ctx.Request.Path);
    await next(ctx); // chuyển sang middleware tiếp theo
    app.Logger.LogInformation("[1] ra -> {Status}", ctx.Response.StatusCode);
});
```

ctx là `HttpContext` — gói mọi thứ về request hiện tại: `Request`, `Response`, `User`, `RequestServices` (DI), `Items` (dữ liệu tạm trong request), `RequestAborted` ... Mã trước `await next(ctx)` chạy lúc đi vào; mã sau chạy lúc đi ra.

Middleware dạng class

Khi cần tái sử dụng hoặc có phụ thuộc:

```
class DoThoiGianMiddleware(RequestDelegate next, ILogger<DoThoiGianMiddleware> log)
{
    public async Task InvokeAsync(HttpContext ctx)
    {
        var dong = Stopwatch.StartNew();
        ctx.Response.OnStarting(() =>
        {
            ctx.Response.Headers["X-Thoi-Gian-Ms"] = dong.ElapsedMilliseconds.ToString();
            return Task.CompletedTask;
        });
        try { await next(ctx); }
        catch (Exception e)
        {
            log.LogError(e, "Loi khong xu ly tai {Path}", ctx.Request.Path);
            ctx.Response.StatusCode = 500;
            await ctx.Response.WriteAsync("Da co loi xay ra.");
        }
    }
}

app.UseMiddleware<DoThoiGianMiddleware>();
```

- Constructor nhận `RequestDelegate next` và các dịch vụ **Singleton**; `InvokeAsync(HttpContext, ...)` có thể nhận thêm dịch vụ **Scoped** làm tham số.
- `Response.OnStarting` : header chỉ thêm được **trước khi** response bắt đầu gửi; sau khi body đã ghi thì không sửa header/status nữa. Callback này chạy đúng lúc trước khi gửi.
- Middleware ở ngoài bắt được ngoại lệ từ mọi middleware/endpoint **phía trong** — nên xử lý lỗi toàn cục đặt **sớm** trong pipeline.

Thường bọc thành extension method (Tập 1, Chương 19): `app.UseDoThoiGian()` .

Ngắn mạch (short-circuit)

```
app.Use(async (ctx, next) =>
{
    if (ctx.Request.Path.StartsWithSegments("/bi-chan"))
```

```

{
    ctx.Response.StatusCode = StatusCodes.Status403Forbidden;
    await ctx.Response.WriteAsync("Bi chan boi middleware.");
    return; // KHÔNG gọi next → endpoint không bao giờ chạy
}
await next(ctx);
});

```

Ứng dụng: từ chối sớm (không xác thực, quá giới hạn tốc độ, IP bị chặn), trả file tĩnh, health check, trả từ bộ nhớ đệm.

Nhánh: Map, MapWhen, Run

```

app.MapWhen(ctx => ctx.Request.Query.ContainsKey("debug"), nhánh =>
    nhánh.Run(async ctx => await ctx.Response.WriteAsync("Che do debug")));

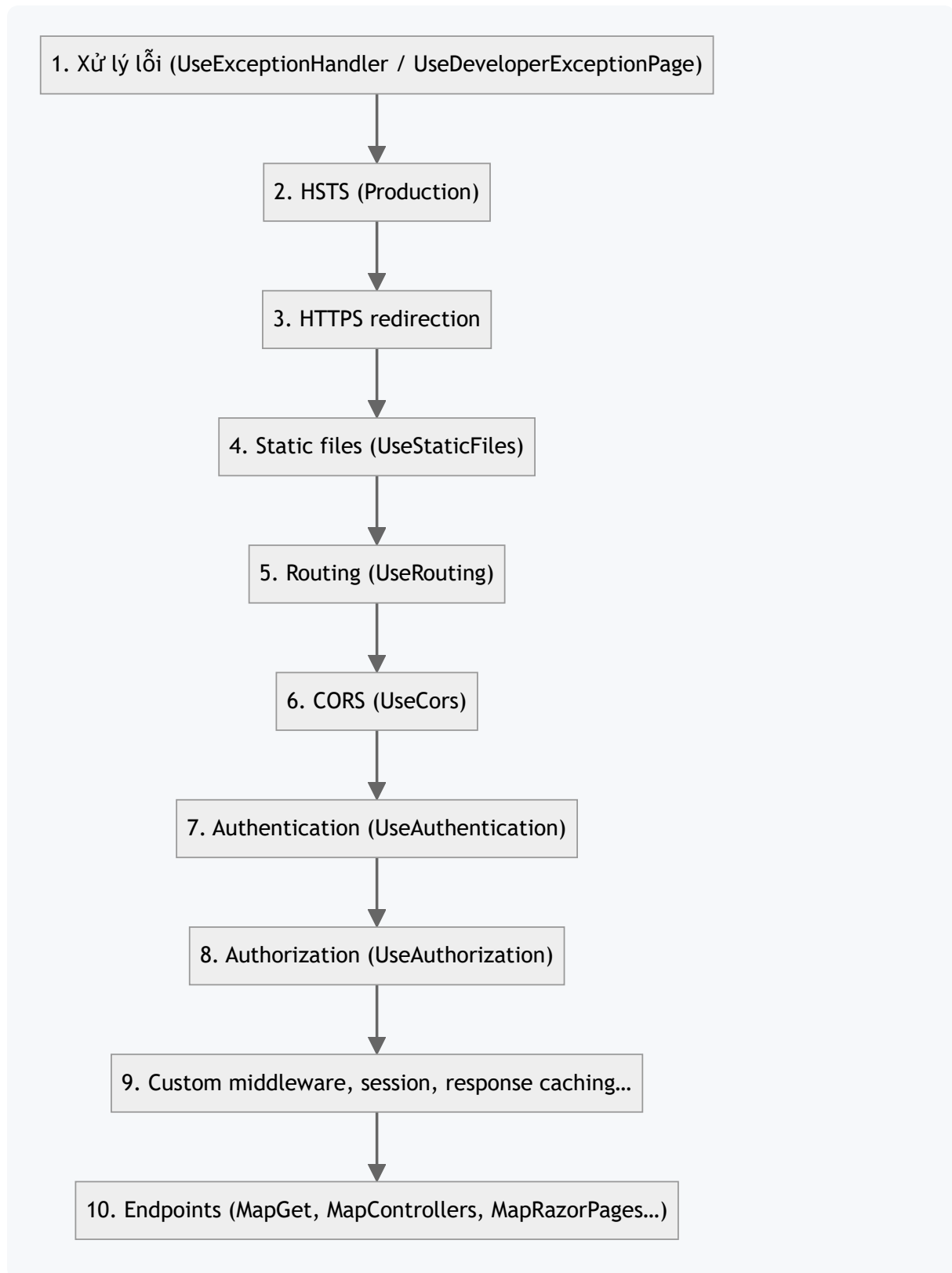
```

Phương thức	Ý nghĩa
<code>app.Use(...)</code>	middleware gọi tiếp next
<code>app.Run(...)</code>	middleware cuối/terminal — không có next
<code>app.Map("/duong-dan", nhánh => ...)</code>	rẽ nhánh theo tiền tố đường dẫn
<code>app.MapWhen(predicate, nhánh => ...)</code>	rẽ nhánh theo điều kiện tùy ý
<code>app.UseWhen(predicate, nhánh => ...)</code>	rẽ nhánh nhưng quay lại pipeline chính sau đó

Bắt quan trọng: trong ứng dụng dùng `MapGet/MapControllers`, **endpoint chạy ở cuối pipeline**. Nếu bạn đặt `app.Run(async ctx => ...)` trước các `MapGet`, middleware terminal đó sẽ chặn mọi request và endpoint **không bao giờ** được gọi (mọi URL đều rơi vào nó). Muốn xử lý "không khớp endpoint nào" hãy dùng `app.MapFallback(...)` như trong code mẫu. Chạy code mẫu và thử `/cham`, `/khong-co` để thấy.

Thứ tự middleware chuẩn

Thứ tự đăng ký = thứ tự thực thi, và nó có ý nghĩa. Thứ tự khuyến nghị của Microsoft:



Lý do: xử lý lỗi phải **bọc ngoài cùng** để bắt lỗi của mọi thứ bên trong; **static files** đặt trước xác thực để phục vụ file công khai nhanh, không tốn tài nguyên; **UseAuthentication phải đứng trước UseAuthorization** (biết bạn là ai rồi mới kiểm tra quyền) và cả hai sau **UseRouting**, trước endpoint (cần biết endpoint nào để đọc `[Authorize]`); CORS phải chạy trước xác thực để phản hồi preflight. Với `WebApplication`, routing và endpoints được thêm tự động, nhưng bạn vẫn phải gọi `UseAuthentication/UseAuthorization/UseCors` đúng vị trí.

Ví dụ pipeline điển hình:

```
var app = builder.Build();

if (app.Environment.IsDevelopment()) app.UseDeveloperExceptionPage();
else { app.UseExceptionHandler("/loi"); app.UseHsts(); }

app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseCors();
app.UseAuthentication();
app.UseAuthorization();

app.MapGet("/", () => "Trang chu");
app.Run();
```

Middleware có sẵn thường dùng

Middleware	Việc làm
UseExceptionHandler	bắt ngoại lệ, trả trang/JSON lỗi (Chương 9)
UseHttpsRedirection, UseHsts	ép dùng HTTPS
UseStaticFiles	phục vụ file trong wwwroot
UseRouting	chọn endpoint theo URL
UseCors	cho phép trình duyệt từ origin khác gọi API
UseAuthentication/Authorization	xác thực/phân quyền (Chương 20)
UseRateLimiter	giới hạn tốc độ (Chương 21)
UseResponseCompression, UseOutputCache	nén, cache đầu ra
UseRequestLocalization	đa ngôn ngữ
UseForwardedHeaders	đọc X-Forwarded-* khi sau reverse proxy

Endpoint filter và middleware — khác nhau thế nào?

Middleware áp dụng cho **mọi request** (trước khi biết endpoint) — hợp cho vấn đề xuyên suốt (log, lỗi, xác thực). **Filter** (endpoint filter/action filter, Chương 4 và 7) áp dụng cho **từng endpoint/nhóm endpoint** và biết tham số đã bind — hợp cho validation, xử lý kết quả theo endpoint.

Lỗi thường gặp

- **Sai thứ tự:** `UseAuthorization` trước `UseAuthentication` → luôn bị từ chối; `UseCors` sau xác thực → preflight bị 401.
- Đặt `app.Run(...)` terminal trước các endpoint → endpoint không chạy.
- Quên `await next(ctx)` → request treo; hoặc gọi `next` **hai lần**.
- Sửa header/status **sau khi** response đã bắt đầu: `InvalidOperationException: Headers are read-only, response has already started`. Dùng `OnStarting`.
- Ghi vào `Response` rồi vẫn gọi `next` (endpoint cũng ghi) → nội dung lộn xộn.
- Middleware nuốt ngoại lệ không log.
- Dùng dịch vụ Scoped trong constructor của middleware (Singleton) → lỗi; đưa vào tham số `InvokeAsync`.

Bài tập

1. Viết middleware `RequestIdMiddleware` thêm header `X-Request-Id` (GUID) vào mọi response và đưa vào `ctx.Items`.
2. Viết middleware chặn mọi request có header `User-Agent` chứa "bot" bằng `403`.
3. Viết middleware chỉ chạy cho đường dẫn `/api` bằng `UseWhen`, tính và log thời gian xử lý.
4. Cố tình đặt `UseAuthorization` trước `UseAuthentication` (ở chương xác thực) rồi giải thích triệu chứng.
5. Vẽ sơ đồ thứ tự thực thi (vào/ra) cho ba middleware A, B, C trong đó B ngắn mạch với một số request.

Chương 4 — Routing và Minimal API

Mục tiêu học

Sau chương này, bạn sẽ:

- Định nghĩa route với `MapGet/MapPost/MapPut/MapDelete`, route constraint và `MapGroup`.
- Hiểu **parameter binding**: route, query, header, body, dịch vụ (DI).
- Trả kết quả bằng `TypedResults` và khai báo rõ kiểu trả về (`Results<...>`).
- Xây một API **CRUD** hoàn chỉnh cho `Todo`, có validation và tài liệu OpenAPI.

Code mẫu: [code/ch04-minimal-api/](#).

Routing là gì?

Routing khớp **phương thức + đường dẫn** của request với một **endpoint** (handler). Trong pipeline (Chương 3), middleware routing chọn endpoint, các middleware sau đọc metadata của nó (vd `[Authorize]`), rồi endpoint chạy.

```
app.MapGet("/chao/{ten}", (string ten) => $"Xin chao, {ten}!");
app.MapPost("/todos", (TaoTodo req) => ...);
app.MapPut("/todos/{id}", ...);
app.MapDelete("/todos/{id}", ...);
app.MapMethods("/x", ["GET", "HEAD"], handler);
```

Tham số route và ràng buộc (constraint)

```
app.MapGet("/todos/{id:int}", (int id) => ...);           // chỉ khớp khi id là số nguy
app.MapGet("/nguoi-dung/{ten:alpha:minlength(3)}", ...);
app.MapGet("/file/{**duongDan}", (string duongDan) => ...); // "catch-all": bắt cả p
app.MapGet("/bai-viet/{slug?}", (string? slug) => ...);    // tham số tùy chọn
```

Constraint thường dùng: `int`, `long`, `guid`, `bool`, `datetime`, `alpha`, `min(1)`, `max(100)`, `range(1,10)`, `length(5)`, `regex(...)`. Không khớp → `404` (không tới handler).

Parameter binding

Minimal API tự suy ra nguồn dữ liệu của mỗi tham số handler:

Nguồn	Khi nào	Ví dụ
Route	tên khớp {tham-so} trong đường dẫn	int id
Query string	kiểu đơn giản, không có trong route	bool? done, string? tim
Body (JSON)	kiểu phức tạp (class/record)	TaoTodoRequest req
Header	[FromHeader]	[FromHeader(Name="X-Khach")] string? khach
Dịch vụ (DI)	kiểu đã đăng ký trong container	TodoStore kho, ILogger<Program> log
Đặc biệt	HttpContext, HttpRequest, ClaimsPrincipal, Cancellation token, IFormFile	

Có thể chỉ định tường minh: `[FromRoute]`, `[FromQuery]`, `[FromBody]`, `[FromServices]`, `[FromForm]`. Gom nhiều tham số query vào một kiểu bằng `[AsParameters]`:

```
record TimKiem(int Trang = 1, int KichThuoc = 10);
app.MapGet("/vi-du/{id:int}", (int id, [AsParameters] TimKiem tk) => new { id, tk.Trang
// GET /vi-du/7?trang=2&kichThuoc=5
```

Kiểu **nullable** (`bool? done`, `string? tim`) nghĩa là **tuỳ chọn**; kiểu không nullable mà thiếu thì `400 Bad Request` tự động. Body chỉ được có **một** tham số kiểu phức tạp.

Kết quả trả về

Trả `string`, `object` ($\rightarrow$ JSON) cho nhanh; hoặc `Results/TypedResults` để kiểm soát mã trạng thái:

```
TypedResults.Ok(todo) // 200 + JSON
TypedResults.Created($"api/todos/{id}", todo) // 201 + header Location
TypedResults.NoContent() // 204
TypedResults.NotFound() // 404
TypedResults.BadRequest("Ly do") // 400
TypedResults.ValidationProblem(loi) // 400 + dạng ProblemDetails (R
TypedResults.Conflict(), .Unauthorized(), .Forbidden(), .File(...), .Redirect(...), .Stre
```

Nên dùng **TypedResults** (kiểu cụ thể) và khai báo kiểu trả về là **Results<A, B, ...>** — trình biên dịch biết mọi trường hợp có thể xảy ra, OpenAPI tự sinh đúng, và test dễ (kiểm tra kiểu kết quả):

```
todos.MapGet("/{id:int}", Results<Ok<Todo>, NotFound> (int id, TodoStore kho)
    => kho.Tim(id) is { } t ? TypedResults.Ok(t) : TypedResults.NotFound())
    .WithName("LayTodo");
```

Đây là cách dùng pattern matching `is { } t` (Tập 1, Chương 32). Khi trả lỗi, luôn đi kèm **mã đúng**: `201` cho tạo mới, `204` cho cập nhật/xoá không body, `404` cho không tìm thấy, `400` cho dữ liệu sai.

MapGroup — nhóm endpoint

```
var todos = app.MapGroup("/api/todos").WithTags("Todos");
todos.MapGet("/", ...); // GET /api/todos
todos.MapGet("/{id:int}", ...); // GET /api/todos/5
todos.MapPost("/", ...); // POST /api/todos
```

Nhóm dùng chung **tiền tố**, **metadata** (`WithTags` , `RequireAuthorization`) , `RequireRateLimiting`), **filter** — đỡ lặp lại. Có thể lồng nhóm (`api.MapGroup("/v1")`).

CRUD hoàn chỉnh

```
// LIST + lọc: GET /api/todos?done=true&tim=hoc
todos.MapGet("/", (TodoStore kho, bool? done, string? tim) =>
{
    var ds = kho.TatCa().AsEnumerable();
    if (done is not null) ds = ds.Where(t => t.Done == done);
    if (!string.IsNullOrEmpty(tim)) ds = ds.Where(t => t.Title.Contains(tim, Stri
    return TypedResults.Ok(ds.ToList());
});

// CREATE: 201 + Location, hoặc 400 khi dữ liệu sai
todos.MapPost("/", Results<Created<Todo>, ValidationProblem> (TaoTodoRequest req, Todo
{
    var loi = req.KiemTra();
    if (loi.Count > 0) return TypedResults.ValidationProblem(loi);
    var moi = kho.Them(req.Title!.Trim());
    return TypedResults.Created($"{api}/todos/{moi.Id}", moi);
});

// UPDATE / DELETE: 204 hoặc 404
todos.MapPut("/{id:int}", ...);
todos.MapDelete("/{id:int}", ...);
```

Thử bằng `curl` :

```
curl -i -X POST http://localhost:5204/api/todos -H "Content-Type: application/json" -d '# HTTP/1.1 201 Created    Location: /api/todos/3'
```

```
curl -X POST http://localhost:5204/api/todos -H "Content-Type: application/json" -d '{# {"type":"https://tools.ietf.org/html/rfc9110#section-15.5.1","title":"One or more va
```

Body lỗi theo chuẩn **ProblemDetails** (RFC 9457) — cách thống nhất mô tả lỗi cho client (Chương 8, 9).

Dữ liệu dùng chung an toàn đa luồng

`TodoStore` đăng ký `AddSingleton` và được **hiều request chạy song song** gọi cùng lúc → mọi thao tác trên `List` bên trong phải **khóa** (`lock` , Tập 1, Chương 34). Ở Chương 12 ta thay bằng cơ sở dữ liệu (EF Core) — lúc đó CSDL lo tính đúng đắn. Quy tắc nhớ: **singleton phải thread-safe**.

Endpoint filter

Filter cho chạy code **trước/sau handler**, đã biết tham số:

```
todos.MapPost("/", handler).AddEndpointFilter(async (ctx, next) =>
{
    var req = ctx.GetArgument<TaoTodoRequest>(0);
    if (string.IsNullOrEmpty(req.Title))
        return TypedResults.ValidationProblem(new Dictionary<string, string[]> { ["tit
    return await next(ctx);    // hợp lệ → chạy handler
});
```

Dùng cho validation dùng chung, log theo endpoint, chuẩn hoá kết quả. Áp cho cả nhóm:

```
todos.AddEndpointFilter(...).
```

OpenAPI — tài liệu API tự sinh

```
builder.Services.AddOpenApi(); // package Microsoft
if (app.Environment.IsDevelopment()) app.MapOpenApi(); // GET /openapi/v1.j
```

Tài liệu mô tả mọi endpoint, tham số, kiểu request/response, mã trạng thái (lấy từ `Results<...>`). Các công cụ như **Swagger UI**, **Scalar**, Postman đọc file này để dựng giao diện thử API và sinh code client. Chi tiết ở Chương 9.

Khi nào Minimal API, khi nào Controller?

	Minimal API	Controller (Chương 7)
Độ dài code	rất ngắn	nhiều "nghỉ thức" hơn
Tổ chức	tự do (nhóm, file riêng, class static)	quy ước rõ ràng, class theo tài nguyên
Tính năng nâng cao (filters, model binding tùy biến, versioning sẵn)	có, đang hoàn thiện	đầy đủ, lâu đời
Hợp với	microservice, API nhỏ—vừa, dự án mới	API lớn, đội quen MVC, cần nhiều convention

Hai phong cách cùng nền tảng, có thể **dùng chung trong một dự án**. Khi dự án lớn, đừng nhét mọi thứ vào `Program.cs` : tách mỗi tài nguyên thành một **extension method** `app.MapTodoEndpoints()` trong file riêng.

Lỗi thường gặp

- `400` do quên `Content-Type: application/json` hoặc JSON sai → kiểm tra body/header.
- Handler có hai tham số phức tạp → lỗi khi khởi động ("cannot infer body"); gộp thành một kiểu hoặc chỉ định `[FromServices]` .
- Route bị xung đột (`AmbiguousMatchException`) — hai endpoint khớp cùng URL; thu hẹp bằng constraint/đổi URL.
- Quên constraint `:int` → `/todos/abc` rơi vào handler và lỗi bind.
- Trả `200` kèm `null` cho "không tìm thấy" thay vì `404` .
- Singleton giữ dữ liệu mà không khoá / không thread-safe.
- Đặt logic nghiệp vụ dài dằng dặc ngay trong lambda — hãy tách vào dịch vụ (Chương 5).

Bài tập

1. Thêm `PATCH /api/todos/{id}/hoan-thanh` đánh dấu xong; trả `204` hoặc `404` .
2. Thêm phân trang `GET /api/todos?trang=1&kichThuoc=10` dùng `[AsParameters]` và trả header `X-Tong-So` .
3. Thêm route constraint để `GET /api/todos/{id:int:min(1)}` từ chối `0` , `-5` .
4. Viết endpoint filter chung cho nhóm `/api/todos` bắt buộc header `X-API-Key` , trả `401` nếu thiếu.
5. Tách các endpoint Todo thành extension method `MapTodoEndpoints` trong file `TodoEndpoints.cs` .

Chương 5 — Dependency Injection

Mục tiêu học

Sau chương này, bạn sẽ:

- Hiểu **DI container** của [ASP.NET](#) Core và vì sao mọi thứ đều "được tiêm".
- Đăng ký dịch vụ với ba vòng đời: **Transient**, **Scoped**, **Singleton** — và chọn đúng.
- Tránh lỗi kinh điển: **captive dependency**, dùng dịch vụ Scoped trong Singleton.
- Dùng đăng ký nhiều cài đặt, **keyed services**, factory.

Code mẫu: [code/ch05-dependency-injection/](#).

Từ nguyên lý đến công cụ

Tập 1, Chương 42 giới thiệu **Dependency Inversion**: lớp nghiệp vụ nhận phụ thuộc qua **constructor dưới dạng interface** thay vì tự `new`. Ai sẽ **lắp ráp** các đối tượng đó? Ở Tập 1 ta làm tay ở *composition root* (`Program.cs`). Khi ứng dụng có hàng trăm lớp phụ thuộc nhau nhiều tầng, việc đó tự động hoá bằng **DI container** (IoC container) có sẵn trong [ASP.NET](#) Core: `builder.Services`.

```
// (1) ĐĂNG KÝ (trước Build): "khi ai xin IGuiThongBao, đưa GuiEmail"
builder.Services.AddScoped<IGuiThongBao, GuiEmail>();

// (2) SỬ DỤNG: khai báo cần gì, container tạo và đưa cho
app.MapGet("/gui", (IGuiThongBao gui) => gui.Gui("xin chào"));

class DatHangService(IGuiThongBao gui, IKhoHang kho) { ... } // tiêm qua constructo
```

Container xây **đồ thị phụ thuộc** tự động: xin `DatHangService` → tự tạo `GuiEmail` và `KhoHang` trước (đệ quy). Bạn viết code gắn kết lỏng (loose coupling); đổi cài đặt (email → SMS, CSDL thật → giả khi test) chỉ sửa **một dòng đăng ký**.

Ba vòng đời (lifetime)

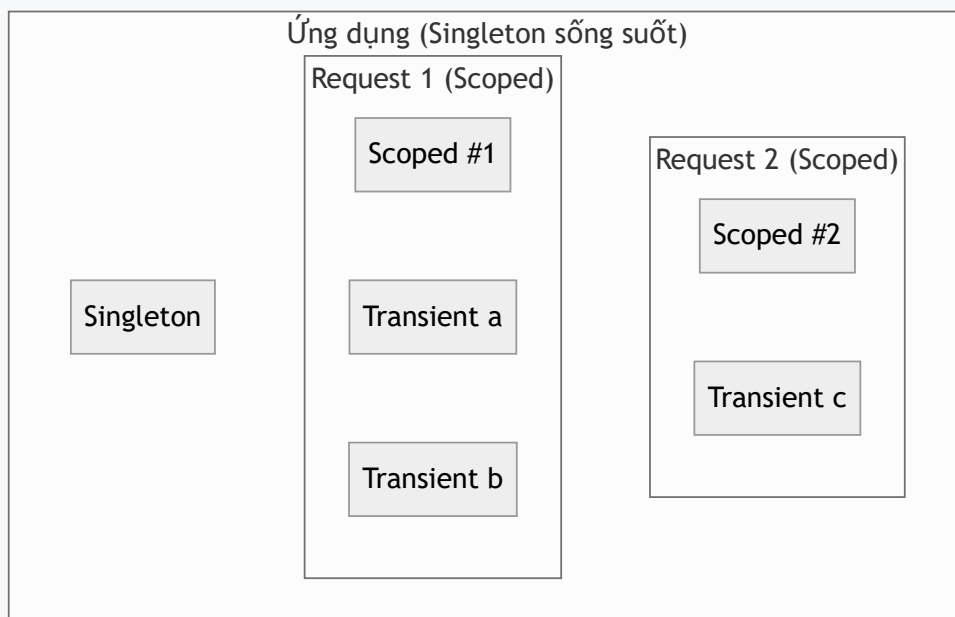
Vòng đời	Đăng ký	Đối tượng được tạo...	Dùng cho
Transient	<code>AddTransient</code>	mỗi lần được xin (mỗi lần tiêm là một bản mới)	dịch vụ nhẹ, không giữ trạng thái

Vòng đời	Đăng ký	Đối tượng được tạo...	Dùng cho
Scoped	AddScoped	một lần cho mỗi scope — trong web, mỗi HTTP request	dịch vụ gắn với request: DbContext, unit of work, user hiện tại
Singleton	AddSingleton	một lần duy nhất cho cả vòng đời ứng dụng	cấu hình, cache, HttpClient factory, dịch vụ không trạng thái/thread-safe

Chạy code mẫu và gọi `/thoi-gian` hai lần để **thấy tận mắt**:

```
{
  "transient": { "lan1": "7c8b...", "lan2": "9051...", "giongNhu": false },
  "scoped": { "lan1": "c7c6...", "lan2": "c7c6...", "giongNhu": true, "dungChungVo": true },
  "singleton": { "lan1": "93aa...", "lan2": "93aa...", "giongNhu": true }
}
```

- **Transient**: hai lần xin cùng một request → hai GUID khác nhau.
- **Scoped**: trong *một* request, mọi nơi (endpoint, `DichVuA`, `DichVuB`) thấy **cùng một** đối tượng; sang request mới → GUID khác.
- **Singleton**: mọi request đều cùng một GUID (thấy ở lần gọi thứ hai vẫn `93aa...`).



Quy tắc chọn vòng đời

1. **Mặc định** `Scoped` cho dịch vụ nghiệp vụ trong web (an toàn, gắn theo request).
2. `Singleton` cho thứ tốn kém khi tạo hoặc cần chia sẻ: cache, cấu hình, bộ đếm — **bắt buộc thread-safe** (nhiều request cùng lúc — Tập 1, Chương 34).

3. **Transient** cho dịch vụ nhỏ, không trạng thái; nếu cài **IDisposable** thì container sẽ giữ tham chiếu để dispose cuối scope.
4. **DbContext** **luôn Scoped** (Chương 12) — không thread-safe.

Captive dependency — lỗi kinh điển

Một dịch vụ **sống lâu** không được phụ thuộc dịch vụ **sống ngắn hơn**:

```
builder.Services.AddSingleton<DichVuToanCuc>(); // sống mãi
builder.Services.AddScoped<AppDbContext>();    // sống một request

class DichVuToanCuc(AppDbContext db) { }        // X Singleton "giam" DbContext của
```

Kết quả: **DbContext** bị dùng lại giữa nhiều request/luồng → lỗi lạ, dữ liệu cũ, đa luồng hỏng. Ở môi trường **Development**, [ASP.NET](#) Core bật **ValidateScopes** và ném lỗi ngay:

```
Cannot consume scoped service 'AppDbContext' from singleton 'DichVuToanCuc'.
```

Cách sửa: hạ Singleton xuống Scoped; hoặc Singleton tự tạo scope khi cần bằng

IServiceScopeFactory :

```
class TacVuNen(IServiceScopeFactory factory)
{
    public async Task ChayAsync()
    {
        using var scope = factory.CreateScope();
        var db = scope.ServiceProvider.GetRequiredService<AppDbContext>(); // Scoped
        ...
    }
}
```

Mẫu này bắt buộc khi dùng dịch vụ Scoped trong **BackgroundService** (Tập 3).

Cách đăng ký

```
services.AddScoped<IX, X>(); // interface → cài đặt (phổ biến nhất)
services.AddScoped<X>();    // chỉ lớp cụ thể
services.AddSingleton<IX>(new X("cau-hinh")); // đối tượng có sẵn
services.AddScoped<IX>(sp => new X(sp.GetRequiredService<Y>())); // factory tùy biến
services.AddSingleton<IX, X>().AddSingleton<IY, Y>();
services.TryAddScoped<IX, X>(); // chỉ thêm nếu chưa đăng ký (dùng k
```

Nhiều cài đặt cho một interface

```
builder.Services.AddSingleton<IQuyTacGiamGia, GiamCuoiTuan>();
builder.Services.AddSingleton<IQuyTacGiamGia, GiamDonLon>();

app.MapGet("/giam-gia", (decimal tien, IEnumerable<IQuyTacGiamGia> quyTac) => ...);
```

Xin `IEnumerable<IQuyTacGiamGia>` nhận **tất cả** theo thứ tự đăng ký; xin `IQuyTacGiamGia` (một) nhận cái **đăng ký sau cùng**. Đây chính là **Strategy** (Tập 1, Chương 41) do container tổ chức: thêm quy tắc mới = thêm một dòng đăng ký, không sửa nơi dùng (Open/Closed).

Keyed services (.NET 8+)

Chọn cài đặt theo khoá:

```
builder.Services.AddKeyedSingleton<IGuiThongBao, GuiEmail>("email");
builder.Services.AddKeyedSingleton<IGuiThongBao, GuiSms>("sms");

app.MapGet("/x", ([FromKeyedServices("sms")] IGuiThongBao gui) => gui.Gui("hi"));
class XuLy([FromKeyedServices("email")] IGuiThongBao gui) { }
```

Hoặc dùng factory `Func<string, IGuiThongBao>` (code mẫu) khi khoá chỉ biết lúc chạy.

Đăng ký gọn bằng extension method

Dự án lớn có hàng chục dịch vụ. Gom theo tính năng:

```
public static class DangKyDichVu
{
    public static IServiceCollection AddDonHang(this IServiceCollection services)
    {
        services.AddScoped<IDonHangService, DonHangService>();
        services.AddScoped<IKhoHang, KhoHang>();
        return services;
    }
}

builder.Services.AddDonHang(); // Program.cs gọn
```

Đó cũng là cách các thư viện tự tích hợp (`AddDbContext` , `AddAuthentication` , `AddOpenApi` ...).

Dùng DI ở nhiều nơi

- **Minimal API**: tham số handler. **Controller/Razor Pages/Blazor**: constructor (primary constructor của C# 12 rất gọn).

- **Middleware:** constructor (Singleton) / `InvokeAsync` (Scoped).
- **Lấy thủ công:** `HttpContext.RequestServices.GetRequiredService<T>()` hay `IServiceProvider` — dùng **rất hạn chế**. Việc rải `GetService<T>()` khắp nơi gọi là **Service Locator** — phản mẫu: che giấu phụ thuộc, khó test.
- **Kiểm thử:** dựng đối tượng bằng tay với bản giả (Tập 1, Chương 38); hoặc thay đăng ký trong `WebApplicationFactory` (Chương 10).

Container có sẵn không làm gì?

Container mặc định cố ý **đơn giản**: không có property injection, interceptor, đăng ký theo quy ước tự động (scan assembly), decorator sẵn. Cần những thứ đó có thể dùng thư viện **Scrutor** (scan + decorate) hoặc container ngoài (Autofac) — nhưng hầu hết dự án chỉ cần bộ mặc định.

Lỗi thường gặp

- `InvalidOperationException: Unable to resolve service for type 'X'` — **quên đăng ký** (hoặc đăng ký sai interface).
- `Cannot consume scoped service from singleton` — captive dependency (xem trên).
- **Vòng phụ thuộc:** A cần B, B cần A → `A circular dependency was detected`. Thiết kế lại (tách lớp thứ ba, dùng sự kiện).
- Dùng `new` bên trong lớp nghiệp vụ, bỏ qua container → không thay được khi test.
- Singleton giữ trạng thái người dùng/request (ví dụ lưu "người dùng hiện tại" vào field) → lẫn dữ liệu giữa các request.
- Constructor nhận quá nhiều (7–8+) phụ thuộc → lớp làm quá nhiều việc (vi phạm SRP).
- Quên rằng `Transient` `IDisposable` không được dispose nếu tạo bằng `new` ngoài container.

Bài tập

1. Thêm dịch vụ `IDemTruyCap` (Singleton) đếm số request bằng `Interlocked` và endpoint `/dem` trả số đếm.
2. Đăng ký `IGuiThongBao` thứ ba (`GuiZalo`); thêm vào factory và kiểm tra bằng `/gui?kenh=zalo`.
3. Cố tình tạo captive dependency (Singleton dùng Scoped) và đọc thông báo lỗi ở Development; sửa bằng `IServiceScopeFactory`.
4. Gom các dịch vụ chương này thành `services.AddNghiepVu()` trong file `DangKyDichVu.cs`.
5. Viết unit test cho `DichVuA` bằng cách tạo tay `IThoiGianScoped` giả, không dùng container.

Chương 6 — Cấu hình, Options và Logging

Mục tiêu học

Sau chương này, bạn sẽ:

- Hiểu cách [ASP.NET](#) Core **nạp cấu hình** từ nhiều nguồn và thứ tự ưu tiên.
- Dùng **Options pattern** (`IOptions` , `IOptionsSnapshot` , `IOptionsMonitor`) và **kiểm tra cấu hình lúc khởi động**.
- Cấu hình theo **môi trường**, giữ **bí mật** đúng cách.
- Ghi log có cấu trúc, cấp độ, scope, **source-generated logging**.

Code mẫu: [code/ch06-cau-hinh-logging/](#) .

Chương này áp dụng vào web những gì Tập 1 (Chương 39–40) đã giới thiệu — `WebApplication.CreateBuilder` đã **cấu hình sẵn** phần lớn cho bạn.

Nguồn cấu hình mặc định

`WebApplication.CreateBuilder(args)` nạp theo thứ tự (**nguồn sau ghi đè nguồn trước**):

1. `appsettings.json`
2. `appsettings.{Environment}.json` (vd `appsettings.Development.json`)
3. **User Secrets** (chỉ ở `Development`)
4. **Biến môi trường**
5. **Tham số dòng lệnh**

Hợp nhất thành một cây khoá–giá trị (`builder.Configuration` , kiểu `IConfiguration`). Khoá lồng nhau dùng dấu `:` ; biến môi trường dùng `__` :

```
CuaHang:Ten           # trong JSON
CuaHang__Ten="Sach Do" # biến môi trường
--CuaHang:Ten="Sach Xanh" # dòng lệnh
```

`appsettings.json` của code mẫu:

```
{
  "Logging": { "LogLevel": { "Default": "Information", "Microsoft.AspNetCore": "Warnin
  "CuaHang": { "Ten": "Sach Xanh", "ThueVat": 0.08, "PhiVanChuyenToiThieu": 15000, "Ph
  "ConnectionStrings": { "Mac_Dinh": "Data Source=app.db" }
}
```

appsettings.Development.json chỉ ghi **phần khác biệt** (đổi tên cửa hàng thành "(DEV)", log Debug).

Đọc cấu hình

```
builder.Configuration["CuaHang:Ten"] // chuỗi thô
builder.Configuration.GetValue<int>("CuaHang:SoNgay", 7) // có kiểu + mặc định
builder.Configuration.GetConnectionString("Mac_Dinh") // "ConnectionStrings:Ma
builder.Configuration.GetSection("CuaHang").Get<CuaHangOptions>()
```

Đọc chuỗi khoá rải rác rất dễ sai chính tả (trả null mà không báo lỗi). **Ưu tiên Options pattern.**

Options pattern

Gom cấu hình liên quan vào một class, gắn vào một mục, thêm vào nơi cần:

```
class CuaHangOptions
{
    public const string TenMuc = "CuaHang";

    [Required, MinLength(2)] public string Ten { get; set; } = "";
    [Range(0, 1)] public decimal ThueVat { get; set; }
    [Range(0, 1_000_000)] public decimal PhiVanChuyenToiThieu { get; set; }
    [Range(0, 1_000_000)] public decimal PhiVanChuyenToiDa { get; set; }
    public List<string> KenhHoTro { get; set; } = [];
}

builder.Services
    .AddOptions<CuaHangOptions>()
    .Bind(builder.Configuration.GetSection(CuaHangOptions.TenMuc))
    .ValidateDataAnnotations() //
    .Validate(o => o.PhiVanChuyenToiThieu <= o.PhiVanChuyenToiDa, "Phi toi thieu khong /
    .ValidateOnStart();

app.MapGet("/cau-hinh", (IOptions<CuaHangOptions> opt) => opt.Value);
```

ValidateOnStart() là chi tiết rất đáng giá: thử chạy với CuaHang__ThueVat=5 , ứng dụng **từ chối khởi động** với thông báo rõ ràng:

```
OptionsValidationException: DataAnnotation validation failed for 'CuaHangOptions' memb
```

Sự cố cấu hình lộ ra **lúc deploy**, không phải lúc một khách hàng đầu tiên gọi vào tính năng đó lúc 3 giờ sáng ("fail fast").

Ba dạng IOptions

Kiểu	Vòng đời	Nạp lại khi file đổi?	Dùng khi
IOptions<T>	Singleton	Không (đọc một lần)	cấu hình cố định suốt đời ứng dụng
IOptionsSnapshot<T>	Scoped	Có, mỗi request đọc lại	cấu hình đổi được, cần ổn định trong một request
IOptionsMonitor<T>	Singleton	Có, theo dõi thay đổi (OnChange)	dịch vụ Singleton cần giá trị mới nhất

`appsettings.json` được nạp với `reloadOnChange: true`, nên đổi file khi đang chạy → `IOptionsSnapshot/IOptionsMonitor` thấy giá trị mới (ví dụ: bật/tắt cờ tính năng mà không khởi động lại).

Môi trường

```
app.Environment.IsDevelopment(); IsStaging(); IsProduction(); EnvironmentName
```

Đặt bằng `ASPNETCORE_ENVIRONMENT`. Dùng `if (env.IsDevelopment())` cho những thứ **chỉ dev** (trang lỗi chi tiết, OpenAPI/Swagger UI, seed dữ liệu). Đừng dùng môi trường làm "công tắc nghiệp vụ" — khác biệt hành vi nghiệp vụ nên là **cấu hình**, không phải rẽ nhánh theo tên môi trường.

Bí mật (secrets)

Nguyên tắc từ Tập 1, Chương 40, giờ áp dụng vào web:

- **Không** để mật khẩu/khoá API/chuỗi kết nối có mật khẩu trong `appsettings.json` được commit.
- **Dev:** `dotnet user-secrets` (lưu ngoài repo).
- **Production:** biến môi trường do nền tảng cấp, hoặc kho bí mật (Azure Key Vault, AWS Secrets Manager, Docker/Kubernetes secrets).
- Không log giá trị bí mật; không trả chúng qua endpoint (code mẫu chỉ in "đã cấu hình" thay vì nội dung chuỗi kết nối).

Logging

`ILogger<T>` được tiêm sẵn; `T` (thường là chính lớp đang dùng) trở thành **category** của dòng log:

```
app.MapGet("/log", (ILogger<Program> log, string? nguoiDung) =>
{
    log.LogDebug("Debug: nguoi dung {NguoiDung}", nguoiDung);
    log.LogInformation("Information: nguoi dung {NguoiDung} vừa gọi /log", nguoiDung);
    log.LogWarning("Warning: canh bao mau");
    log.LogError(new InvalidOperationException("loi mau"), "Error: mot loi minh hoa");
    return "ok";
});
```

Kết quả console (ở `Development`, mức `Debug`):

```
dbug: Program[0]
      Debug: nguoi dung an
info: Program[0]
      Information: nguoi dung an vừa gọi /log
warn: Program[0]
      Warning: canh bao mau
fail: Program[0]
      Error: mot loi minh hoa
      System.InvalidOperationException: loi mau
```

Nguyên tắc (nhắc lại Tập 1, Chương 39): dùng **tham số đặt tên** `{NguoiDung}` chứ không nội suy `$"..."`; truyền `Exception` làm tham số **đầu** của `LogError`; không log dữ liệu nhạy cảm.

Cấp độ và lọc theo category

Cấu hình trong `appsettings.json`, **không cần sửa code**:

```
"Logging": { "LogLevel": {
  "Default": "Information",
  "Microsoft.AspNetCore": "Warning", // framework "ồn ào": chỉ Warn
  "Microsoft.EntityFrameworkCore.Database.Command": "Information", // xem câu SQL
  "MyApp.DonHang": "Debug"
} }
```

Đặt thấp hơn cho vùng đang điều tra, cao hơn cho vùng ồn — giảm log rác trên production.

Scope

```
using (log.BeginScope("CorrelationId={CorrelationId}", id))
{
```

```
log.LogInformation("Bat dau");    // mọi dòng trong khối đều kèm CorrelationId
}
```

Console mặc định **không hiển thị scope**; bật bằng `builder.Logging.AddSimpleConsole(o => o.IncludeScopes = true)`. Framework tự tạo scope cho mỗi request (có `RequestId`, `TraceId`) — công cụ log tập trung dùng để nối các dòng thuộc cùng một request.

Source-generated logging

```
static partial class NhatKy
{
    [LoggerMessage(EventId = 1001, Level = LogLevel.Information,
        Message = "Dat hang thanh cong: don {SoDon}, tong {TongTien}")]
    public static partial void DatHangThanhCong(this ILogger logger, int soDon, decimal tong)
    {
        log.DatHangThanhCong(42, 250_000m);
    }
}
```

Trình sinh mã tạo phương thức: **nhANH hơn** (không cấp phát/boxing khi mức log tắt), **an toàn kiểu** (tham số sai là lỗi biên dịch), mẫu thông điệp tập trung. Khuyến nghị cho log ở đường nóng và cho log nghiệp vụ quan trọng.

Log ra đâu?

Mặc định: console, debug, event source. Cho production dùng **Serilog** hoặc NLog (cắm vào `ILogger`, ghi file/Seq/Elasticsearch) hoặc **OpenTelemetry** để gửi log/trace/metric tới hệ thống quan sát (Tập 3). Trong container, cứ ghi ra **console (stdout)** — nền tảng sẽ thu thập.

Lỗi thường gặp

- Cấu hình đọc ra `null` /mặc định vì gõ sai khoá hoặc sai cấp lồng — dùng `Options + ValidateOnStart`.
- Tường `IOptions<T>` tự nạp lại khi file đổi (phải `IOptionsSnapshot` / `IOptionsMonitor`).
- Dùng `IOptionsSnapshot` (Scoped) trong dịch vụ Singleton → lỗi captive dependency.
- Commit `appsettings.Production.json` có mật khẩu.
- Biến môi trường dùng `:` thay `__` (một số shell/OS không cho `:`).
- Log bằng nội suy chuỗi, log mật khẩu/token, log mỗi vòng lặp ở mức `Information`.
- Để `Default: Debug/Trace` trên production → tràn dung lượng, lộ dữ liệu.

Bài tập

1. Thêm mục `Email` (`MayChu`, `Cong`, `TenDangNhap`) vào cấu hình, bind vào `EmailOptions` với kiểm tra `[Required]`, `[Range(1, 65535)]` và `ValidateOnStart`.

2. Ghi đè `CuaHang:Ten` bằng biến môi trường và bằng tham số dòng lệnh; xác nhận thứ tự ưu tiên.
3. Đặt `Microsoft.AspNetCore` ở `Information` và xem log của từng request; sau đó đặt lại `Warning`.
4. Viết `[LoggerMessage]` cho ba sự kiện nghiệp vụ (`DonHangTao` , `KhoSapHet` , `ThanhToanLoi`) với `EventId` khác nhau.
5. Dùng `IOptionsMonitor<T>.OnChange` để in log mỗi khi bạn sửa `appsettings.json` khi ứng dụng đang chạy.

Chương 7 — Web API với Controller

Mục tiêu học

Sau chương này, bạn sẽ:

- Viết Web API theo kiểu **controller**: `ControllerBase`, `[ApiController]`, `[Route]`, `[HttpGet]` ...
- Hiểu vòng đời request qua **MVC pipeline**: routing → model binding → validation → action → result.
- Trả kết quả đúng chuẩn: `ActionResult<T>`, `CreatedAtAction`, `NoContent`, `NotFound`.
- Tách nghiệp vụ khỏi controller (controller **mỏng**), nhận dịch vụ qua **DI**.

Code mẫu: [code/ch07-controller-api/](#) — bản hiện đại hoá của `CustomersController` và `UserController` mà repo này bắt đầu từ đó.

Controller hay Minimal API?

Chương 4 đã viết API bằng Minimal API. **Controller** là kiểu truyền thống (từ [ASP.NET MVC](#)), vẫn được dùng rất rộng rãi — bạn sẽ gặp nó trong hầu hết code có sẵn. Cùng nền tảng (routing, DI, middleware, JSON), khác cách tổ chức:

- Mỗi **controller** là một class gom các **action** (phương thức) của một tài nguyên.
- Nhiều **quy ước** có sẵn: lấy tên route từ tên class, tự validation model, filter theo nhóm, `[Authorize]` trên class...

Chọn cái nào ở cuối Chương 4; điều quan trọng là **hiểu cả hai**.

Cài đặt

```
builder.Services.AddControllers(); // đăng ký MVC controllers
builder.Services.AddRouting(o => o.LowercaseUrls = true); // URL sinh ra viết thường
...
app.MapControllers(); // gắn [Route] của controller vào
```

Controller đầu tiên

Ta viết lại `CustomersController` gốc của repo:

```
[ApiController]
[Route("api/[controller]")]
```

```
[Produces("application/json")]
public class CustomersController(ICustomerService customers) : ControllerBase
{
    [HttpGet]
    public async Task<ActionResult<IEnumerable<Customer>>> GetCustomers([FromQuery] string search, CancellationToken ct)
        => Ok(await customers.GetAllAsync(search, ct));

    [HttpGet("{id:int}")]
    public async Task<ActionResult<Customer>> GetCustomerById(int id, CancellationToken ct)
    {
        var customer = await customers.GetByIdAsync(id, ct);
        return customer is null ? NotFound() : Ok(customer);
    }

    [HttpPost]
    public async Task<ActionResult<Customer>> CreateCustomer(Customer customer, CancellationToken ct)
    {
        var created = await customers.CreateAsync(customer, ct);
        return CreatedAtAction(nameof(GetCustomerById), new { id = created.Id }, created);
    }

    [HttpPut("{id:int}")]
    public async Task<IActionResult> UpdateCustomer(int id, Customer customer, CancellationToken ct)
        => await customers.UpdateAsync(id, customer, ct) ? NoContent() : NotFound();

    [HttpDelete("{id:int}")]
    public async Task<IActionResult> DeleteCustomer(int id, CancellationToken ct)
        => await customers.DeleteAsync(id, ct) ? NoContent() : NotFound();
}
```

So với bản gốc: ba chỗ đã được **thay đổi có chủ đích**:

1. Dữ liệu không còn "giả cứng" trong action mà nằm trong **dịch vụ** `ICustomerService` được tiêm qua constructor (**primary constructor** C# 12) — controller chỉ lo HTTP.
2. Action **bất đồng bộ** (`async Task`) và nhận `CancellationToken` — khi client ngắt kết nối, việc dở dang được huỷ (Tập 1, Chương 33).
3. Route `{id:int}` có **constraint**; `PUT / DELETE` trả `404` khi không có, thay vì luôn `204` .

Các thành phần cần hiểu

ControllerBase và Controller

`ControllerBase` cho API (không có view). `Controller` (kế thừa `ControllerBase`) thêm hỗ trợ **View** cho MVC trả HTML (Chương 15). Web API dùng `ControllerBase` .

[ApiController]

Bật cả bộ hành vi tiện cho API:

- **Tự trả 400** với `ValidationProblemDetails` khi model không hợp lệ (không cần tự viết `if (!ModelState.IsValid)`).
- **Suy ra nguồn binding**: kiểu phức tạp → `[FromBody]` ; tham số trùng tên route → `[FromRoute]` ; kiểu đơn giản còn lại → `[FromQuery]` .
- Bắt buộc dùng **attribute routing**; trả lỗi `4xx` dạng `ProblemDetails`.
- Body sai JSON → `400` .

Route

```
[Route("api/[controller]")]           // [controller] = tên class bỏ hậu tố "Controller"
[HttpGet("{id:int}")]                 // ghép: GET /api/customers/{id}
[HttpPost]                             // POST /api/customers
[HttpGet("~/thong-ke")]               // "~/ " bỏ tiền tố controller: GET /thong-ke
```

Các token: `[controller]` , `[action]` , `[area]` . Nhiều dự án đặt tên tường minh (`[Route("api/khach-hang")]`) để URL không phụ thuộc tên class (đổi tên class → vỡ URL).

Model binding — dữ liệu request → tham số

Attribute	Nguồn
<code>[FromRoute]</code>	phần <code>{id}</code> trong URL
<code>[FromQuery]</code>	query string
<code>[FromBody]</code>	body (JSON) — chỉ một tham số
<code>[FromHeader]</code>	header
<code>[FromForm]</code>	form/multipart (upload file: <code>IFormFile</code>)
<code>[FromServices]</code>	DI

Binding **lỗi kiểu** (`/api/customers/abc` , hay số truyền chuỗi) → `400` hoặc `404` (nếu vướng constraint). Trong bản mẫu, `GET /api/customers/abc` trả `404` vì constraint `:int` không khớp.

Kiểu trả về

Kiểu	Khi nào
<code>ActionResult<T></code>	khuyến nghị : trả T (200) hoặc kết quả lỗi; OpenAPI biết kiểu T
<code>IActionResult</code>	khi không có body/nhiều dạng khác nhau

Kiểu	Khi nào
T hoặc Task<T>	luôn 200 (không nên nếu có thể lỗi)
IAsyncEnumerable<T>	trả luồng lớn

Helper (kế thừa từ ControllerBase): `Ok()`, `Created()`, `CreatedAtAction(...)`, `NoContent()`, `BadRequest()`, `NotFound()`, `Conflict()`, `Unauthorized()`, `Forbid()`, `ValidationProblem()`, `Problem()`, `File()`, `Redirect()`.

`CreatedAtAction(nameof(GetCustomerById), new { id = created.Id }, created)` trả 201 Created **kèm header Location** trỏ tới action lấy đối tượng mới:

```
HTTP/1.1 201 Created
Location: http://localhost:5207/api/customers/4
{"id":4,"name":"Alice","email":"a@x.com"}
```

Khai báo mã trả về cho tài liệu API: `[ProducesResponseType<Customer> (StatusCodes.Status201Created)]`,
`[ProducesResponseType(StatusCodes.Status404NotFound)]`.

Controller mỏng, nghiệp vụ trong dịch vụ

Một quy tắc quan trọng: **controller chỉ làm việc của tầng HTTP**:

1. Nhận dữ liệu (đã bind/validate),
2. Gọi **một** dịch vụ nghiệp vụ,
3. Chuyển kết quả thành **mã HTTP + body**.

Quy tắc kinh doanh, truy cập CSDL, gửi email... thuộc về dịch vụ/domain. Lợi ích: nghiệp vụ test được **không cần HTTP** (unit test Tập 1, Chương 38); dùng lại được (cho worker, gRPC, console); đổi CSDL không đụng controller.

```
public interface ICustomerService
{
    Task<IReadOnlyList<Customer>> GetAllAsync(string? search, CancellationToken ct);
    Task<Customer?> GetByIdAsync(int id, CancellationToken ct);
    ...
}
builder.Services.AddSingleton<ICustomerService, InMemoryCustomerService>(); // Chươn
```

`UserController` gốc dùng `IUserRepository` đúng theo hướng này; bản mới chỉ đổi thành primary constructor và trả 201.

Thử API

File `customers.http` đi kèm (mở bằng VS/VS Code hoặc dùng `curl`):

```
curl -s http://localhost:5207/api/customers?search=jo
curl -i -X POST http://localhost:5207/api/customers -H "Content-Type: application/json"
curl -i -X PUT http://localhost:5207/api/customers/2 -H "Content-Type: application/json"
curl -i -X DELETE http://localhost:5207/api/customers/3
```

Filters — chèn xử lý quanh action

Tương tự middleware nhưng **biết action, tham số, kết quả**:

Authorization → Resource → [Model binding] → Action filter (trước) → ACTION → Action filter (sau)

```
public class GhiThoiGianAttribute : ActionFilterAttribute
{
    public override void OnActionExecuting(ActionExecutingContext ctx) { /* trước action */
    public override void OnActionExecuted(ActionExecutedContext ctx) { /* sau action */
}

[GhiThoiGian] public class CustomersController { ... } // áp cho cả controller (hoặc action)
```

Loại filter: **Authorization** (`[Authorize]`), **Resource**, **Action**, **Exception**, **Result**. Dùng cho vấn đề xuyên suốt gắn với MVC (audit, chuẩn hoá kết quả). Xử lý lỗi toàn cục tốt hơn nên ở `middleware/ExceptionHandler` (Chương 9).

Lỗi thường gặp

- `404` cho endpoint tồn tại — quên `app.MapControllers()`, hoặc route/constraint không khớp (`abc` cho `{id:int}`).
- `415 Unsupported Media Type` — body JSON nhưng thiếu `Content-Type: application/json`.
- `AmbiguousMatchException: The request matched multiple endpoints` — hai action cùng route + phương thức.
- Action trả `200` cho mọi tình huống thay vì `404/201/204` đúng nghĩa.
- Nhét nghiệp vụ và truy cập CSDL vào controller ("fat controller").
- Trả thẳng entity CSDL (lộ trường nhạy cảm) — dùng DTO (Chương 8).
- Quên `CancellationToken`, hoặc chặn luồng bằng `.Result` thay vì `await`.
- Trả `List<T>` lớn không phân trang.

Bài tập

1. Thêm `GET /api/customers/{id}/don-hang` (route lồng) — trả danh sách rỗng, chú ý đặt tên và mã trạng thái.
2. Thêm `PATCH /api/customers/{id}/email` nhận body `{ "email": "..."} .`
3. Viết `ActionFilter` `[GhiLog]` in tên action và thời gian; gắn vào controller.
4. Đổi tên class `CustomersController` thành `KhachHangController` và quan sát URL; sau đó cố định URL bằng `[Route("api/customers")]` .
5. Viết unit test (không HTTP) cho `InMemoryCustomerService` và một test kiểm tra `CustomersController` bằng dịch vụ giả.

Chương 8 — DTO và Validation

Mục tiêu học

Sau chương này, bạn sẽ:

- Hiểu **DTO** là gì và vì sao **không bao giờ** trả/nhận thẳng entity nội bộ.
- Kiểm tra dữ liệu bằng **DataAnnotations**, `InvalidOperationException` và attribute tự viết.
- Đọc và tùy chỉnh lỗi validation theo chuẩn **ProblemDetails**.
- Validation cho Minimal API; biết tới FluentValidation; phân biệt kiểm tra **định dạng** và **nghiệp vụ**.

Code mẫu: [code/ch08-dto-validation/](#) .

Vấn đề: đừng lộ entity

Ở Chương 7, controller nhận/trả thẳng class `Customer` . Với dữ liệu thật, đó là con đường dẫn tới lỗi và lộ bí mật:

```
public class SanPham           // entity nội bộ
{
    public int Id { get; set; }
    public string Ten { get; set; } = "";
    public decimal Gia { get; set; }
    public decimal GiaVon { get; set; }      // bí mật kinh doanh!
    public string? EmailNhaCungCap { get; set; }
}

[HttpPost] public IActionResult Tao(SanPham sp) { ... }    // X client tự gửi "id", "{
[HttpGet]  public List<SanPham> Lay() ...                // X trả cả GiaVon, email
```

Hai lỗi hổng cùng một gốc:

- **Over-posting (mass assignment)**: client gửi thêm trường không được phép (`"id": 1` , `"isAdmin": true` , `"giaVon": 1`) và server gán luôn.
- **Data leak**: trả về mọi thuộc tính, gồm cả thứ nhạy cảm.

Ngoài ra, ràng buộc entity với API làm **mọi thay đổi bên trong** (đổi tên cột, tách bảng) vỡ hợp đồng với client.

DTO — Data Transfer Object

DTO là kiểu chỉ dùng để **truyền dữ liệu qua ranh giới** (API), tách biệt với entity. Nguyên tắc: **DTO vào $\neq$ DTO ra $\neq$ entity**.

```
// DTO vào: chỉ những trường client được phép gửi
public class TaoSanPhamRequest { public string? Ten { get; init; } public decimal Gia

// DTO ra: chỉ những gì client cần biết
public record SanPhamResponse(int Id, string Ma, string Ten, decimal Gia, IReadOnlyList
```

Anh xa (mapping) qua extension method:

```
public static SanPhamResponse ToResponse(this SanPham s) => new(s.Id, s.Ma, s.Ten, s.G
public static SanPham ToEntity(this TaoSanPhamRequest r) => new() { Ma = r.Ma!, Ten =
```

Chạy code mẫu: GET /api/san-pham/1 **không** có giaVon, dù entity có. Có thư viện mapping (**AutoMapper**, **Mapster**) nhưng nhiều đội chọn code tay: tường minh, kiểm tra được lúc biên dịch, không "phép thuật" — hợp với dự án vừa và nhỏ. Với record / init (Tập 1, Chương 32) DTO gọn và bất biến.

Quy ước đặt tên: TaoSanPhamRequest / CapNhatSanPhamRequest (vào), SanPhamResponse / SanPhamDto (ra); mỗi thao tác có DTO riêng vì trường bắt buộc khác nhau (tạo mới cần Ma; cập nhật thì không cho đổi Ma).

DataAnnotations

```
public class TaoSanPhamRequest
{
    [Required(ErrorMessage = "Ten san pham la bat buoc")]
    [StringLength(100, MinimumLength = 2, ErrorMessage = "Ten phai tu 2 den 100 ky tu")
    public string? Ten { get; init; }

    [Required(ErrorMessage = "Ma san pham la bat buoc")]
    [RegularExpression("^[A-Z]{2}[0-9]{3}$", ErrorMessage = "Ma phai co dang 2 chu HOA
    public string? Ma { get; init; }

    [Range(1_000, 1_000_000_000, ErrorMessage = "Gia phai tu 1.000 den 1 ty")]
    public decimal Gia { get; init; }

    [EmailAddress] public string? EmailNhaCungCap { get; init; }
    [MaxLength(5)] public List<string> The { get; init; } = [];
}
```

Attribute	Kiểm tra
[Required]	không null (và với string: không rỗng)
[StringLength], [MinLength], [MaxLength]	độ dài chuỗi/số phần tử
[Range]	khoảng số/ngày
[RegularExpression]	khớp mẫu regex
[EmailAddress], [Url], [Phone], [CreditCard]	định dạng thường gặp
[Compare("Field")]	bằng trường khác (xác nhận mật khẩu)

Lưu ý nullable: thuộc tính `string Ten` (không nullable) bị coi là `[Required]` ngầm định dưới nullable reference types; nhưng để **thông báo lỗi** như mong muốn và phân biệt "thiếu" với "rỗng", khai báo `string?` kèm `[Required]` như trên rất phổ biến. Với kiểu giá trị (`decimal` Gia), thiếu trong JSON sẽ mang giá trị `0` (không phải "thiếu") — nếu cần bắt buộc thật sự hãy dùng `decimal?` + `[Required]` hoặc `required` (C# 11).

Kết quả: 400 với ValidationProblemDetails

Với `[ApiController]`, request sai **không chạm tới action**; framework trả ngay:

```
{
  "title": "Du lieu gui len khong hop le",
  "status": 400,
  "instance": "/api/san-pham",
  "errors": {
    "Ma": ["Ma phai co dang 2 chu HOA + 3 so, vi du LT001"],
    "Gia": ["Gia phai tu 1.000 den 1 ty"],
    "Ten": ["Ten khong duoc chua tu 're rach'"],
    "The": ["Toi da 5 the"],
    "EmailNhaCungCap": ["Email lien he khong dung dinh dang"]
  },
  "traceId": "0HN..."
}
```

Mọi lỗi được trả **một lần** (không dừng ở lỗi đầu tiên) để client hiển thị tất cả bên cạnh các ô nhập. Dạng này chuẩn **RFC 9457 Problem Details** (`application/problem+json`). Tùy chỉnh (đổi tiêu đề, thêm `traceId`) qua `ConfigureApiBehaviorOptions(o => o.InvalidModelStateResponseFactory = ...)` như trong `Program.cs` mẫu.

Attribute tự viết

Dùng lại một quy tắc ở nhiều DTO (nhờ Tập 1, Chương 37 — attribute):

```
[AttributeUsage(AttributeTargets.Property | AttributeTargets.Parameter)]
public sealed class KhongChuaTuCamAttribute(params string[] tuCam) : ValidationAttribute
{
    protected override ValidationResult? IsValid(object? value, ValidationContext ctx)
    {
        if (value is string s)
            foreach (var tu in tuCam)
                if (s.Contains(tu, StringComparison.OrdinalIgnoreCase))
                    return new ValidationResult($"{ctx.DisplayName} không được chứa tu");
        return ValidationResult.Success;
    }
}

[KhongChuaTuCam("re rach", "hang gia")] public string? Ten { get; init; }
```

Kiểm tra chéo nhiều trường: IValidatableObject

```
public class TaoSanPhamRequest : IValidatableObject
{
    public DateOnly? KhuyenMaiTu { get; init; }
    public DateOnly? KhuyenMaiDen { get; init; }

    public IEnumerable<ValidationResult> Validate(ValidationContext ctx)
    {
        if (KhuyenMaiTu is { } tu && KhuyenMaiDen is { } den && den < tu)
            yield return new ValidationResult("Ngày kết thúc khuyến mãi phải sau ngày" +
                $"{nameof(KhuyenMaiTu)}, {nameof(KhuyenMaiDen)}");
    }
}
```

Bẫy: Validate của IValidatableObject **chỉ chạy khi mọi validation cấp thuộc tính đã đạt**. Trong ví dụ "nhiều lỗi" ở trên, lỗi ngày khuyến mãi không xuất hiện vì các lỗi khác đã chặn; gửi riêng một request chỉ sai ngày sẽ thấy nó. Đây là hành vi có chủ đích, đừng ngạc nhiên.

Định dạng ≠ nghiệp vụ

Hai tầng kiểm tra khác nhau:

	Validation đầu vào (định dạng)	Kiểm tra nghiệp vụ
Ví dụ	tên không rỗng, email đúng dạng, giá trong khoảng	mã sản phẩm chưa tồn tại , đủ tồn kho, khách chưa bị khoá
Cần dữ liệu ngoài request?	Không	Có (CSDL, dịch vụ khác)
Nơi thực hiện	DataAnnotations/filter	dịch vụ nghiệp vụ (ném exception nghiệp vụ hoặc trả Result)
Lỗi	400 Bad Request	thường 409 Conflict, 422, 404... tùy trường hợp

Trong controller mẫu: kiểm tra trùng mã là nghiệp vụ (cần kho dữ liệu), nên viết trong action/dịch vụ, và vẫn dùng `ModelState.AddModelError` + `ValidationProblem(ModelState)` để định dạng lỗi cùng dạng với các lỗi khác:

```
if (kho.TonTaiMa(req.Ma!))
{
    ModelState.AddModelError(nameof(req.Ma), $"Ma {req.Ma} da ton tai");
    return ValidationProblem(ModelState);
}
```

Đừng tin chỉ validation phía client (JavaScript) — dễ bị bỏ qua. **Server luôn validate lại.**

Validation cho Minimal API

Minimal API **không** tự validate DataAnnotations (trước .NET 10; .NET 10 đã thêm `AddValidation()` cho Minimal API — kiểm tra tài liệu phiên bản bạn dùng). Cách chung, chạy trên mọi phiên bản: **endpoint filter** dùng `Validator.TryValidateObject` :

```
app.MapPost("/minimal/san-pham", (TaoSanPhamRequest req) => ...)
    .AddEndpointFilter(async (ctx, next) =>
    {
        foreach (var arg in ctx.Arguments.Where(a => a is not null))
        {
            var ketQua = new List<ValidationResult>();
            if (!Validator.TryValidateObject(arg!, new ValidationContext(arg!), ketQua,
                return Results.ValidationProblem(GomLoi(ketQua));
        }
        return await next(ctx);
    });
```

Mã đầy đủ trong `Program.cs` mẫu; nên rút thành filter dùng lại cho cả nhóm route.

FluentValidation

Khi quy tắc phức tạp, phụ thuộc lẫn nhau hay cần test riêng, nhiều dự án dùng thư viện **FluentValidation** — viết quy tắc bằng code thay vì attribute:

```
public class TaoSanPhamValidator : AbstractValidator<TaoSanPhamRequest>
{
    public TaoSanPhamValidator()
    {
        RuleFor(x => x.Ten).NotEmpty().Length(2, 100);
        RuleFor(x => x.Gia).InclusiveBetween(1_000, 1_000_000_000);
        RuleFor(x => x.KhuyenMaiDen).GreaterThan(x => x.KhuyenMaiTu).When(x => x.Khuye
    }
}
```

Ưu: quy tắc **tách khỏi DTO**, dễ đọc, dễ unit test, hỗ trợ điều kiện/quy tắc bất đồng bộ (gọi CSDL). Nhược: thêm phụ thuộc. Sẽ dùng ở Tập 3 (pipeline validation).

Lỗi thường gặp

- Trả thẳng entity ra API → lộ trường nhạy cảm; nhận thẳng entity → over-posting.
- Dùng cùng một DTO cho tạo và cập nhật nên phải nới lỏng `[Required]`.
- Trường `[Required]` trên `int/decimal` bắt được trường vắng (giá trị mặc định `0` vẫn "có giá trị").
- Chỉ validate ở client.
- Nhồi kiểm tra nghiệp vụ (cần CSDL) vào attribute → khó test, phụ thuộc DI khó chịu.
- Trả lỗi `500` thay vì `400/409` khi dữ liệu sai; hoặc trả `200` kèm `"error": ...`.
- Thông báo lỗi lộ chi tiết nội bộ (tên bảng, câu SQL).
- Quên `IsValidatableObject` chỉ chạy sau khi các lỗi thuộc tính đã sạch.

Bài tập

1. Thêm `CapNhatSanPhamRequest` (không cho đổi `Ma`) và action `PUT /api/san-pham/{id}`; ánh xạ vào entity.
2. Viết attribute `[NgayTuongLai]` chỉ chấp nhận `DateOnly` sau hôm nay.
3. Thêm `SanPhamSummaryResponse` (id, tên, giá) cho danh sách và `SanPhamDetailResponse` cho chi tiết.
4. Viết một endpoint filter dùng lại (generic) `ValidationFilter<T>` cho Minimal API và áp dụng cho `MapGroup`.
5. Thử gửi `{"id":999,"giaVon":1,"ten":"A"}` tới action nhận DTO và tới action nhận entity; so sánh hậu quả.

Chương 9 — Xử lý lỗi, versioning và OpenAPI

Mục tiêu học

Sau chương này, bạn sẽ:

- Xử lý lỗi **tập trung** bằng `ExceptionHandler` và `ProblemDetails`, ánh xạ exception nghiệp vụ → mã HTTP.
- Thiết kế **versioning** cho API để đổi hợp đồng mà không làm vỡ client cũ.
- Sinh và tùy biến tài liệu **OpenAPI**, dùng giao diện **Scalar** để thử API.
- Bật **CORS** đúng cách và **health checks**.

Code mẫu: [code/ch09-loi-openapi/](#).

Xử lý lỗi tập trung

Nếu mỗi action tự `try/catch` và tự dựng response lỗi, code lặp, định dạng lỗi mỗi nơi một kiểu, dễ lộ chi tiết nội bộ. Giải pháp: để exception **nổi lên** và có **một nơi duy nhất** biến nó thành response.

1. Exception nghiệp vụ có ý nghĩa

```
public abstract class LoiNghiepVu(string message) : Exception(message);
public class KhongTimThayException(string loai, object khoa) : LoiNghiepVu($"Khong tim
public class XungDotException(string message) : LoiNghiepVu(message);

// Trong dịch vụ / endpoint:
var sp = kho.Tim(id) ?? throw new KhongTimThayException("San pham", id);
```

Tầng nghiệp vụ nói bằng ngôn ngữ **nghiep vu** (không thấy HTTP); tầng web dịch sang mã HTTP.

2. IExceptionHandler (.NET 8+)

```
public class GlobalExceptionHandler(ILogger<GlobalExceptionHandler> log, IProblemDetail
{
    public async ValueTask<bool> TryHandleAsync(HttpContext ctx, Exception e, Cancellat
    {
        var (status, title) = e switch
        {
            KhongTimThayException => (404, "Khong tim thay"),
            XungDotException => (409, "Xung dot du lieu"),
```

```

        OperationCanceledException => (499, "Client huy yeu cau"),
        _ => (500, "Loi may chu"),
    };

    if (status >= 500) log.LogError(e, "Loi khong xu ly tai {Path}", ctx.Request.Path)
    else log.LogWarning("Loi nghiep vu {Loai}: {Message}", e.GetType().Name, e.Message);

    ctx.Response.StatusCode = status;
    return await problemDetails.TryWriteAsync(new ProblemDetailsContext
    {
        HttpContext = ctx,
        Exception = e,
        ProblemDetails = new ProblemDetails
        {
            Status = status, Title = title,
            Detail = status < 500 ? e.Message : null,           // KHÔNG lộ chi tiết
            Instance = ctx.Request.Path,
        },
    });
}
}

builder.Services.AddProblemDetails();
builder.Services.AddExceptionHandler<GlobalExceptionHandler>();
app.UseExceptionHandler();           // sớm nhất trong pipeline (Chương 3)
app.UseStatusCodePages();             // 404/405 không body → cũng dạng ProblemDetails

```

Kết quả thực tế khi chạy code mẫu:

```

GET /loi/khong-tim-thay → 404 application/problem+json
{"type":"https://tools.ietf.org/html/rfc9110#section-15.5.5","title":"Khong tim thay",
  "detail":"Khong tim thay San pham '999',"instance":"/loi/khong-tim-thay","traceId":"

GET /loi/bat-ngo → 500 (KHÔNG có detail – chi tiết chỉ nằm trong log)
{"type":"...#section-15.6.1","title":"Loi may chu","status":500,"instance":"/loi/bat-n

```

Điểm mấu chốt:

- **Lỗi nghiệp vụ (4xx):** hiển thị thông điệp cho client (họ cần biết để sửa), log mức `Warning` không kèm stack trace.
- **Lỗi bất ngờ (5xx):** không trả `e.Message` /stack trace (lộ cấu trúc nội bộ, câu SQL, đường dẫn máy chủ — thông tin có giá trị cho kẻ tấn công); trả thông điệp chung + `traceId` để người dùng báo lại và bạn tra log. Log `Error` đầy đủ.
- Mọi lỗi cùng **một định dạng** (`ProblemDetails`) — client xử lý thống nhất.

- Trong `Development` `UseDeveloperExceptionPage` (mặc định) hiện trang lỗi chi tiết cho lập trình viên; production dùng handler trên.

Khi hai dịch vụ gọi nhau, `traceId` cho phép lần theo chuỗi request xuyên nhiều hệ thống (Tập 3: OpenTelemetry).

Lỗi vs Result

Không phải mọi kết quả "thất bại" đều là ngoại lệ. Lỗi **dự kiến, thường xuyên** ("không đủ hàng") có thể trả về như **giá trị** (`Result`, `Conflict<ProblemDetails>`, như endpoint `/mua`), tránh chi phí và độ ồn của exception. Ngoại lệ dành cho **bất thường** hoặc khi nhiều tầng cần "bỏ dờ". Tập 3 nói kỹ Result pattern.

Versioning

API công khai là **hợp đồng**: khi client đã dùng, bạn không được tùy tiện đổi tên trường, xoá trường, đổi kiểu. Khi cần thay đổi phá vỡ tương thích (**breaking change**), phát hành **phiên bản mới** và giữ phiên bản cũ đủ lâu.

Ba cách phổ biến:

Cách	Ví dụ	Ghi chú
Đường dẫn	<code>/api/v1/san-pham</code> , <code>/api/v2/san-pham</code>	rõ ràng, dễ cache/test, phổ biến nhất
Query	<code>/api/san-pham?api-version=2</code>	dễ dùng thử trên trình duyệt
Header	<code>X-Api-Version: 2</code> hoặc media type (<code>Accept: application/vnd.x.v2+json</code>)	URL sạch, khó thử

Code mẫu dùng đường dẫn với `MapGroup` :

```
var v1 = app.MapGroup("/api/v1/san-pham");
var v2 = app.MapGroup("/api/v2/san-pham");

v1.MapGet("/{id:int}", ...); // {"id":1,"ten":"Laptop","gia":18000000}
v2.MapGet("/{id:int}", ...); // {"id":1,"ten":"Laptop","gia":{"soTien":18000000,"don
```

v2 đổi `gia` từ số sang đối tượng (thêm đơn vị tiền) — nếu đổi trực tiếp, mọi client v1 vỡ. Cho dự án lớn dùng thư viện **Asp.Versioning** (versioning theo attribute, tự đưa vào OpenAPI, báo phiên bản lỗi thời qua header `api-supported-versions`, `Deprecation` / `Sunset`).

Quy tắc: **thay đổi tương thích thì không cần version mới** — thêm trường tùy chọn, thêm endpoint. **Phá vỡ** — xoá/đổi tên/đổi kiểu trường, đổi ý nghĩa, thêm tham số bắt buộc — mới

cần. Thiết kế client theo "tolerant reader" (bỏ qua trường lạ) để dễ tiến hoá.

OpenAPI

OpenAPI (trước kia Swagger) là chuẩn mô tả API dạng JSON/YAML: mọi endpoint, tham số, body, phản hồi, xác thực. Từ .NET 9, [ASP.NET](#) Core sinh sẵn bằng

`Microsoft.AspNetCore.OpenApi` :

```
builder.Services.AddOpenApi(options =>
{
    options.AddDocumentTransformer((doc, ctx, ct) =>
    {
        doc.Info = new OpenApiInfo { Title = "API Cua Hang", Version = "v1", Descripti
        return Task.CompletedTask;
    });
});

if (app.Environment.IsDevelopment())
{
    app.MapOpenApi(); // GET /openapi/v1.json
    app.MapScalarApiReference(); // giao diện /scalar/v1 (package Scalar.AspNetCore)
}
```

Thông tin lấy từ code: route, kiểu tham số, kiểu trả về khai báo bằng

`Results<...> / [ProducesResponseType]` , XML doc comment (bật `GenerateDocumentationFile`), attribute `[Tags]` , `WithSummary` , `WithDescription` .

Giá trị của tài liệu OpenAPI:

- **Giao diện thử API** (Scalar, Swagger UI, Postman nhập file) — người dùng API không cần đọc code.
- **Sinh code client** (C#, TypeScript, Java...) bằng NSwag/Kiota/openapi-generator — tránh viết tay DTO phía client.
- **Kiểm thử hợp đồng** và phát hiện breaking change trong CI.

Đặt `MapOpenApi /UI` ở `Development` hoặc sau xác thực khi production (không muốn công khai bản đồ API nội bộ).

CORS

Trình duyệt chặn trang ở origin A gọi API ở origin B (**same-origin policy**) trừ khi server B cho phép bằng header CORS. `curl` và server-to-server **không bị** ảnh hưởng — nên lỗi CORS chỉ hiện trong trình duyệt.

```
builder.Services.AddCors(o => o.AddPolicy("web-app", p => p
    .WithOrigins("https://cuahang.example.com", "http://localhost:3000")
    .WithMethods("GET", "POST", "PUT", "DELETE")
    .AllowAnyHeader()));

app.UseCors("web-app"); // sau UseRouting, TRƯỚC UseAuthentication/UseAuthorizati
```

Thử: request có Origin: `http://localhost:3000` nhận `Access-Control-Allow-Origin: http://localhost:3000`; Origin: `http://evil.com` **không** nhận header nào. Nguyên tắc: liệt kê **chính xác** origin được phép; **không** dùng `AllowAnyOrigin()` cùng `AllowCredentials()` (bị chặn và nguy hiểm). CORS là cơ chế của **trình duyệt bảo vệ người dùng**, không phải cơ chế bảo mật API (kẻ tấn công vẫn gọi được bằng công cụ khác — xác thực mới là cái bảo vệ).

Health checks

Endpoint cho hệ thống giám sát/orchestrator (Docker, Kubernetes, load balancer) hỏi "ứng dụng còn khỏe không?":

```
builder.Services.AddHealthChecks().AddCheck("bo-nho", () => GC.GetTotalMemory(false) <
    ? HealthCheckResult.Healthy("Bo nho on dinh") : HealthCheckResult.Degraded("Bo nho
app.MapHealthChecks("/health"); // 200 "Healthy" / 503 "Unhealthy"
```

Thực tế thêm check CSDL, dịch vụ ngoài (`AspNetCore.HealthChecks.*`). Tách **liveness** (tiến trình còn sống) và **readiness** (đã sẵn sàng nhận tải — phụ thuộc kết nối được) bằng `Tags` /nhiều endpoint (Tập 3).

Lỗi thường gặp

- Quên `app.UseExceptionHandler()` (đăng ký handler nhưng không bật middleware).
- Đặt middleware xử lý lỗi **sau** middleware khác nên không bắt được lỗi của chúng.
- Trả `e.Message` /stack trace ra client ở production.
- Bắt `Exception` chung chung trong từng action rồi trả `500` với thông điệp riêng lẻ.
- Đổi hợp đồng API ngay trên phiên bản đang dùng.
- Lỗi CORS trong trình duyệt nhưng `curl` chạy tốt — thiếu origin trong policy, hoặc `UseCors` đặt sai thứ tự.
- Công khai OpenAPI/Scalar trên production ngoài ý muốn.
- Health check gọi thao tác nặng → chính nó làm sập ứng dụng.

Bài tập

1. Thêm `LoiDuLieuKhongHopLeException` → `422 Unprocessable Entity` vào `GlobalExceptionHandler`.

2. Thêm `TimeoutException` → `504 Gateway Timeout` với thông điệp chung.
3. Tạo `/api/v3/san-pham/{id}` thêm `hinhAnh` (tương thích) và giải thích vì sao không cần phiên bản mới nếu chỉ *thêm* trường.
4. Dùng `WithSummary/WithDescription` mô tả các endpoint; mở `/scalar/v1` và kiểm tra.
5. Thêm health check kiểm tra ghi được vào thư mục tạm; thử làm nó thất bại và quan sát `503`.

Chương 10 — Kiểm thử API

Mục tiêu học

Sau chương này, bạn sẽ:

- Phân biệt **unit test**, **integration test** và khi nào dùng loại nào cho web.
- Viết integration test chạy **cả ứng dụng thật trong bộ nhớ** bằng `WebApplicationFactory<Program>`.
- **Thay thế dịch vụ** (cổng thanh toán, CSDL...) bằng bản giả trong test.
- Kiểm tra mã trạng thái, header, body JSON, xác thực, validation; giữ test độc lập và nhanh.

Code mẫu: [code/ch10-kiem-thu-api/](#) (API `DonHangApi` + project `DonHangApi.Tests`).

Nền tảng: xUnit, Arrange–Act–Assert, mock/stub — đã học ở Tập 1, Chương 38. Chương này áp dụng vào tầng HTTP.

Hai tầng kiểm thử cho web

	Unit test	Integration test (API)
Đối tượng	một lớp/hàm (dịch vụ nghiệp vụ, validator)	cả pipeline : routing, binding, validation, filter, serialization
Phụ thuộc	mọi phụ thuộc bị thay bằng giả	dịch vụ ngoại vi bị thay; phần còn lại là thật
Tốc độ	mili-giây	vài chục mili-giây (không mạng thật)
Bắt được	lỗi logic	lỗi kết nối các phần : route sai, thiếu Content-Type, JSON sai tên, thứ tự middleware, DI thiếu đăng ký

Unit test không thể phát hiện "quên `app.UseAuthentication()`" hay "đăng ký DI sai vòng đời" — integration test thì được. Cần cả hai: **nhều unit test nhanh cho nghiệp vụ + vài chục integration test cho các luồng chính** (hình kim tự tháp).

WebApplicationFactory<Program>

Package `Microsoft.AspNetCore.Mvc.Testing`. Nó khởi động **ứng dụng thật của bạn** trong tiến trình test bằng **TestServer** (không mở cổng mạng), và đưa cho bạn `HttpClient` gửi thẳng request vào pipeline.

Điều kiện: project test tham chiếu project web, và lớp `Program` phải nhìn thấy được. Với top-level statements `Program` là `internal` ẩn — thêm một dòng cuối `Program.cs`:

```
public partial class Program;
```

Test đầu tiên:

```
public class DonHangApiTests : IClassFixture<WebApplicationFactory<Program>>
{
    private readonly HttpClient _client;
    public DonHangApiTests(WebApplicationFactory<Program> factory) => _client = factory

    [Fact]
    public async Task TrangChu_TraVeChuoixacNhan()
    {
        var res = await _client.GetAsync("/");

        Assert.Equal(HttpStatusCode.OK, res.StatusCode);
        Assert.Contains("dang chay", await res.Content.ReadAsStringAsync());
    }
}
```

`IClassFixture<T>` : tạo factory **một lần cho cả class test** (khởi động app tốn chút thời gian) và chia sẻ. Vì thế trạng thái Singleton (kho dữ liệu trong bộ nhớ) **dùng chung giữa các test trong class** — viết test **không phụ thuộc thứ tự** (mỗi test tự tạo dữ liệu nó cần, không giả định "danh sách đang rỗng").

Thay thế phụ thuộc bằng bản giả

API tạo đơn hàng gọi cổng thanh toán bên ngoài (`ICongThanhToan`). Test **không được** gọi cổng thật. Ghi đè đăng ký DI:

```
public class ApiFactory : WebApplicationFactory<Program>
{
    public ICongThanhToan CongThanhToanGia { get; } = Substitute.For<ICongThanhToan>()

    protected override void ConfigureWebHost(IWebHostBuilder builder)
    {
        builder.UseEnvironment("Testing");
        builder.ConfigureServices(services =>
        {
            services.AddScoped(_ => CongThanhToanGia); // đăng ký SAU → thắng đ
        });
    }
}
```

Đăng ký cuối cùng cho cùng một kiểu **thăng** — nên `AddScoped` trong `ConfigureServices` của test ghi đè bản thật. Trong test dàn kết quả:

```
_factory.CongThanhToanGia
    .ThanhToanAsync(Arg.Any<decimal>(), Arg.Any<CancellationToken>())
    .Returns(new KetQuaThanhToan(true, "GD-TEST-01", null));
```

Đây là phần thưởng của việc **lập trình theo interface + DI** (Tập 1, Chương 42; Chương 5): thay phụ thuộc mà không sửa code sản phẩm. Tương tự thay `DbContext` bằng SQLite in-memory hoặc `IHttpClientFactory` bằng handler giả.

Các loại kiểm tra thường gặp

```
// 1. Tạo mới → 201 + Location + body
var res = await _client.PostAsJsonAsync("/api/don-hang", new { kháchHang = "An", soTien
Assert.Equal(HttpStatusCode.Created, res.StatusCode);
var don = await res.Content.ReadFromJsonAsync<DonHang>();
Assert.Equal($"api/don-hang/{don!.Id}", res.Headers.Location?.OriginalString);

// 2. Tạo rồi đọc lại → đúng đối tượng (kiểm tra cả vòng khứ hồi)
var lay = await _client.GetFromJsonAsync<DonHang>($"api/don-hang/{don.Id}");
Assert.Equal(don, lay); // record: so sánh theo giá trị

// 3. Không tồn tại → 404
Assert.Equal(HttpStatusCode.NotFound, (await _client.GetAsync("/api/don-hang/99999")));

// 4. Dữ liệu sai (Theory) → 400 + problem+json
[Theory]
[InlineData("", 1000)] [InlineData("An", 0)] [InlineData("An", -5)]
public async Task TaoDonHang_DuLieuSai_Tra400(string kháchHang, decimal soTien) { ...

// 5. Xác thực: client không có header → 401
var khongKhoa = _factory.CreateClient(); // client mới, không header mã
Assert.Equal(HttpStatusCode.Unauthorized, (await khongKhoa.GetAsync("/api/don-hang")));

// 6. Nhánh lỗi từ dịch vụ ngoài → 402 và KHÔNG lưu đơn
```

Mẹo:

- `PostAsJsonAsync`, `GetFromJsonAsync`, `ReadFromJsonAsync<T>` (`System.Net.Http.Json`) gọn hơn tự dựng chuỗi JSON.
- Dùng **DTO/record của chính dự án** để đọc response — nếu đổi tên trường, test biên dịch lỗi, báo ngay.
- Kiểm tra `Content-Type` cho lỗi (`application/problem+json`).

- Client mặc định **tự theo redirect**; cần kiểm tra redirect thì `CreateClient(new WebApplicationFactoryClientOptions { AllowAutoRedirect = false })`.
- `builder.UseSetting("ApiKey", "khac")` hoặc `ConfigureAppConfiguration` để đổi cấu hình trong test.

Kiểm thử với cơ sở dữ liệu

Từ Chương 12 có EF Core. Ba chiến lược cho test:

Chiến lược	Ưu	Nhược
SQLite in-memory (<code>DataSource=:memory:</code> , giữ kết nối mở)	nhanh, gần CSDL thật, chạy khắp nơi	không giống 100% SQL Server/PostgreSQL
EF InMemory provider	rất nhanh	không mô phỏng ràng buộc, transaction, SQL → dễ "xanh giả"; không khuyến khích
CSDL thật trong container (Testcontainers)	chính xác nhất	chậm hơn, cần Docker (Tập 3)

Nguyên tắc: test khớp môi trường thật nhất có thể chấp nhận về chi phí; luôn chạy migration/khởi tạo schema trong fixture; dọn dữ liệu (hoặc mỗi test một CSDL/transaction rollback).

Tổ chức và chất lượng

- Đặt tên test mô tả tình huống: `TaoDonHang_ThanhToanThatBai_Tra402VaKhongLuuDon`.
- Test **hợp đồng** (contract): status code, header, hình dạng JSON — đây là thứ client phụ thuộc.
- Test **hành vi**, không test chi tiết cài đặt (đừng verify từng lời gọi nội bộ trừ khi đó là tác dụng phụ quan trọng, như gửi email).
- Test độc lập: không phụ thuộc thứ tự; dùng dữ liệu ngẫu nhiên/riêng (`Guid.NewGuid()` cho tên).
- Chạy trong **CI** mỗi commit (Tập 3: GitHub Actions `dotnet test`).
- Một số ít test **smoke** đầu–cuối trên môi trường thật sau deploy (gọi `/health`).

File .http và kiểm thử thủ công

Song song test tự động, giữ file `.http` (như `customers.http` ở Chương 7) làm **tài liệu sống + thử nhanh** khi phát triển — khác test: chạy bằng tay, cho người đọc.

Lỗi thường gặp

- The type 'Program' is inaccessible — quên `public partial class Program;` .
- Test chạy được riêng lẻ nhưng lỗi khi chạy cả bộ — **phụ thuộc thứ tự / trạng thái chung** trong Singleton dùng chung qua `IClassFixture` .
- Gọi dịch vụ ngoài thật trong test (chậm, không ổn định, tác dụng phụ).
- Không cấu hình môi trường/thiết lập → ứng dụng đọc `appsettings.Development.json` ngoài ý muốn (dùng `UseEnvironment("Testing")`).
- Dùng EF InMemory rồi tin rằng "test qua = CSDL chạy đúng".
- Bỏ qua kiểm tra nhánh lỗi (400/401/404/409/5xx) — chính là nơi lỗi hay ẩn.
- Test quá chi tiết vào JSON (so chuỗi thô) → giòn; nên đọc thành DTO.
- Quên `await` với `PostAsJsonAsync` ... → test "xanh" nhưng không kiểm tra gì.

Bài tập

1. Thêm `PUT /api/don-hang/{id}` (đổi tên khách) và viết bốn test: thành công (204), không tồn tại (404), dữ liệu sai (400), không có API key (401).
2. Viết test cho tình huống `ICongThanhToan` **ném exception** — API nên trả gì? Chỉnh code cho hợp lý (502 / 503) rồi viết test.
3. Đổi API key bằng cấu hình trong test (`builder.UseSetting("ApiKey", "khoa-khac")`) và kiểm tra khoá cũ bị từ chối.
4. Viết test kiểm tra header `Location` của `POST` dùng được để `GET` (theo `Location` , không tự ghép URL).
5. Chạy `dotnet test --collect:"XPlat Code Coverage"` và tìm nhánh chưa được test trong `Program.cs` .

Chương 11 — SQL cơ bản và SQLite

Mục tiêu học

Sau chương này, bạn sẽ:

- Hiểu cơ sở dữ liệu **quan hệ**: bảng, khoá chính, khoá ngoại, ràng buộc, chỉ mục.
- Viết SQL cơ bản: `SELECT`, `WHERE`, `JOIN`, `GROUP BY`, `HAVING`, truy vấn con, `INSERT/UPDATE/DELETE`.
- Hiểu **giao dịch (transaction)** và vì sao ràng buộc bảo vệ dữ liệu.
- Biết **SQL injection** là gì và cách chặn bằng **truy vấn có tham số**.

Code mẫu: [code/ch11-sql-sqlite/](#) — `schema.sql` (chạy được trong mọi công cụ SQLite) và chương trình console dùng `Microsoft.Data.Sqlite`.

Vì sao học SQL khi đã có EF Core?

Chương 12–14 dùng **Entity Framework Core** để làm việc với CSDL bằng C#. EF Core chỉ là lớp phiên dịch: mọi LINQ cuối cùng thành **SQL**. Nếu không hiểu SQL bạn sẽ không đọc được câu lệnh EF sinh ra, không biết vì sao truy vấn chậm, không thiết kế được bảng đúng. Hiểu SQL là điều kiện để dùng EF hiệu quả.

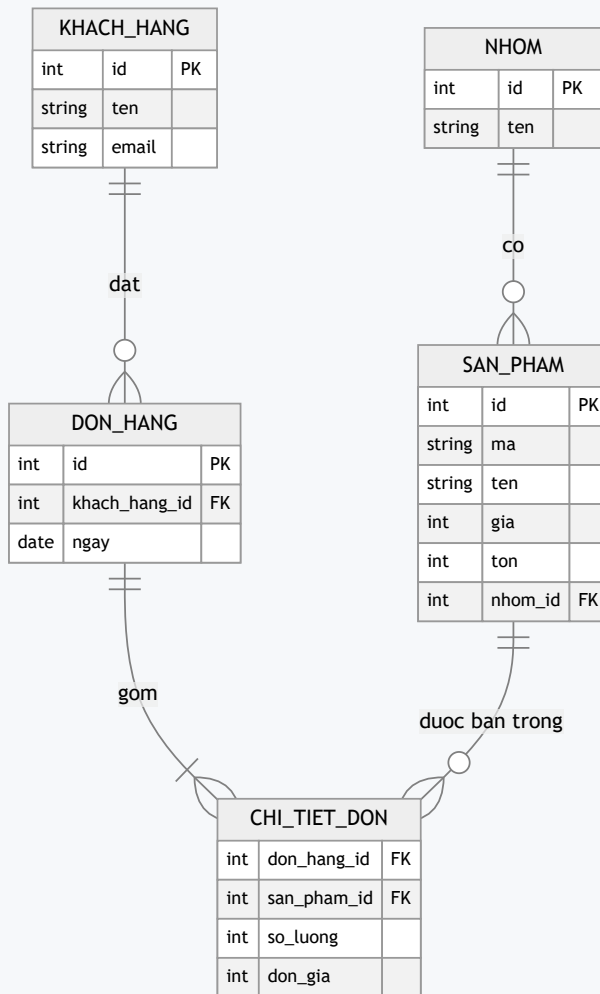
SQLite — CSDL để học

SQLite là CSDL nằm gọn trong **một file** (hoặc bộ nhớ), không cần cài server: hợp học tập, thử nghiệm, test, ứng dụng nhỏ. Sản phẩm thật nhiều người dùng thường chọn **PostgreSQL**, **SQL Server** hoặc **MySQL** — SQL căn bản giống nhau, khác ở chi tiết (kiểu dữ liệu, hàm ngày giờ, cú pháp phân trang). Sách dùng SQLite để mọi ví dụ chạy được ngay; các chỗ khác biệt sẽ được ghi chú.

Công cụ: `sqlite3` (dòng lệnh), **DB Browser for SQLite**, extension SQLite cho VS Code, hoặc chính chương trình mẫu.

Mô hình quan hệ

Dữ liệu chia vào các **bảng**; mỗi bảng có **cột** (kiểu dữ liệu) và **dòng** (bản ghi). Bảng liên kết nhau bằng khoá:



- **Khoá chính (PK)**: định danh duy nhất một dòng (id).
- **Khoá ngoại (FK)**: cột trỏ tới khoá chính của bảng khác (san_pham.nhom_id → nhom.id), CSDL **bảo đảm** không trỏ vào chỗ không tồn tại.
- **Quan hệ**: 1–nhiều (một nhóm có nhiều sản phẩm), nhiều–nhiều (đơn hàng ↔ sản phẩm, qua bảng nối chi_tiet_don — bảng này còn mang dữ liệu riêng như số lượng, đơn giá).

Chuẩn hoá: mỗi sự thật chỉ lưu **một chỗ** (tên nhóm nằm ở bảng nhom, không lặp trên từng sản phẩm). Lưu ý chi_tiet_don.don_gia : **chụp lại** giá lúc bán, vì giá sản phẩm sẽ đổi mà hoá đơn cũ thì không được đổi.

Định nghĩa bảng và ràng buộc

```
CREATE TABLE san_pham (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    ma          TEXT      NOT NULL UNIQUE,
    ten         TEXT      NOT NULL,
    gia         INTEGER NOT NULL CHECK (gia >= 0),
    ton         INTEGER NOT NULL DEFAULT 0 CHECK (ton >= 0),
```

```

    nhom_id INTEGER NOT NULL REFERENCES nhom(id)
);
CREATE INDEX ix_san_pham_nhom ON san_pham(nhom_id);

```

Ràng buộc	Ý nghĩa
PRIMARY KEY	định danh duy nhất, không NULL
NOT NULL	bắt buộc có giá trị
UNIQUE	không trùng (mã sản phẩm, email)
CHECK (điều kiện)	giá trị phải thoả điều kiện (giá ≥ 0)
DEFAULT	giá trị mặc định
FOREIGN KEY / REFERENCES	phải trỏ tới dòng tồn tại; ON DELETE CASCADE/RESTRICT quyết định khi xoá dòng cha

Ràng buộc là hàng phòng thủ cuối cùng. Ứng dụng có validation (Chương 8) nhưng có bug, có nhiều ứng dụng cùng ghi vào một CSDL, có người sửa tay — chỉ CSDL mới bảo đảm dữ liệu không bao giờ sai. Chạy code mẫu, mục 9: chèn trùng mã, giá âm, nhóm không tồn tại đều bị từ chối:

```

UNIQUE constraint failed: san_pham.ma
CHECK constraint failed: gia >= 0
FOREIGN KEY constraint failed

```

(SQLite cần `PRAGMA foreign_keys = ON` cho mỗi kết nối để thực thi khoá ngoại; các CSDL khác luôn bật.)

Kiểu tiền tệ: đừng dùng số thực (`REAL/FLOAT`) cho tiền vì sai số (Tập 1, Chương 5). Dùng số nguyên (đồng) hoặc `DECIMAL(18,2)` ở CSDL hỗ trợ.

Truy vấn dữ liệu

SELECT, WHERE, ORDER BY, LIMIT

```

SELECT ma, ten, gia
FROM san_pham
WHERE gia >= 200000
ORDER BY gia DESC
LIMIT 3;

```

Thứ tự **xử lý** logic: `FROM` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `ORDER BY` → `LIMIT` .

Toán tử: `=` `<` `>` `<=` `>=` , `AND` `OR` `NOT` , `IN (...)` , `BETWEEN a AND b` , `LIKE '%an%'` , `IS NULL`

(không dùng `= NULL`). `NULL` nghĩa là "không biết": `NULL = NULL` không phải đúng.

JOIN — ghép bảng

```
SELECT sp.ma, sp.ten, n.ten AS nhóm, sp.gia
FROM san_pham sp
JOIN nhóm n ON n.id = sp.nhóm_id      -- INNER JOIN: chỉ dòng có cặp khớp ở cả hai b
ORDER BY n.ten, sp.ten;
```

- `INNER JOIN` : chỉ giữ dòng khớp cả hai phía.
- `LEFT JOIN` : giữ **mọi dòng bảng trái**, phía phải không có thì `NULL` :

```
SELECT k.ten, COUNT(d.id) AS so_don
FROM khách_hang k
LEFT JOIN đơn_hang d ON d.khách_hang_id = k.id
GROUP BY k.id;
-- Le Chi: 0 đơn (INNER JOIN sẽ làm khách này biến mất)
```

GROUP BY và hàm tổng hợp

```
SELECT k.ten, SUM(c.so_luong * c.dơn_gia) AS doanh_thu
FROM khách_hang k
JOIN đơn_hang d ON d.khách_hang_id = k.id
JOIN chi_tiet_dơn c ON c.dơn_hang_id = d.id
GROUP BY k.id
HAVING SUM(c.so_luong * c.dơn_gia) > 1000000    -- HAVING lọc SAU khi gom nhóm (WHERE
ORDER BY doanh_thu DESC;
```

Hàm: `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`. Mọi cột trong `SELECT` không nằm trong hàm tổng hợp phải nằm trong `GROUP BY` (một số CSDL, như SQLite, nới lỏng — đừng dựa vào).

Truy vấn con

```
SELECT ma, ten FROM san_pham sp
WHERE NOT EXISTS (SELECT 1 FROM chi_tiet_dơn c WHERE c.san_pham_id = sp.id);    -- sản
```

Ghi dữ liệu

```
INSERT INTO khách_hang (ten, email) VALUES ('Pham Dung', 'dung@example.com');
UPDATE san_pham SET ton = ton + 10 WHERE ma = 'TN001';
DELETE FROM khách_hang WHERE email = 'dung@example.com';
```

Nhớ WHERE ! UPDATE / DELETE thiếu WHERE tác động **cả bảng**. Thói quen: viết SELECT với cùng WHERE để kiểm tra trước, và thao tác quan trọng trong giao dịch.

Chỉ mục (index)

Không có chỉ mục, tìm WHERE ten = '...' phải **quét toàn bộ bảng** (chậm dần theo số dòng).

Chỉ mục là cấu trúc dữ liệu phụ (thường cây B-tree) giúp tìm nhanh, đổi lại tốn bộ nhớ và làm ghi chậm hơn một chút. EXPLAIN QUERY PLAN cho thấy CSDL định làm gì:

```
SEARCH san_pham USING INDEX ix_san_pham_nhom (nhom_id=?)    -- có chỉ mục: tìm nhanh
SCAN san_pham                                              -- không có: quét toàn bả
```

Quy tắc: đặt chỉ mục trên **khoá ngoại** và các cột hay dùng trong WHERE/JOIN/ORDER BY ;
PRIMARY KEY và UNIQUE tự có chỉ mục. Đừng đặt bừa mọi cột.

Giao dịch (transaction)

Một thao tác nghiệp vụ thường gồm nhiều lệnh (trừ kho **và** tạo đơn). Giao dịch bảo đảm **tất cả cùng thành công hoặc cùng huỷ** (nguyên tắc **ACID**: Atomicity, Consistency, Isolation, Durability):

```
await using var tx = (SqliteTransaction)await conn.BeginTransactionAsync();
try
{
    await LenhTrongGiaoDichAsync(conn, tx, "UPDATE san_pham SET ton = ton - 1 WHERE ma
    await LenhTrongGiaoDichAsync(conn, tx, "INSERT INTO chi_tiet_don VALUES (999, 5, 1
    await tx.CommitAsync();
}
catch (SqliteException) { await tx.RollbackAsync(); } // lệnh UPDATE trước đó cũng
```

Chạy mẫu: sau ROLLBACK, tồn kho LT001 vẫn là 3. Với EF Core (Chương 12), SaveChanges() tự bọc mọi thay đổi trong một giao dịch.

SQL từ C#: truy vấn có tham số

Chương trình mẫu dùng Microsoft.Data.Sqlite (trình điều khiển SQLite hiện đại của .NET; các CSDL khác có Npgsql cho PostgreSQL, Microsoft.Data.SqlClient cho SQL Server — cùng mô hình):

```
await using var cmd = conn.CreateCommand();
cmd.CommandText = "SELECT ten, gia FROM san_pham WHERE ten LIKE $tuKhoa AND gia <= $giaToiDa";
cmd.Parameters.AddWithValue("$tuKhoa", "%an%");
cmd.Parameters.AddWithValue("$giaToiDa", 1_000_000);
await using var r = await cmd.ExecuteReaderAsync();
while (await r.ReadAsync()) Console.WriteLine($"{r.GetString(0)} - {r.GetInt64(1):N0}");
```

Bạn sẽ hiếm khi viết mã như vậy khi đã có EF Core, nhưng cần hiểu nền tảng — và cần SQL thuần cho báo cáo phức tạp/tối ưu (Chương 13).

SQL injection — lỗ hổng nguy hiểm bậc nhất

Khi **nối chuỗi** dữ liệu người dùng vào câu SQL, kẻ tấn công thay đổi ý nghĩa câu lệnh:

```
string dauVao = "x' OR '1'='1";
string sql = $"SELECT COUNT(*) FROM khach_hang WHERE email = '{dauVao}'"; // X NGUYEN
// → SELECT COUNT(*) FROM khach_hang WHERE email = 'x' OR '1'='1' → luôn đúng
```

Ví dụ điển hình: form đăng nhập bị bypass mật khẩu; tệ hơn là `' ; DROP TABLE khach_hang; --` xoá dữ liệu hoặc đánh cắp toàn bộ CSDL qua `UNION`. Nó nằm trong nhóm lỗ hổng phổ biến nhất của OWASP nhiều năm.

Cách phòng duy nhất đáng tin: tham số hoá — dữ liệu và lệnh đi **hai kênh riêng**, CSDL không bao giờ hiểu dữ liệu là mã:

```
cmd.CommandText = "SELECT COUNT(*) FROM khach_hang WHERE email = $email"; // ✓
cmd.Parameters.AddWithValue("$email", dauVao); // →
```

Quy tắc bắt buộc:

1. **Không bao giờ** nối/nội suy dữ liệu bên ngoài vào SQL. Kể cả dữ liệu "có vẻ an toàn" (số, tham số sắp xếp).
2. Tên bảng/cột không tham số hoá được → dùng **whitelist** (chỉ chấp nhận `"gia"`, `"ten"` như Chương 14).
3. EF Core với LINQ tự tham số hoá; `FromSqlInterpolated($"... {x}")` cũng tham số hoá (khác `FromSqlRaw` với chuỗi nối — nguy hiểm).
4. Tài khoản CSDL của ứng dụng chỉ nên có quyền tối thiểu (không `DROP`).

Lỗi thường gặp

- `UPDATE / DELETE` thiếu `WHERE`.

- Dùng `INNER JOIN` khi cần `LEFT JOIN` (mất dòng không có cặp), hoặc `JOIN` nhân đôi dòng rồi `SUM` sai.
- So sánh `= NULL` thay vì `IS NULL`.
- Lưu tiền bằng `REAL/FLOAT`.
- Thiếu chỉ mục trên khoá ngoại/cột lọc → chậm khi dữ liệu lớn; đặt chỉ mục thừa → ghi chậm.
- `SELECT *` trong code ứng dụng (lấy thừa cột, vỡ khi đổi schema).
- Nối chuỗi SQL với dữ liệu người dùng.
- Quên bật `foreign_keys` trên SQLite rồi thắc mắc vì sao khoá ngoại không có tác dụng.

Bài tập

1. Viết truy vấn: 3 sản phẩm bán nhiều nhất (theo tổng số lượng) kèm tên nhóm.
2. Tìm khách hàng chưa mua sản phẩm nào thuộc nhóm "Thiết bị".
3. Thêm cột `ngay_tao` vào `san_pham` và chỉ mục cho `don_hang.ngay`; dùng `EXPLAIN QUERY PLAN` để so sánh trước/sau khi có chỉ mục khi lọc theo ngày.
4. Thử `ON DELETE CASCADE`: xoá một `don_hang` và kiểm tra `chi_tiet_don`; sau đó thử xoá một `nhom` đang có sản phẩm.
5. Viết lại đoạn "SQL injection" với dữ liệu đầu vào `'`; `DELETE FROM khach_hang`; `--` ở cả hai cách (nối chuỗi có thể chạy nhiều lệnh tùy trình điều khiển) và giải thích vì sao tham số hoá miễn nhiễm.

Chương 12 — Entity Framework Core cơ bản

Mục tiêu học

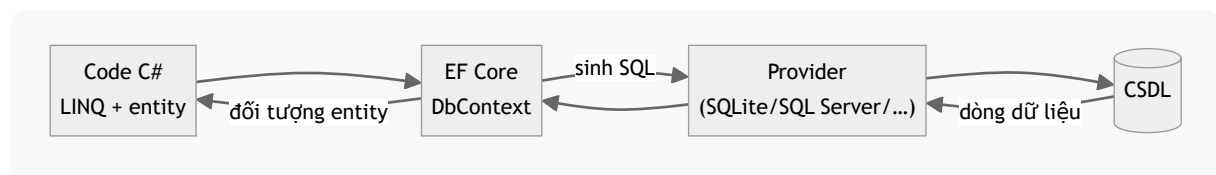
Sau chương này, bạn sẽ:

- Hiểu **ORM** và vai trò của EF Core: `entity`, `DbContext`, `DbSet`, **change tracking**.
- Thực hiện **CRUD** bằng LINQ; đọc được SQL mà EF sinh ra (`ToQueryString`).
- Cấu hình mô hình bằng quy ước, attribute và **Fluent API**; tạo/áp dụng **migration**.
- Phân biệt `AsNoTracking`, `ExecuteUpdate/ExecuteDelete`; hiểu `SaveChanges` là một giao dịch.

Code mẫu: [code/ch12-efcore-co-ban/](#) (có sẵn thư mục `Migrations/` sinh từ `dotnet ef`).

ORM là gì?

Chương 11 bạn viết SQL và đọc từng cột bằng `r.GetString(0)` — dài dòng, dễ sai, không có kiểm tra kiểu lúc biên dịch. **ORM** (Object-Relational Mapper) ánh xạ **bảng** ↔ **class**, **dòng** ↔ **đối tượng**, **cột** ↔ **property**; bạn thao tác bằng C#/LINQ, ORM sinh SQL. **Entity Framework Core (EF Core)** là ORM chính thức, mã nguồn mở, đa nền tảng của .NET (hỗ trợ SQL Server, PostgreSQL, MySQL, SQLite, Cosmos DB...).



Lợi: năng suất, an toàn kiểu, tham số hoá tự động (chống SQL injection), migration, chuyển đổi CSDL dễ hơn. Cái giá: **trừu tượng che khuất SQL** — dùng bừa sẽ gây truy vấn chậm (N+1, Chương 13). Nguyên tắc: **dùng EF cho phần lớn công việc, và biết SQL để kiểm soát nó**.

Cài đặt

```
dotnet add package Microsoft.EntityFrameworkCore.Sqlite # provider (đối provi
dotnet add package Microsoft.EntityFrameworkCore.Design # công cụ migration (
dotnet tool install --global dotnet-ef # lệnh dotnet ef
```

Entity và DbContext

Entity là class C# thường (POCO) ứng với một bảng:

```

public class SanPham
{
    public int Id { get; set; }           // quy ước: "Id" hoặc "<Tên>Id" là khoá c
    public string Ma { get; set; } = "";
    public string Ten { get; set; } = "";
    public decimal Gia { get; set; }
    public int Ton { get; set; }
    public int NhomId { get; set; }      // khoá ngoại
    public Nhom? Nhom { get; set; }     // navigation property (Chương 13)
}

```

DbContext là đối tượng trung tâm: một **phiên làm việc** với CSDL (mẫu **Unit of Work + Repository**). Mỗi `DbSet<T>` đại diện một bảng:

```

public class CuaHangDbContext : DbContext
{
    public DbSet<Nhom> Nhoms => Set<Nhom>();
    public DbSet<SanPham> SanPhams => Set<SanPham>();

    public CuaHangDbContext() { }
    public CuaHangDbContext(DbContextOptions<CuaHangDbContext> options) : base(options)

    protected override void OnConfiguring(DbContextOptionsBuilder options)
    {
        if (!options.IsConfigured) options.UseSqlite("Data Source=cuahang.db");
    }

    protected override void OnModelCreating(ModelBuilder mb) { /* Fluent API */ }
}

```

Trong web, `DbContext` được **đăng ký DI** (`AddDbContext` , Chương 14) với vòng đời **Scoped** — một `DbContext` cho mỗi request. Trong console (như mẫu) tạo bằng `new` trong khối `await using` . **DbContext** **rẻ để tạo, ngắn để sống, và không thread-safe** — đừng dùng chung nhiều luồng, đừng giữ mãi trong Singleton.

Cấu hình mô hình

Ba tầng, cái sau thắng cái trước:

1. **Quy ước** (mặc định): `Id` là khoá chính; `NhomId` + `Nhom` là quan hệ; `string?` cho phép NULL.
2. **Attribute** (`[Key]` , `[Required]` , `[MaxLength(100)]` , `[Column("ten")]` , `[Table("san_pham")]` , `[NotMapped]`).

3. **Fluent API** trong `OnModelCreating` — mạnh nhất, gom cấu hình ra khỏi entity (nên ưu tiên trong dự án lớn):

```
mb.Entity<SanPham>(e =>
{
    e.ToTable("san_pham");
    e.HasIndex(s => s.Ma).IsUnique();
    e.Property(s => s.Ma).IsRequired().HasMaxLength(20);
    e.ToTable(t => t.HasCheckConstraint("ck_san_pham_gia", "Gia >= 0"));
    e.HasOne(s => s.Nhom).WithMany(n => n.SanPhams).HasForeignKey(s => s.NhomId)
        .OnDelete(DeleteBehavior.Restrict); // không cho xóa nhóm còn sản phẩm
});

mb.Entity<Nhom>().HasData(new Nhom { Id = 1, Ten = "Phu kien" }, new Nhom { Id = 2, Ten = "Dung cu" });
```

Ghi chú SQLite: nó không có kiểu `decimal` thật. Mấu dùm `cfg.Properties<decimal>().HaveConversion<double>()` để `Gia` so sánh/sắp xếp/tổng được trong SQL. Với SQL Server/PostgreSQL dùng `.HasPrecision(18, 2)` và kiểu `decimal` gốc.

Migration — quản lý phiên bản schema

Schema (bảng/cột) thay đổi theo thời gian. **Migration** là các file C# mô tả từng bước thay đổi, được sinh từ khác biệt giữa mô hình và snapshot trước đó:

```
dotnet ef migrations add KhoiTao           # sinh Migrations/2026..._KhoiTao.cs (+ DesignTime)
dotnet ef database update                   # áp dụng migration chưa chạy lên CSDL
dotnet ef migrations script                 # xuất SQL để DBA xem/duyệt
dotnet ef migrations remove                 # bỏ migration cuối (nếu chưa áp dụng)
```

```
public partial class KhoiTao : Migration
{
    protected override void Up(MigrationBuilder mb) // đi tới phiên bản này
    {
        mb.CreateTable(name: "san_pham", columns: table => new { ... }, constraints: t => t.PrimaryKey("Ma"));
    }
    protected override void Down(MigrationBuilder mb) => ... // quay lui
}
```

Quy trình: **sửa entity/cấu hình** → `migrations add <Mô-tả>` → **xem lại file sinh ra** → `database update` → **commit cả file migration vào Git**. Migration đã áp dụng lên môi trường chung **không sửa lại** — thêm migration mới. Trong code, `db.Database.Migrate()` áp dụng migration khi khởi động (thuận tiện cho dev/demo; production thường chạy bước migrate riêng trong

pipeline triển khai để kiểm soát rủi ro). `EnsureCreated()` chỉ tạo bảng theo mô hình hiện tại **không** dùng migration — chỉ để thử nhanh/test, không trộn với migration.

CRUD

Create

```
db.SanPhams.Add(new SanPham { Ma = "CH001", Ten = "Chuot khong day", Gia = 150_000, To
db.SanPhams.AddRange(a, b, c);
int soDong = await db.SaveChangesAsync(); // ← MỖI ghi xuống CSDL, trong MỘT giao
```

`Add` chỉ **đánh dấu** entity là "sẽ thêm" trong bộ nhớ. `SaveChanges` mới gửi SQL. Nhiều thay đổi gom lại được gửi theo **lô**, cùng một giao dịch.

Read — LINQ thành SQL

```
var truyVan = db.SanPhams
    .Where(s => s.Gia >= 200_000)
    .OrderByDescending(s => s.Gia)
    .Select(s => new { s.Ma, s.Ten, s.Gia });

Console.WriteLine(truyVan.ToQueryString());
```

```
SELECT "s"."Ma", "s"."Ten", "s"."Gia"
FROM "san_pham" AS "s"
WHERE "s"."Gia" >= 200000.0
ORDER BY "s"."Gia" DESC
```

`ToQueryString()` là công cụ học và gỡ lỗi quý giá. Truy vấn **chỉ chạy khi bạn duyệt/hiện thực hoá**: `ToListAsync`, `FirstOrDefaultAsync`, `SingleAsync`, `CountAsync`, `AnyAsync`, `SumAsync` ... (thực thi trì hoãn, Tập 1, Chương 30 — `IQueryable` khác `IEnumerable` ở chỗ biểu thức được **dịch sang SQL** thay vì chạy trong bộ nhớ).

```
await db.SanPhams.FindAsync(1); // theo khoá chính; kiểm
await db.SanPhams.FirstOrDefaultAsync(s => s.Ma == "BP001"); // null nếu không có
await db.SanPhams.SingleAsync(s => s.Ma == "CH001"); // đúng 1 dòng, không th
```

Quan trọng: trong `Where/Select/OrderBy` chỉ dùng thứ EF dịch được sang SQL. Gọi hàm C# tuỳ ý (`s.Ten.LamDep()`) sẽ lỗi (hoặc EF đành lấy dữ liệu về xử lý trong bộ nhớ). Với chuỗi, `Contains` dịch sang `instr / LIKE` **có thể phân biệt hoa/thường tuỳ CSDL** — dùng `EF.Functions.Like(...)` khi cần rõ ràng.

Update — change tracking

```
var sp = await db.SanPhams.SingleAsync(s => s.Ma == "CH001"); // "tracked": EF nhớ g
sp.Gia = 175_000; // chỉ sửa đối tượng
Console.WriteLine(db.Entry(sp).State); // Modified
await db.SaveChangesAsync(); // EF so sánh → UPDAT
```

Change tracker theo dõi mọi entity nạp qua context. Trạng thái: `Detached`, `Unchanged`, `Added`, `Modified`, `Deleted`. `SaveChanges` sinh `INSERT/UPDATE/DELETE` tương ứng. Bạn không cần gọi `"Update()"` cho entity đã theo dõi.

Delete

```
var sp = await db.SanPhams.SingleAsync(s => s.Ma == "MH001");
db.SanPhams.Remove(sp);
await db.SaveChangesAsync();
```

AsNoTracking — đọc thì đừng theo dõi

Với dữ liệu **chỉ để đọc/trả về API**, tracking là phí bộ nhớ và CPU:

```
var ds = await db.SanPhams.AsNoTracking().ToListAsync();
// db.ChangeTracker.Entries().Count() == 0
```

Quy tắc: truy vấn **đọc** → `AsNoTracking()` (hoặc `Select` projection — không theo dõi tự nhiên); truy vấn để **sửa** → mặc định tracking.

ExecuteUpdate / ExecuteDelete — thao tác hàng loạt

Với cập nhật/xoá nhiều dòng, nạp từng entity lên rồi sửa là lãng phí:

```
int n = await db.SanPhams.Where(s => s.NhomId == 1)
    .ExecuteUpdateAsync(s => s.SetProperty(x => x.Ton, x => x.Ton + 10)); // MỘT c
await db.SanPhams.Where(s => s.Ton == 0 && s.DaXoa).ExecuteDeleteAsync();
```

Chúng chạy **ngay** (không cần `SaveChanges`), **bỏ qua change tracker** và không kích hoạt logic của entity. Cũng là cách tăng `Ton = Ton + n` **nguyên tử** (tránh đọc-rồi-ghi gây mất cập nhật khi đồng thời).

SaveChanges là một giao dịch

Mọi thay đổi trong một lần `SaveChanges` cùng thành công hoặc cùng huỷ. Mục 8 của code mẫu thêm hai sản phẩm, một hợp lệ và một trùng mã; kết quả: `DbUpdateException` và **cả hai** bị huỷ (0K001 không tồn tại). Khi cần giao dịch bao nhiêu lần `SaveChanges` hay lệnh SQL: `await using var tx = await db.Database.BeginTransactionAsync(); ... await tx.CommitAsync();`.

Vi phạm ràng buộc (UNIQUE, CHECK, FK) trở thành `DbUpdateException` với `InnerException` mang lỗi CSDL — nơi chuyển thành 409 Conflict / 400 ở web (Chương 14).

Xem SQL mà EF chạy

```
options.LogTo(Console.WriteLine, LogLevel.Information);           // hoặc lọc theo sự k
options.EnableSensitiveDataLogging();                             // CHỈ dev: hiện cả
```

Trong [ASP.NET](#) Core đặt mức log `Microsoft.EntityFrameworkCore.Database.Command = Information` trong `appsettings.json` (Chương 6). Luôn nhìn SQL khi truy vấn có vẻ chậm.

Khi nào không dùng EF?

EF rất tốt cho CRUD và nghiệp vụ thông thường. Với báo cáo phức tạp, xử lý hàng triệu dòng, hoặc tối ưu tuyệt đối, dùng SQL thuần (`FromSql` , `Database.SqlQuery<T>`) hoặc micro-ORM **Dapper** cho đúng chỗ đó — có thể cùng tồn tại trong một dự án.

Lỗi thường gặp

- Quên `SaveChanges` → "không thấy dữ liệu đâu".
- Quên `await` (hoặc dùng `.Result`) với phương thức `...Async` .
- Dùng `DbContext` dùng chung nhiều luồng (`InvalidOperationException: A second operation was started on this context`).
- `ToList()` quá sớm rồi lọc trong bộ nhớ — kéo cả bảng về (hãy `Where` trên `IQueryable`).
- Dùng hàm C# EF không dịch được trong truy vấn.
- Sửa migration đã áp dụng, hoặc quên commit file migration.
- `EnsureCreated` lẫn `Migrate` → schema không khớp.
- Lưu `decimal` trên SQLite mà không có converter → sắp xếp/tổng sai hoặc lỗi.
- `DbContext` Singleton ("captive dependency", Chương 5).

Bài tập

1. Thêm entity `KhachHang` (Ten, Email unique), tạo migration `ThemKhachHang` , xem file sinh ra và áp dụng.
2. Viết truy vấn đếm sản phẩm theo nhóm (`GroupBy`); in SQL bằng `ToQueryString()` .

3. Viết hàm `GiamGiaNhom(int nhomId, decimal phanTram)` bằng `ExecuteUpdateAsync` và so với cách nạp-sửa-lưu; đếm số câu SQL bằng `LogTo` .
4. Cố tình `Add` hai entity, một vi phạm `CHECK` (giá âm); chứng minh bằng truy vấn rằng entity còn lại cũng không được lưu.
5. Chạy `dotnet ef migrations script` và đọc SQL của migration `KhoiTao` ; đối chiếu với `schema.sql` của Chương 11.

Chương 13 — Quan hệ, truy vấn nâng cao và hiệu năng

Mục tiêu học

Sau chương này, bạn sẽ:

- Mô hình hoá quan hệ **1–nhiều, nhiều–nhiều, kiểu sở hữu (owned)** trong EF Core.
- Chọn cách tải dữ liệu liên quan: `Include`, `AsSplitQuery`, **projection**, explicit loading — và **nhận ra/sửa N+1**.
- Dùng **query filter** (xoá mềm), **concurrency token** (xung đột đồng thời), SQL thuần, **phân trang** (offset và keyset).

Code mẫu: [code/ch13-efcore-quan-he/](#) — mỗi mục in ra **số câu SQL thực sự chạy** để bạn thấy khác biệt bằng số liệu.

Quan hệ trong EF Core

1–nhiều

```
public class Nhóm { public int Id { get; set; } public List<SanPham> SanPhams { get; set; } }
public class SanPham { public int NhómId { get; set; } public Nhóm Nhóm { get; set; } }
```

Cặp **khoá ngoại** (`NhómId`) + **navigation** (`Nhóm`, `SanPhams`) được EF tự nhận theo quy ước; cấu hình rõ bằng Fluent API:

```
e.HasOne(s => s.Nhóm).WithMany(n => n.SanPhams).HasForeignKey(s => s.NhómId);
```

Thêm bằng navigation: `new SanPham { Nhóm = phuKien }` — EF tự điền khoá ngoại khi `SaveChanges`.

Nhiều–nhiều

Sản phẩm có nhiều thể, thể gắn nhiều sản phẩm. Từ EF Core 5 **không cần** class cho bảng nối:

```
public class SanPham { public List<Thể> Thể { get; set; } = []; }
public class Thể { public List<SanPham> SanPhams { get; set; } = []; }
e.HasMany(s => s.Thể).WithMany(t => t.SanPhams); // EF tự tạo bảng nối SanPhamT
```

Khi bảng nối **có dữ liệu riêng** (số lượng, đơn giá của `ChiTietDon`), hãy tạo entity nối tường minh với **khoá chính kết hợp**:

```
e.HasKey(c => new { c.DonHangId, c.SanPhamId });
```

Kiểu sở hữu (owned type) — value object

Một khái niệm không có danh tính riêng (địa chỉ) nhưng muốn đóng gói thành class:

```
public class DiaChi { public string Duong { get; set; } = ""; public string ThanhPho {  
public class KhachHang { public DiaChi? DiaChi { get; set; } }  
mb.Entity<KhachHang>().OwnsOne(k => k.DiaChi); // cột DiaChi_Duong, DiaChi_T
```

Truy vấn tự nhiên: `db.KhachHangs.Count(k => k.DiaChi!.ThanhPho == "Ha Noi")`. Đây là cách ánh xạ **value object** của thiết kế hướng miền (Tập 3).

Tải dữ liệu liên quan

Mặc định EF **không** nạp navigation — `don.ChiTiets` rỗng nếu bạn không yêu cầu. Bốn cách, đánh đổi khác nhau. Chạy code mẫu với 20 đơn hàng, mỗi đơn 2 dòng:

Cách	Số câu SQL	Ghi chú
Explicit loading trong vòng lặp	21	X N+1
Include + ThenInclude	1	JOIN một lần
Include + AsSplitQuery	2	tách thành câu đơn giản
Projection (Select)	1	chỉ lấy cột cần, tính trong SQL

N+1 — vấn đề hiệu năng kinh điển

```
var ds = await db.DonHangs.ToListAsync(); // 1 câu: lấy 20 đơ  
foreach (var d in ds)  
    await db.Entry(d).Collection(x => x.ChiTiets).LoadAsync(); // + 20 câu: mỗi đ
```

"N+1": **1** câu lấy danh sách + **N** câu cho từng phần tử. Với 10.000 đơn là 10.001 lượt gọi CSDL — mỗi lượt tốn độ trễ mạng — ứng dụng chậm dần theo dữ liệu. Nó cũng xảy ra **ăn** khi bạn truy cập `d.KhachHang.Ten` trong `foreach` với **lazy loading** (nên tắt/không dùng), hoặc khi gọi truy vấn trong vòng lặp.

Cách phát hiện: đếm câu SQL (như mẫu, hoặc log

`Microsoft.EntityFrameworkCore.Database.Command`), MiniProfiler, Application Insights. Quy tắc: số câu SQL của một request phải là hằng số nhỏ, không tăng theo số dòng.

Include — nạp kèm bằng JOIN

```
var ds = await db.DonHangs.Include(d => d.ChiTiets).ThenInclude(c => c.SanPham).ToList
```

Đơn giản nhưng nạp **toàn bộ cột** của mọi entity liên quan, và các collection lồng nhau tạo **cartesian explosion**: đơn hàng × dòng × thẻ... thành hàng trăm nghìn dòng trùng lặp. Khi có ≥ 2 collection cùng `Include`, cần nhắc `.AsSplitQuery()` — EF chạy **một câu cho mỗi collection** (ít dữ liệu truyền hơn, đổi lại nhiều lượt gọi hơn và có thể không nhất quán tuyệt đối nếu dữ liệu đổi giữa các câu).

Projection — cách tốt nhất khi chỉ đọc

```
var tong = await db.DonHangs
    .Select(d => new
    {
        d.Id,
        KhachHang = d.KhachHang.Ten,
        SoDong = d.ChiTiets.Count,
        TongTien = d.ChiTiets.Sum(c => c.SoLuong * c.DonGia),
    })
    .OrderByDescending(x => x.TongTien).Take(3).ToListAsync(); // 1 câu, EF t
```

Chỉ những cột bạn `Select` được lấy, kết quả không bị theo dõi (không tốn bộ nhớ tracking), phép tính chạy ở CSDL. **Quy tắc**: truy vấn cho **API/báo cáo (đọc)** → dùng projection sang DTO. Truy vấn để **sửa** entity → tải entity (kèm `Include` cần thiết).

Explicit loading

`db.Entry(d).Collection(x => x.ChiTiets).LoadAsync()` — nạp có chủ đích cho **một** entity đã có. Hợp khi cần nạp tùy điều kiện, không hợp trong vòng lặp.

Nhiều–nhiều, truy vấn qua quan hệ

```
var coTheMoi = await db.SanPhams.Where(s => s.The.Any(t => t.Ten == "moi")).Select(s =
```

EF dịch `Any` trên collection thành `EXISTS` trong SQL — viết theo cách nghĩ đối tượng, không cần tự viết JOIN bảng nối.

Query filter — xoá mềm (soft delete)

Nhiều hệ thống không xoá thật mà đánh dấu `DaXoa = true` (giữ lịch sử, hoá đơn cũ vẫn tham chiếu được). **Global query filter** tự thêm điều kiện vào **mọi** truy vấn:

```
e.HasQueryFilter(s => !s.DaXoa);

await db.SanPhams.CountAsync(); // 3 (ẩn sản phẩm đã xóa mềm)
await db.SanPhams.IgnoreQueryFilters().CountAsync(); // 4 (bỏ qua filter khi cần,
```

Dùng nhiều cho **đa tenant** (`TenantId == hiện tại`). Cảnh báo quan trọng: filter trên entity là đầu **bắt buộc** của một quan hệ khiến các dòng phụ thuộc "biến mất" (EF cảnh báo khi tạo model). Ví dụ `ChiTietDon → SanPham` bị lọc làm đơn hàng cũ mất dòng — xử lý bằng `IgnoreQueryFilters()` ở truy vấn đơn hàng (như Chương 14) hoặc cấu hình navigation tùy chọn.

Xung đột đồng thời (optimistic concurrency)

Hai người cùng mở sản phẩm có tồn kho 30, mỗi người bán vài cái rồi lưu — nếu không phòng ngừa, người lưu sau **ghi đè** thay đổi của người trước ("lost update"). **Optimistic concurrency**: không khoá dòng khi đọc (lạc quan là hiếm khi xung đột), nhưng khi ghi kiểm tra "dòng còn nguyên như lúc tôi đọc không?":

```
e.Property(s => s.PhienBan).IsConcurrencyToken(); // hoặc [ConcurrencyCheck] / [T
```

EF sinh: `UPDATE ... SET Ton=@moi, PhienBan=@pb WHERE Id=@id AND PhienBan=@pbCu`. Nếu không cập nhật được dòng nào (vì người khác đã đổi), ném `DbUpdateConcurrencyException`:

```
try { await b.SaveChangesAsync(); }
catch (DbUpdateConcurrencyException) { /* tải lại dữ liệu mới, hoặc báo 409 để client
```

Mục 5 của mẫu mô phỏng hai `DbContext`: người A lưu được, người B nhận `DbUpdateConcurrencyException`. Ở API trả `409 Conflict`. Trên SQL Server dùng cột `rowversion` (`[Timestamp] byte[]`) để DB tự đổi phiên bản mỗi lần sửa; ở PostgreSQL dùng `xmin`.

SQL thuần khi cần

```
decimal giaToiThieu = 200_000;
var ds = await db.SanPhams
    .FromSqlInterpolated($"SELECT * FROM SanPhams WHERE Gia >= {giaToiThieu} ORDER BY
    .ToListAsync();
```

`FromSqlInterpolated` **tham số hoá** các giá trị chèn qua `{}` — an toàn. **Không** dùng `FromSqlRaw` với chuỗi nối. Kết quả có thể ghép tiếp LINQ. Với truy vấn trả kiểu không phải entity: `db.Database.SqlQuery<T>(...)`. Câu SQL phải trả **đủ cột** của entity.

Phân trang

```
// Offset: Skip/Take (đơn giản; trang càng sâu càng chậm vì CSDL phải đếm bỏ qua)
var trang3 = await db.DonHangs.OrderBy(d => d.Id).Skip(2 * kíchThuoc).Take(kíchThuoc).

// Keyset / cursor: "sau khoá cuối cùng đã xem" (nhanh, ổn định ở mọi trang)
var tiếp = await db.DonHangs.Where(d => d.Id > idCuoi).OrderBy(d => d.Id).Take(kíchThuoc).
```

- Luôn **OrderBy** trước **Skip/Take** (không có thứ tự, kết quả không xác định).
- **Offset** cho phép nhảy tới trang bất kỳ, hiển thị "trang 3/12" (cần **Count**); nhược điểm chậm ở trang sâu và dữ liệu trùng/thiếu nếu có thêm/xoá giữa các lần gọi.
- **Keyset** hợp cho cuộn vô hạn/nguồn cấp dữ liệu lớn; chỉ đi tuần tự "trang kế".
- Luôn **giới hạn** **kíchThuoc** phía server (**Math.Clamp**) — đừng để client xin "1 triệu dòng".

Danh sách kiểm tra hiệu năng truy vấn

1. Số câu SQL mỗi request có phải hằng số nhỏ? (không N+1)
2. Truy vấn đọc dùng **projection**/ **AsNoTracking** ?
3. Có **chỉ mục** cho cột lọc/sắp xếp/khoá ngoại? (Chương 11 — đọc **EXPLAIN**)
4. Có **phân trang** và giới hạn kích thước?
5. Không kéo cả bảng về rồi lọc trong bộ nhớ (**ToList()** trước **Where**)?
6. Không gọi CSDL trong vòng lặp?
7. **Include** có kéo thừa quá nhiều? Xem xét split query hoặc projection.
8. Đo bằng số liệu (log, profiler), đừng đoán.

Lỗi thường gặp

- N+1 do truy cập navigation/gọi truy vấn trong vòng lặp.
- **NullReferenceException** vì navigation chưa nạp (**don.ChiTiets** rỗng, **d.KhachHang** null) — quên **Include** .
- **Include** mọi thứ "cho chắc" rồi ngạc nhiên vì chậm/tốn bộ nhớ.
- Vòng tham chiếu khi serialize entity ra JSON (**Nhom** → **SanPham** → **Nhom**) — đừng trả entity, dùng DTO (Chương 8).
- Quên **OrderBy** khi phân trang.
- Dùng **FromSqlRaw** nối chuỗi (SQL injection).
- Quên concurrency token cho dữ liệu như số dư/tồn kho cập nhật đồng thời.
- Query filter làm dữ liệu liên quan "biến mất" ngoài ý muốn.

Bài tập

1. Viết truy vấn projection trả về mỗi khách hàng cùng số đơn và tổng chi tiêu (một câu SQL); kiểm chứng bằng bộ đếm.
2. Cố tình tạo N+1 bằng lazy loading (`UseLazyLoadingProxies`) rồi sửa bằng `Include` và bằng projection.
3. Thêm thẻ vào sản phẩm bằng cách gán `sp.The.Add(the)` và kiểm tra bảng nối trong SQLite.
4. Viết phân trang keyset theo cặp (`Ngay`, `Id`) để ổn định khi nhiều đơn cùng ngày.
5. Tạo hai `DbContext` xung đột nhau theo mẫu mục 5, bắt `DbUpdateConcurrencyException` và viết vòng **thử lại** (tối đa 3 lần) tải lại giá trị mới rồi áp dụng phép trừ.

Chương 14 — Web API hoàn chỉnh với EF Core

Mục tiêu học

Sau chương này, bạn sẽ:

- Tích hợp EF Core vào [ASP.NET](#) Core: `AddDbContext`, **DbContext theo request**, migration khi khởi động.
- Xây API CRUD có **lọc, sắp xếp, phân trang** an toàn, DTO/projection, xoá mềm.
- Viết nghiệp vụ **đặt hàng** dùng giao dịch và xử lý **xung đột đồng thời** → 409.
- Kiểm thử API với **SQLite in-memory** bằng `WebApplicationFactory`.

Code mẫu: [code/ch14-api-voi-efcore/](#) — `CuaHangApi` (API) + `CuaHangApi.Tests` (10 test tích hợp).

Chương này ghép các mảnh: Minimal API (Ch. 4), DI (5), cấu hình (6), DTO/validation (8), xử lý lỗi (9), kiểm thử (10), EF Core (12–13).

Cấu trúc dự án

```
CuaHangApi/
├─ Program.cs           # cấu hình DI, pipeline, migrate + seed
├─ Dtos.cs              # request/response + exception nghiệp vụ
├─ Data/
│   └─ Entities.cs      # Nhóm, SanPham, KhachHang, DonHang, ChiTietDon
│   └─ CuaHangDbContext.cs # cấu hình mô hình (Fluent API)
├─ Endpoints/
│   └─ SanPhamEndpoints.cs # /api/san-pham, /api/nhom
│   └─ DonHangEndpoints.cs # /api/don-hang
│   └─ KiemTraHopLeFilter.cs # endpoint filter validation dùng chung
├─ Services/DonHangService.cs # nghiệp vụ đặt hàng
└─ Migrations/          # sinh từ `dotnet ef migrations add KhoiTao`
```

Nguyên tắc tổ chức: **endpoint mỏng** (HTTP), **nghiệp vụ phức tạp trong service** (`DonHangService`), **truy vấn đơn giản có thể để trong endpoint** (không cần bọc `Repository` thừa mỏng lên `DbContext` — Tập 3 bàn khi nào cần).

Đăng ký DbContext

```
builder.Services.AddDbContext<CuaHangDbContext>(o =>
    o.UseSqlite(builder.Configuration.GetConnectionString("CuaHang") ?? "Data Source=c
```

- Vòng đời **Scoped: một DbContext cho mỗi HTTP request**, hết request thì được dispose — đúng với mẫu **unit of work** (Chương 5).
- Chuỗi kết nối lấy từ cấu hình (`ConnectionStrings:CuaHang`), không viết cứng; bí mật để ngoài Git (Chương 6).
- `DbContext` nhận `DbContextOptions<T>` qua constructor để **DI và test thay CSDL** dễ dàng:

```
public class CuaHangDbContext(DbContextOptions<CuaHangDbContext> options) : DbContext(
```

Đổi sang PostgreSQL/SQL Server chỉ thay dòng `UseSqlite` (và package) — code endpoint không đổi (lưu ý kiểu `decimal`, phân biệt hoa/thường, hàm ngày giờ).

`AddDbContextPool` tái sử dụng đối tượng để giảm cấp phát; `IDbContextFactory<T>` dành cho nơi cần tự tạo context (Blazor Server, tác vụ nền).

Migration và dữ liệu mẫu khi khởi động

[illegible]

Tiện cho dev/demo/test. **Production:** tránh tự migrate khi nhiều instance khởi động cùng lúc và khi migration có thể phá dữ liệu — chạy migration ở bước triển khai riêng (`dotnet ef migrations bundle` hoặc script SQL đã duyệt), bằng cờ `MigrateOnStartup=false` để tắt. Khi tạo migration bằng công cụ, truyền cờ: `dotnet ef migrations add KhoiTao -- --MigrateOnStartup false` (để công cụ chạy `Program.cs` mà không mở CSDL).

API danh sách: lọc, sắp xếp, phân trang

GET /api/san-pham?tim=laptop&nhomId=2&sapXep=-gia&trang=1&kichThuoc=10

```

IQueryable<SanPham> q = db.SanPhams.AsNoTracking();
if (!string.IsNullOrEmpty(tim))
    q = q.Where(s => EF.Functions.Like(s.Ten, $"%{tim}%") || EF.Functions.Like(s.Ma, $
if (nhomId is not null) q = q.Where(s => s.NhomId == nhomId);

q = sapXep switch
{
    // WHITELIST: không bao giờ

```

```

        "gia" => q.OrderBy(s => s.Gia),
        "-gia" => q.OrderByDescending(s => s.Gia),
        "ten" => q.OrderBy(s => s.Ten),
        _ => q.OrderBy(s => s.Id),
    };

    int tongSo = await q.CountAsync(ct);
    var muc = await q.Skip((trang - 1) * kichThuoc).Take(kichThuoc)
        .Select(s => new SanPhamDto(s.Id, s.Ma, s.Ten, s.Gia, s.Ton, s.Nhom.Ten)) // pro
        .ToListAsync(ct);
    return TypedResults.Ok(new TrangKetQua<SanPhamDto>(muc, trang, kichThuoc, tongSo));

```

Điểm đáng nhớ:

- **Xây IQueryable từng bước**, thêm điều kiện khi tham số có mặt — EF chỉ sinh **một** câu SQL cuối cùng.
- **Math.Clamp(kichThuoc, 1, 100)** chặn client đòi cả bảng.
- **Whitelist sắp xếp**: cho client gửi `sapXep=...` và ghép vào SQL/ `OrderBy` động là lỗ hổng (SQL injection/lộ dữ liệu).
- **Tìm kiếm không phân biệt hoa/thường**: `Contains()` trên SQLite phân biệt (dịch sang `instr`), còn SQL Server mặc định không — chênh lệch giữa CSDL. `EF.Functions.Like` cho hành vi rõ ràng (đã phát hiện qua test thất bại đầu tiên khi viết chương này!). Với tìm kiếm văn bản thật sự dùng full-text search của CSDL.
- Trả `TrangKetQua<T>` chứa cả `TongSo`, `TongTrang`, `CoTrangSau` để client dựng phân trang.
- Trả `AsNoTracking` + projection: nhanh, nhẹ.

Tạo, sửa, xoá

```

// POST: 201 + Location, 404 nếu nhóm không có, 409 nếu trùng mã
var sp = new SanPham { Ma = req.Ma!, Ten = req.Ten!.Trim(), Gia = req.Gia, Ton = req.T
db.SanPhams.Add(sp);
try { await db.SaveChangesAsync(ct); }
catch (DbUpdateException) { return TypedResults.Conflict($"Ma san pham {req.Ma} da ton

```

Ở đây **CSDL là người canh gác cuối cùng**: dù ứng dụng có kiểm tra "mã đã tồn tại chưa" trước, hai request đồng thời vẫn có thể cùng vượt qua kiểm tra — chỉ ràng buộc `UNIQUE` chặn được. Bắt `DbUpdateException` và dịch sang `409`.

```

// DELETE: xoá mềm bằng MỘT câu UPDATE, không nạp entity
int n = await db.SanPhams.Where(s => s.Id == id).ExecuteUpdateAsync(s => s.SetProperty
return n == 0 ? TypedResults.NotFound() : TypedResults.NoContent();

```

```
// POST /{id}/nhap-kho: tăng tồn NGUYÊN TỬ, không đọc-rồi-ghi → không mất cập nhật khi  
await db.SanPhams.Where(s => s.Id == id).ExecuteUpdateAsync(s => s.SetProperty(x => x.
```

Validation đầu vào dùng `KiemTraHopLeFilter<T>` (Chương 8) gắn lên endpoint:

```
.AddEndpointFilter<KiemTraHopLeFilter<TaoSanPhamRequest>>>().
```

Nghiệp vụ đặt hàng: giao dịch và xung đột

POST /api/don-hang với { "khachHangId": 1, "dong": [{ "sanPhamId": 3, "soLuong": 2 }, ...] }. Yêu cầu: **kiểm tra tồn kho, trừ kho và tạo đơn cùng thành công hoặc cùng thất bại**.

```
var can = req.Dong.GroupBy(d => d.SanPhamId).ToDictionary(g => g.Key, g => g.Sum(d =>  
var sanPhams = await db.SanPhams.Where(s => can.Keys.Contains(s.Id)).ToDictionaryAsync  
  
var don = new DonHang { KháchHang = khách, Ngay = DateOnly.FromDateTime(dongHo.GetLoca  
foreach (var (id, soLuong) in can)  
{  
    var sp = sanPhams[id] ?? throw new KhôngTimThayException(...);  
    if (sp.Ton < soLuong) throw new KhôngDuHangException(sp.Ma, soLuong, sp.Ton);  
    sp.Ton -= soLuong;  
    don.ChiTiets.Add(new ChiTietDon { SanPham = sp, SoLuong = soLuong, DonGia = sp.Gia  
}  
db.DonHangs.Add(don);  
await db.SaveChangesAsync(ct); // MỘT giao dịch: mọi UPDATE kho + INSERT đơn
```

- Kiểm tra **mọi dòng trước khi ghi**; một dòng thiếu hàng → ném exception → **không có gì được lưu** (test `DatHang_NhieuDong_MotDongThieuHang_KhongDongNaoBiTru` chứng minh dòng A không bị trừ).
- **Chốt DonGia** vào chi tiết đơn: giá sản phẩm đổi sau này không làm thay đổi hoá đơn cũ (Chương 11).
- **Concurrency token trên Ton**: hai đơn cùng đọc tồn 5 và cùng trừ 4 → lệnh `UPDATE ... WHERE Ton = 5` của đơn thứ hai không khớp dòng nào → `DbUpdateConcurrencyException`, bắt và chuyển thành `XungDotException` → 409 `Conflict` ("Ton kho vừa thay doi"). Không có nó, tồn kho có thể thành số âm hoặc sai (mất cập nhật). Ràng buộc `CHECK (Ton >= 0)` là lưới an toàn cuối.
- Exception nghiệp vụ (`KhôngTimThay` → 404, `KhôngDuHang` / `XungDot` → 409, `LoiNghiepVu` khác → 422) được **handler toàn cục** (Chương 9) biến thành `ProblemDetails` — endpoint không cần `try/catch`.

Bạn cũng có thể để client **tự thử lại** khi nhận 409 (idempotency: nếu mạng lỗi, gửi lại cùng đơn có thể tạo đơn thứ hai — thiết kế khoá idempotency ở Tập 3).

Lưu ý về xoá mềm và quan hệ

Sản phẩm đã xoá mềm bị `HasQueryFilter` giấu khỏi mọi truy vấn — kể cả khi **hiển thị đơn hàng cũ có chứa nó**. Vì vậy `DonHangService.LayAsync` dùng `.IgnoreQueryFilters()` (Chương 13). Khi tạo migration EF cảnh báo đúng về điều này: đó là **thiết kế cần chủ động xử lý**, không phải lỗi.

Kiểm thử với SQLite trong bộ nhớ

Test chạy API **thật** với CSDL SQLite **thật** (có ràng buộc, SQL thật), nhưng nằm trong bộ nhớ:

```
public class ApiFactory : WebApplicationFactory<Program>
{
    private readonly SqlConnection _conn = new("DataSource=:memory:");    // giữ MỞ

    protected override void ConfigureWebHost(IWebHostBuilder builder)
    {
        _conn.Open();
        builder.UseEnvironment("Testing");
        builder.ConfigureServices(s =>
        {
            s.RemoveAll<DbContextOptions<CuaHangDbContext>>();
            s.AddDbContext<CuaHangDbContext>(o => o.UseSqlite(_conn));
        });
    }
}
```

Lúc khởi động app chạy `Migrate()` + seed trên CSDL bộ nhớ, nên test có dữ liệu mẫu sẵn. Ưu điểm so với "EF InMemory provider" (Chương 10): các ràng buộc `UNIQUE/CHECK/FK`, giao dịch, SQL thật đều hoạt động → test bắt được lỗi thật (ví dụ 409 khi trùng mã đến từ CSDL). Mười test bao phủ: phân trang/sắp xếp, tìm kiếm, trùng mã (409), mã sai (400), xoá mềm, nhập kho, đặt hàng thành công/thiếu hàng/nhiều dòng/khách không tồn tại.

Nhớ viết test **độc lập thứ tự** (mỗi test tạo mã sản phẩm riêng như `"DH001"`, `"DH002"`) vì factory chia sẻ một CSDL cho cả class.

Chạy thử

```
cd CuaHangApi
dotnet run
curl "http://localhost:5000/api/san-pham?sapXep=-gia&kichThuoc=3"
curl -X POST http://localhost:5000/api/don-hang -H "Content-Type: application/json" \
```

```
-d '{"khachHangId":1,"dong":[{"sanPhamId":1,"soLuong":2}]}'  
dotnet test ../CuaHangApi.Tests
```

Lỗi thường gặp

- `DbContext` dùng trong `Singleton/ BackgroundService` mà không tạo scope (Chương 5).
- Trả entity (có navigation vòng) thay vì DTO → lỗi serialize/lộ dữ liệu.
- Quên `await` → request trả về trước khi `SaveChanges` xong.
- Tin kiểm tra "trùng mã" ở tầng ứng dụng mà quên ràng buộc UNIQUE.
- Đọc-rồi-ghi `Ton` không có concurrency token → âm kho / mất cập nhật.
- Migrate khi khởi động trên production với nhiều instance.
- Test dùng EF InMemory rồi "xanh giả".
- `Skip/Take` không `OrderBy` ; không giới hạn `kichThuoc` .
- Đưa `sapXep` từ client thẳng vào `OrderBy` động.
- Quên `IgnoreQueryFilters` ở truy vấn lịch sử/đơn hàng cũ.

Bài tập

1. Thêm `PUT /api/khach-hang/{id}` và `POST /api/khach-hang` (email duy nhất → 409), kèm test.
2. Thêm `GET /api/don-hang?khachHangId=1&trang=1` trả `TrangKetQua<DonHangDto>` bằng **một** câu SQL (đo bằng `LogTo`).
3. Thêm endpoint `POST /api/don-hang/{id}/huy` hoàn lại tồn kho trong cùng một giao dịch; viết test.
4. Đổi provider sang PostgreSQL (`Npgsql.EntityFrameworkCore.PostgreSQL`) bằng biến cấu hình và liệt kê những chỗ code/migration phải sửa (gợi ý: `decimal` , `Like`).
5. Viết test kiểm chứng xung đột đồng thời: hai request đặt hàng song song (`Task.WhenAll`) cho sản phẩm còn 1 cái; chỉ một thành công, một nhận 409 .

Chương 15 — [ASP.NET Core MVC](#)

Mục tiêu học

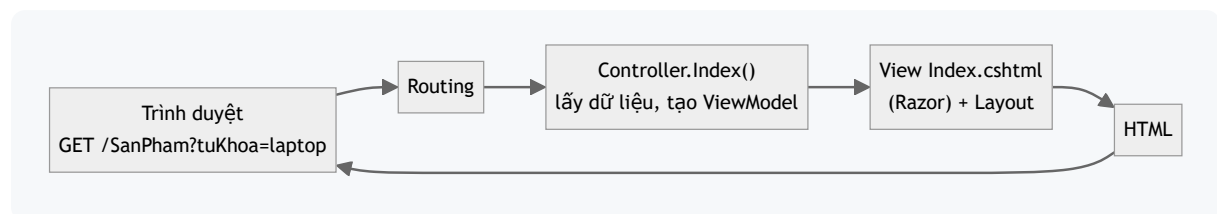
Sau chương này, bạn sẽ:

- Hiểu mô hình **MVC** (Model – View – Controller) và cách nó xử lý một request trả về **HTML**.
- Viết controller, action, **view Razor**, layout, partial view, **tag helper**.
- Dùng **ViewModel**, `ModelState`, `TempData`, pattern **Post-Redirect-Get** cho form.
- Biết khi nào chọn MVC, khi nào Razor Pages hay Blazor.

Code mẫu: [code/ch15-mvc/](#) — ứng dụng quản lý sản phẩm (danh sách, tìm kiếm, phân trang, xem, thêm, sửa, xóa).

Từ API JSON sang HTML phía server

Ở Phần 2–3 bạn trả **JSON** cho một client khác (app di động, JavaScript) lo giao diện. **MVC** cho phép server **tự dựng trang HTML** rồi gửi cho trình duyệt (*server-side rendering*). Ưu điểm: đơn giản, SEO tốt, trang hiển thị nhanh, không cần framework JavaScript nặng; hợp cho trang quản trị, website nội dung, ứng dụng nghiệp vụ nội bộ.



- **Model**: dữ liệu và nghiệp vụ (entity, dịch vụ).
- **View**: mẫu HTML (Razor `.cshtml`) — chỉ lo **hiển thị**.
- **Controller**: nhận request, gọi dịch vụ, chọn View, truyền dữ liệu — chỉ lo **điều phối**.

Tách ba phần giúp mỗi phần thay đổi/kiểm thử độc lập (đây là ứng dụng của SRP, Tập 1, Chương 42).

Cài đặt

```
builder.Services.AddControllersWithViews();           // controller + view (khác
...
app.UseStaticFiles();                                 // wwwroot: css, js, ảnh
app.UseRouting();
app.MapControllerRoute(name: "default", pattern: "{controller=Home}/{action=Index}/{id
```

Route quy ước `{controller=Home}/{action=Index}/{id?}`: `/SanPham/Details/3` → `SanPhamController.Details(id: 3)`; / → `HomeController.Index`. (Khác Web API dùng attribute route.)

Cấu trúc thư mục:

```
ch15-mvc/
├─ Controllers/HomeController.cs, SanPhamController.cs
├─ Models/SanPham.cs                # entity, ViewModel, form
├─ Views/
│   ├── _ViewImports.cshtml        # @using + @addTagHelper cho mọi view
│   ├── _ViewStart.cshtml          # gán Layout mặc định
│   ├── Shared/_Layout.cshtml      # khung trang dùng chung
│   ├── Home/Index.cshtml, Loi.cshtml
│   └─ SanPham/Index, Details, Form, Delete.cshtml
└─ wwwroot/css/site.css
```

Quy ước: `View()` trong action `Details` của `SanPhamController` tìm `Views/SanPham/Details.cshtml`, nếu không có tìm `Views/Shared/Details.cshtml`.

Controller và action

```
public class SanPhamController(SanPhamStore store) : Controller // DI qua primary
{
    public IActionResult Index(string? tuKhoa, int trang = 1)
    {
        var (muc, tongSo) = store.Tim(tuKhoa, trang, KichThuocTrang);
        var vm = new DanhSachSanPhamViewModel { TuKhoa = tuKhoa, Trang = trang, TongTr
        return View(vm); // truyền ViewMo
    }

    public IActionResult Details(int id) => store.Lay(id) is { } sp ? View(sp) : NotFo
}
```

Tham số action được **model binding** từ route (`id`), query (`tuKhoa` , `trang`) hoặc form — như Chương 7. Kết quả `IActionResult` thường dùng: `View()` , `RedirectToAction()` , `NotFound()` , `RedirectToPage` , `Content()` , `Json()` , `File()` , `StatusCode()` .

View Razor

Razor trộn HTML với C#: `@` đưa vào C#.

```
@model DanhSachSanPhamViewModel
```

```
@* kiểu của Model, kiểm tra lúc biên dịch
```

```

<h1>Sàn phẩm</h1>

@foreach (var sp in Model.SanPhams)
{
    <tr class="@ (sp.Ton == 0 ? "het-hang" : null)">
        <td>@sp.Ma</td>
        <td><a asp-action="Details" asp-route-id="@sp.Id">@sp.Ten</a></td>
        <td class="so">@sp.Gia.ToString("N0")</td>
    </tr>
}
@if (Model.Trang < Model.TongTrang) { <a asp-action="Index" asp-route-trang="@ (Model.T

```

Quy tắc cốt lõi:

- **@expr** tự mã hoá HTML (`<script>` trong dữ liệu thành `<script>`) — **chống XSS mặc định**. Chỉ dùng `@Html.Raw(...)` với nội dung bạn tin tưởng tuyệt đối (Chương 21).
- `@{ ... }` khối lệnh; `@if/@foreach/@switch`; `@(biểu thức phức tạp)`; `@*` chú thích `*@`.
- **Không nhồi logic nghiệp vụ vào view**. View chỉ định dạng dữ liệu đã chuẩn bị.
- Mã hoá Unicode: mặc định Razor mã hoá chữ tiếng Việt thành `ộ`; (trình duyệt vẫn hiển thị đúng). Để HTML nguồn đọc được, cấu hình `WebEncoderOptions` với `UnicodeRanges.All` (có trong `Program.cs` mẫu).

Truyền dữ liệu Controller → View

Cách	Khi nào
Model / ViewModel (@model)	mặc định : kiểu mạnh, kiểm tra lúc biên dịch
<code>ViewData["Title"] / ViewBag.X</code>	mẫu dữ liệu nhỏ (tiêu đề trang) — không kiểm tra kiểu
<code>TempData</code>	tồn tại qua đúng một redirect (thông báo "Đã lưu")

ViewModel là class dành riêng cho *một view*, chứa đúng những gì view cần (`DanhSachSanPhamViewModel` có từ khoá, trang, tổng trang, danh sách). Đừng đưa entity thô vào view: bạn sẽ bị kéo dính cấu trúc CSDL, và với form còn bị **over-posting** (Chương 17).

Layout, partial view

`_Layout.cshtml` là **khung chung** (menu, footer); mỗi view chèn vào chỗ `@RenderBody()` :

```

<title>@ViewData["Title"] - Cửa hàng</title>
<nav><a asp-controller="SanPham" asp-action="Index">Sàn phẩm</a> ...</nav>
<main>
    @if (TempData["ThongBao"] is string tb) { <p class="flash">@tb</p> }
    @RenderBody()

```

```
</main>
@await RenderSectionAsync("Scripts", required: false)      @* view tùy chọn thêm scrip
```

`_ViewStart.cshtml` gán `Layout` cho mọi view; `_ViewImports.cshtml` khai báo `@using` và tag helper dùng chung.

Partial view (`<partial name="_Form" model="..." />`) là đoạn giao diện dùng lại (form dùng chung cho Thêm/Sửa). **View component** (`@await Component.InvokeAsync("GioHang")`) là partial có logic riêng (giỏ hàng trên header) nhưng vẫn tách khỏi controller.

Tag helper

Thẻ HTML "thông minh" xử lý phía server, viết giống HTML thường:

```
<a asp-controller="SanPham" asp-action="Details" asp-route-id="@sp.Id">...</a>      @* tự s
<form method="post">...</form>                                                    @* tự
<label asp-for="Ten"></label>                                                       @* nhã
<input asp-for="Ten" />                                                             @* nam
<span asp-validation-for="Ten"></span>                                              @* thô
<div asp-validation-summary="ModelOnly"></div>                                    @* lỗi
<link rel="stylesheet" href="~/css/site.css" asp-append-version="true" />         @* th
```

Ưu điểm so với tự viết `href="/SanPham/Details/3"` : **URL sinh từ routing** — đổi route không phải sửa mọi link; đúng dù ứng dụng chạy dưới thư mục con (`/app`).

Form và validation

```
// GET: hiện form rỗng
public IActionResult Create() => View("Form", new SanPhamForm());

// POST: xử lý
[HttpPost, ValidateAntiForgeryToken]
public IActionResult Create(SanPhamForm form)
{
    if (store.TonTaiMa(form.Ma ?? ""))
        ModelState.AddModelError(nameof(form.Ma), "Mã sản phẩm đã tồn tại");    // lỗi

    if (!ModelState.IsValid)
        return View("Form", form);                                              // hiện lại form + lỗi + dữ

    var sp = store.Them(form);
    TempData["ThongBao"] = $"Đã thêm sản phẩm {sp.Ten}";
    return RedirectToAction(nameof(Details), new { id = sp.Id });    // Post-Redirect-
}
```

Thứ tự luôn là: **bind** → **validate** (DataAnnotations tự chạy, điền ModelState) → **kiểm tra nghiệp vụ** (thêm lỗi vào ModelState) → nếu sai **trả lại view** chứa form; nếu đúng **ghi dữ liệu rồi redirect**. Trong Form.cshtml, tag helper hiện lỗi cạnh từng ô.

Chạy code mẫu và thử POST thiếu token → 400; sai dữ liệu → 200 kèm thông báo tiếng Việt và giá trị đã nhập; hợp lệ → 302 tới trang chi tiết. Chương 17 nói kỹ Post-Redirect-Get, chống CSRF, over-posting, tải file.

Xoá luôn là POST (không bao giờ GET): trình duyệt/crawler/bộ tải trước có thể gọi GET bất kỳ lúc nào (Chương 1). Mẫu có trang xác nhận (GET /SanPham/Delete/3) và POST thực hiện.

Xử lý lỗi

Ở môi trường thường: app.UseExceptionHandler("/Home/Loi") — hiện trang lỗi thân thiện (kèm mã theo dõi), **không** lộ stack trace. NotFound() trả 404; muốn trang 404 đẹp dùng app.UseStatusCodePagesWithReExecute("/Home/Loi", "?ma={0}").

Khi nào MVC, Razor Pages, Blazor?

	MVC	Razor Pages	Blazor
Đơn vị	controller + action + view (tách 3 nơi)	trang = file .cshtml + PageModel	component
Hợp với	ứng dụng lớn nhiều hành vi, dùng chung logic nhiều view; trả cả HTML lẫn API	trang theo tính năng, form CRUD, nội dung	giao diện tương tác cao, viết bằng C#
Mức "nghỉ thức"	nhiều hơn	ít hơn, code gần giao diện	khác hẳn (Chương 18)

Cả ba dùng chung nền tảng và có thể **cùng tồn tại trong một dự án**. Cho ứng dụng theo trang (đa số trang quản trị/CRUD), Microsoft khuyến nghị Razor Pages là điểm bắt đầu đơn giản; MVC vẫn phổ biến trong hệ thống có sẵn và nhiều đội yêu thích.

Lỗi thường gặp

- InvalidOperationException: The view 'X' was not found — sai tên/thư mục view; view không nằm trong Views/<Controller>/.
- InvalidOperationException: The model item passed into the ViewDataDictionary is of type 'A' but this ViewDataDictionary instance requires 'B' — truyền sai kiểu model.

- 400 khi POST form: thiếu token chống CSRF (form viết tay không dùng tag helper) hoặc ModelState không hợp lệ.
- Quên return sau ModelState lỗi, hoặc quên redirect sau POST (F5 gửi lại form).
- NullReferenceException trong view vì model null .
- Nhồi logic truy vấn/nghiệp vụ vào view hoặc controller.
- Xóa/sửa bằng GET .
- Dùng entity làm model của form (over-posting).

Bài tập

1. Thêm sắp xếp (sapXep=gia|ten) cho trang danh sách với liên kết trên tiêu đề cột.
2. Tách form thành partial view _SanPhamForm dùng lại ở nơi khác.
3. Viết view component SanPhamSapHet hiển thị 3 sản phẩm tồn thấp nhất ở thanh bên.
4. Thêm trang /Home/Loi khác nhau cho 404 bằng UseStatusCodePagesWithReExecute .
5. Thay SanPhamStore bằng EF Core + SQLite (Chương 12–14) mà **không sửa controller/view** — chỉ đổi cài đặt dịch vụ; ghi lại những gì phải đổi.

Chương 16 — Razor Pages

Mục tiêu học

Sau chương này, bạn sẽ:

- Dùng **Razor Pages** — mỗi trang là một cặp `.cshtml` + `PageModel` — để xây ứng dụng CRUD.
- Nắm **handler** (`OnGet`, `OnPost`, handler có tên), `[BindProperty]`, route của trang.
- Kết nối **EF Core**, dropdown (`SelectList`), partial dùng chung Create/Edit, Post-Redirect-Get.
- Phân biệt với MVC và biết khi nào chọn Razor Pages.

Code mẫu: [code/ch16-razor-pages/](#) — CRUD sản phẩm có nhóm, tìm kiếm, phân trang, dùng EF Core + SQLite.

Ý tưởng: một trang, một tệp lô-gic

Trong MVC (Chương 15) một chức năng "thêm sản phẩm" rải ở ba nơi: controller, view, ViewModel. **Razor Pages** gom về **một chỗ**: mỗi trang gồm

```
Pages/SanPhams/Create.cshtml      # giao diện (Razor)
Pages/SanPhams/Create.cshtml.cs   # PageModel: dữ liệu + các "handler" xử lý request
```

URL suy ra từ đường dẫn file: `Pages/SanPhams/Create.cshtml` → `/SanPhams/Create` ;
`Pages/Index.cshtml` → `/` . Không cần bảng route. Trang chỉ chạy khi có dòng `@page` ở đầu `.cshtml` (đó chính là thứ phân biệt Razor Page với view thường).

```
builder.Services.AddRazorPages();
...
app.MapRazorPages();
```

Với chức năng xoay quanh **trang** (form, danh sách, chi tiết), cách tổ chức này gọn và dễ tìm hơn MVC.

PageModel và handler

```
public class CreateModel(CuaHangDb db) : PageModel           // DI qua constructor
{
    [BindProperty] public SanPhamInput Input { get; set; } = new();
    public SelectList DanhSachNhom { get; private set; } = null!;
```

```

public async Task OnGetAsync() => await NapNhomAsync(); // GET /SanPhams/Creat

public async Task<IActionResult> OnPostAsync() // POST /SanPhams/Creat
{
    if (await db.SanPhams.AnyAsync(s => s.Ma == Input.Ma))
        ModelState.AddModelError("Input.Ma", "Mã sản phẩm đã tồn tại");
    if (!ModelState.IsValid) { await NapNhomAsync(); return Page(); } // hiện

    var sp = new SanPham { Ma = Input.Ma!, Ten = Input.Ten!.Trim(), Gia = Input.Gi
    db.SanPhams.Add(sp);
    await db.SaveChangesAsync();

    TempData["ThongBao"] = $"Đã thêm {sp.Ten}";
    return RedirectToPage("Details", new { id = sp.Id }); // Post-Redirect-Get
}
}

```

- **Handler** đặt tên `On<Verb>[Tên]Async` : `OnGet` , `OnPost` , `OnPostXoa` , `OnGetExport` ... Framework chọn theo phương thức HTTP và tham số `?handler=Tên` .
- Kiểu trả về: `void` / `Task` (hiển thị trang mặc định), hoặc `IActionResult` : `Page()` , `RedirectToPage(...)` , `NotFound()` , `File(...)` , `StatusCode(...)` .
- `PageModel` có `ModelState` , `TempData` , `User` , `Request` , `HttpContext` sẵn.

[BindProperty] — gán dữ liệu form

`[BindProperty]` cho phép framework **gán dữ liệu form vào thuộc tính** trước khi chạy handler `POST` (nên handler chỉ cần đọc `Input`). Mặc định **chỉ áp dụng khi POST**. Với trang tìm kiếm cần lấy từ query khi `GET` : `[BindProperty(SupportsGet = true)] public string? TuKhoa { get; set; } .`

Quan trọng về bảo mật: chỉ `[BindProperty]` cho kiểu **nhập liệu riêng** (`SanPhamInput`), **không** cho entity (`SanPham`) — nếu không kẻ tấn công thêm trường ngoài ý muốn vào form (`Id` , `GiaVon` , `IsAdmin`) và bị gán (over-posting). Ở handler `Edit` , tải entity thật rồi **chỉ gán các trường cho phép**.

Tham số từ route

```
@page "{id:int}" @* /SanPhams/Edit/5, id phải là số *@
```

```
public async Task<IActionResult> OnGetAsync(int id) { ... } // id lấy từ rout
```

Handler có tên — nhiều nút trên cùng trang

```
<form method="post" asp-page-handler="Xoa" asp-route-id="@sp.Id">
    <button type="submit" class="btn nguy-hiem">Xoá</button>
</form>
```

```
public async Task<IActionResult> OnPostXoaAsync(int id)
{
    await db.SanPhams.Where(s => s.Id == id).ExecuteDeleteAsync();
    TempData["ThongBao"] = "Đã xoá sản phẩm";
    return RedirectToPage(new { TuKhoa, Trang }); // giữ lại từ khoá/trang hi
}
```

Mỗi hàng của bảng có form POST riêng (xoá phải là POST + có token chống CSRF).

Giao diện: tag helper, partial, layout

Cùng Razor và tag helper như MVC (Chương 15), nhưng dùng `asp-page` thay `asp-controller/asp-action`:

```
<a asp-page="Details" asp-route-id="@sp.Id">@sp.Ten</a>           @* /SanPhams/Deta
<a asp-page="/SanPhams/Create">Thêm mới</a>                   @* đường dẫn tuy
<input asp-for="Input.Ma" />                                   @* name="Input.Ma
<select asp-for="Input.NhomId" asp-items="Model.DanhSachNhom"></select>
```

Form dùng chung Create/Edit bằng partial `_Form.cshtml`:

```
<form method="post">
    <partial name="_Form" for="Input" view-data="ViewData" />   @* for="Input": ti
    <button type="submit" class="btn">Lưu</button>
</form>
```

Danh sách chọn: dựng `SelectList` từ CSDL trong `PageModel` (`new SelectList(nhoms, nameof(Nhom.Id), nameof(Nhom.Ten), Input.NhomId)`), truyền vào `asp-items`. Nhớ **nạp lại** danh sách khi trả `Page()` sau lỗi validation, vì request POST không còn giữ nó.

`Pages/Shared/_Layout.cshtml`, `_ViewStart.cshtml`, `_ViewImports.cshtml` hoạt động như MVC (đặt trong `Pages/`). Với Razor Pages, `_ViewImports` thường có `@namespace WebRazor.Pages` và `@addTagHelper`.

Cẩn thận namespace: namespace của `PageModel` mặc định theo thư mục (`WebRazor.Pages.SanPhams`). Đặt tên thư mục trùng tên kiểu (`SanPham`) sẽ gây nhập

nhằng `WebRazor.Pages.SanPham` với entity `SanPham` — lý do mẫu dùng thư mục `SanPhams`.

Luồng CRUD hoàn chỉnh

Chức năng	Trang / handler	Kết quả
Danh sách + tìm + phân trang	Index · OnGetAsync	GET /SanPhams? TuKhoa=laptop&Trang=2
Xem chi tiết	Details · OnGetAsync(id)	404 nếu không có
Thêm	Create · OnGet/OnPost	lỗi → hiện lại form; thành công → 302 tới Details
Sửa	Edit · OnGet(id)/OnPost(id)	nạp Input từ entity; POST cập nhật
Xoá	Index · OnPostXoa(id)	302 về danh sách kèm thông báo

Thử code mẫu: tạo sản phẩm sai → thấy lỗi từng ô bằng tiếng Việt; trùng mã → "Mã sản phẩm đã tồn tại"; hợp lệ → `302 Location: /SanPhams/Details/5` và thông báo "Đã thêm ..." hiện đúng một lần (TempData).

Razor Pages hay MVC?

- Không có "đúng/sai". Hai cái **cùng chạy trong một ứng dụng**.
- Razor Pages hợp khi bạn nghĩ theo **trang** (mỗi URL một giao diện + xử lý riêng): CRUD, form, báo cáo, trang đăng nhập.
- MVC hợp khi controller phục vụ **nhều view** hoặc bạn cần tách bạch mạnh/tái sử dụng action; hoặc kết hợp trả HTML và JSON cùng controller.
- Kiến thức bên dưới (routing, DI, model binding, validation, tag helper, filter, xác thực) là **một**.

Lỗi thường gặp

- Quên dòng `@page` → trang trả `404` (được coi là view thường).
- `[BindProperty]` không nhận giá trị ở `GET` — cần `SupportsGet = true`.
- Quên nạp lại `SelectList` khi trả `Page()` sau lỗi → dropdown trống hoặc `NullReferenceException`.
- Tên trường lỗi sai tiền tố: `ModelState.AddModelError("Ma", ...)` không hiện cạnh `Input.Ma` — phải `"Input.Ma"`.
- `asp-page` sai đường dẫn tương đối/tuyệt đối.

- Bind entity thay vì kiểu `Input` (over-posting).
- Thiếu `TempData` giữ qua redirect (thông báo mất) hoặc đọc `TempData` nhiều lần (chỉ đọc được một lần).
- Handler trả `Page()` sau `POST` thành công (F5 gửi lại form) — phải redirect.

Bài tập

1. Thêm cột "Xoá mềm" (`DaXoa`) và query filter (Chương 13); trang Index chỉ hiện sản phẩm chưa xoá.
2. Thêm handler `OnPostTangTonAsync(int id)` để nút "+10 tồn" trên mỗi hàng (một câu `ExecuteUpdate`).
3. Thêm trang `Nhoms/Index` quản lý nhóm sản phẩm (CRUD) dùng lại mẫu của `SanPhams` .
4. Thêm sắp xếp qua `[BindProperty(SupportsGet = true)] string? SapXep` với whitelist (Chương 14).
5. So sánh: viết lại trang `Create` bằng MVC (Chương 15) và liệt kê số file/tầng khác nhau.

Chương 17 — Form, validation phía server và chống CSRF

Mục tiêu học

Sau chương này, bạn sẽ:

- Thiết kế form **đúng luồng**: bind → validate → nghiệp vụ → **Post-Redirect-Get**.
- Hiểu và chống **CSRF** bằng **antiforgery token** (cho form và cho Minimal API/JS).
- Phòng **over-posting**, xử lý **mật khẩu** đúng cách (băm, không lưu plaintext), **tải file lên** an toàn.
- Phân biệt validation phía client (UX) và phía server (bắt buộc).

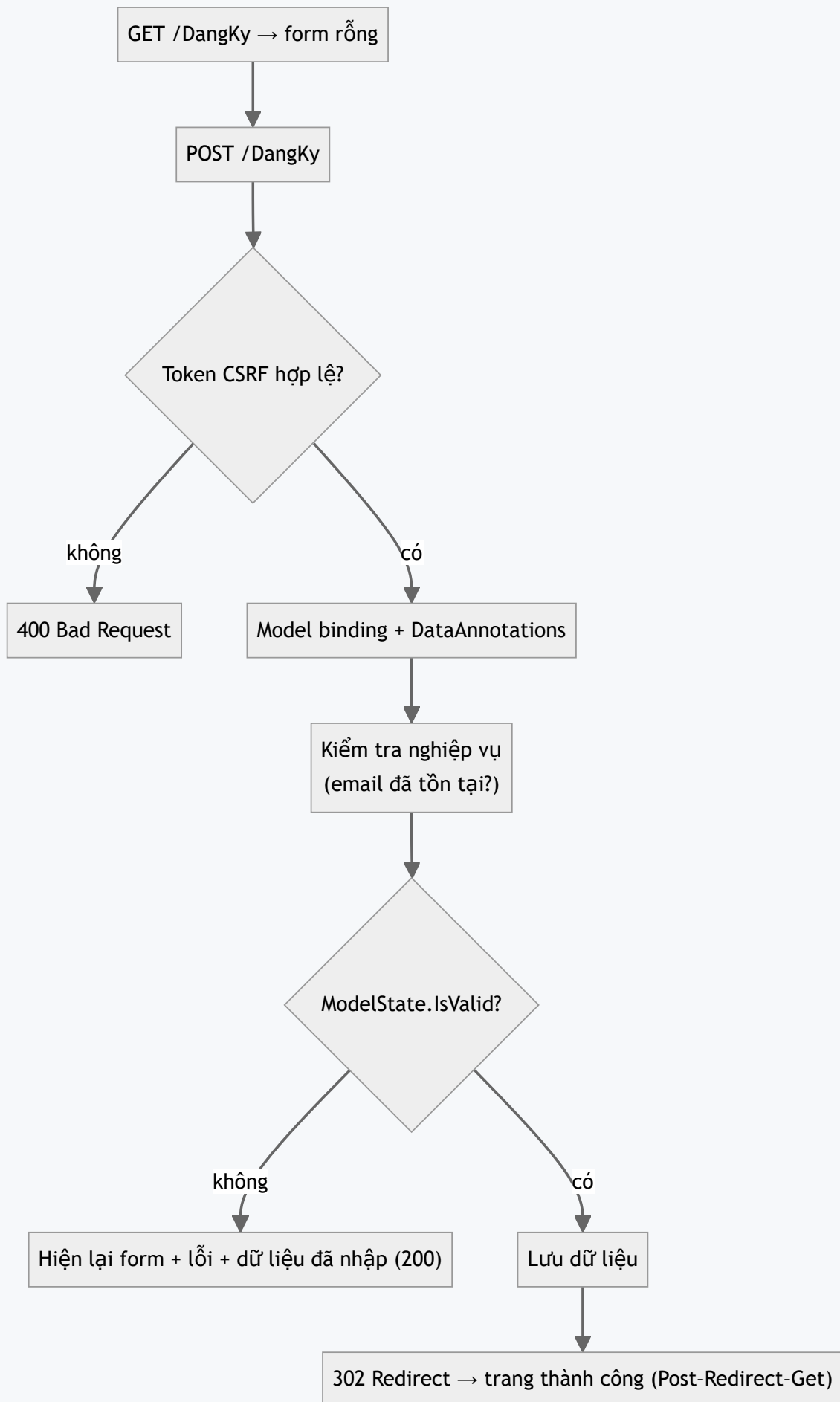
Code mẫu: [code/ch17-form-validation/](#) — form đăng ký (Razor Pages) và API tải ảnh (Minimal API).

Form là ranh giới không tin cậy

Mọi thứ đến từ form/query/header/file đều là **dữ liệu không tin cậy** — kẻ tấn công không dùng form của bạn, họ gửi request bằng `curl`. Vì vậy:

1. **Validate lại ở server** mọi thứ (client-side chỉ để tiện cho người dùng).
2. **Chỉ nhận đúng các trường** bạn cần.
3. **Mã hoá đầu ra** khi hiển thị (Razor tự làm).
4. **Không bao giờ dựa vào** phần tử ẩn, `disabled`, `maxlength` của HTML để bảo vệ.

Luồng chuẩn xử lý form



Post-Redirect-Get (PRG)

Nếu sau POST thành công bạn **trả luôn trang HTML**, trình duyệt ghi nhớ đó là kết quả của một POST ; người dùng nhấn **F5 (tải lại)** → trình duyệt hỏi "gửi lại biểu mẫu?" → đăng ký **hai lần** / đặt hàng **hai lần**. Giải pháp: sau POST thành công **redirect (302/303)** tới một trang GET :

```
return RedirectToPage("ThanhCong", new { id = nd.Id }); // 302 Location: /ThanhCon
```

F5 giờ chỉ tải lại trang GET vô hại. Thông báo "đã lưu" đi qua redirect bằng TempData (Chương 15). Khi validation **thất bại** thì *không* redirect: trả lại form (return Page()) để giữ dữ liệu người dùng đã nhập.

Validation phía server

```
public class DangKyInput
{
    [Required(ErrorMessage = "Vui lòng nhập họ tên")]
    [StringLength(80, MinimumLength = 2)]
    public string? HoTen { get; set; }

    [Required, EmailAddress] public string? Email { get; set; }

    [Required, TuoiToiThieu(16), DataType(DataType.Date)]
    public DateOnly? NgaySinh { get; set; }

    [Required, StringLength(64, MinimumLength = 8)]
    [RegularExpression(@"^(?=.*[A-Za-z])(?=.*\d).+$", ErrorMessage = "Mật khẩu phải có [DataType(DataType.Password)]
    public string? MatKhu { get; set; }

    [Compare(nameof(MatKhu), ErrorMessage = "Mật khẩu xác nhận không khớp")]
    public string? XacNhan { get; set; }

    [Range(typeof(bool), "true", "true", ErrorMessage = "Bạn phải đồng ý điều khoản")]
    public bool DongY { get; set; }
}
```

Điểm mới so với Chương 8:

- **[Compare]** đối chiếu hai trường (xác nhận mật khẩu).
- **Attribute tự viết có tham số** [TuoiToiThieu(16)] : tính tuổi từ DateOnly (chú ý cả tháng/ngày sinh chưa tới trong năm).
- **bool "phải tích"**: [Range(typeof(bool), "true", "true")] .
- DateOnly? / DateTime? nullable để phân biệt "chưa nhập" với giá trị mặc định.

- Checkbox HTML chỉ gửi khi được tích; tag helper thêm input ẩn `false` cùng tên nên bind về `bool` đúng (hai giá trị `true, false` khi tích).
- Thông báo lỗi tùy biến cả cho **lỗi bind kiểu** (chữ vào ô số) bằng localization (`ModelBindingMessageProvider`) — mặc định là tiếng Anh ("The field Giá must be a number").

Kết quả thực tế khi gửi form gần như rỗng:

```
Input.HoTen    → Vui lòng nhập họ tên
Input.Email    → Email không đúng định dạng
Input.NgaySinh → Bạn phải từ 16 tuổi trở lên
Input.MatKhau  → Mật khẩu phải có cả chữ và số
Input.XacNhan  → Mật khẩu xác nhận không khớp
Input.DongY    → Bạn phải đồng ý điều khoản
```

Kiểm tra nghiệp vụ

Chuyện cần dữ liệu ngoài (email đã dùng chưa) không hợp làm attribute. Viết trong handler và **gắn vào trường** để hiện cạnh ô:

```
if (!string.IsNullOrEmpty(Input.Email) && store.TonTaiEmail(Input.Email))
    ModelState.AddModelError("Input.Email", "Email này đã được đăng ký");
```

Tránh **user enumeration**: với form *đăng nhập/quên mật khẩu* đừng tiết lộ "email không tồn tại" — dùng thông báo chung. (Đăng ký bắt buộc phải báo trùng, nhưng nên giới hạn tốc độ — Chương 21.)

Validation phía client

`asp-for` sinh thuộc tính `data-val-*` (xem HTML nguồn: `data-val-required`, `data-val-regex-pattern` ...). Thư viện **jQuery Validation + Unobtrusive Validation** đọc chúng để báo lỗi **ngay trên trình duyệt**, không cần gửi lên server → trải nghiệm tốt hơn. Nó **tuỳ chọn** (thêm `<partial name="_ValidationScriptsPartial" />`), chỉ là tiện ích UX: server vẫn phải validate lại như trên. (Blazor có cơ chế riêng — Chương 18.)

Chống CSRF

CSRF (Cross-Site Request Forgery): kẻ tấn công dụ người dùng *đã đăng nhập* vào `bank.example` mở trang `evil.example`, trang đó có form ẩn tự gửi `POST bank.example/chuyen-tien`. Trình duyệt **tự đính kèm cookie đăng nhập** của `bank.example` → server tưởng là người dùng thật.

Antiforgery token chặn điều này: server phát một token ngẫu nhiên vừa đặt trong **cookie**, vừa nhúng vào **form ẩn**; khi `POST`, phải gửi *cả hai* và chúng phải khớp. Trang của kẻ tấn công

không đọc được token (same-origin policy) nên không giả mạo được.

- **Razor Pages:** tự động (kiểm tra mọi POST/PUT/DELETE mặc định). **MVC:** `[ValidateAntiForgeryToken]` hoặc filter toàn cục `AutoValidateAntiForgeryToken`.
- Tag helper `<form method="post">` **tự chèn** `<input type="hidden" name="__RequestVerificationToken" value="...">`. Form viết tay bằng HTML thuần thì **thiếu token** → **400** (thử POST thiếu token trong code mẫu).
- **Minimal API:** `app.UseAntiforgery()` (sau `UseRouting`, và sau xác thực nếu có); endpoint nhận `[FromForm]` **bắt buộc** token (mặc định). Cho JavaScript: lấy token qua `IAntiforgery.GetAndStoreTokens` và gửi trong header (mẫu dùng `X-CSRF-TOKEN`):

```
builder.Services.AddAntiforgery(o => o.HeaderName = "X-CSRF-TOKEN");
app.MapGet("/api/csrf", (IAntiforgery af, HttpContext ctx) => new { token = af.GetAndS
```

```
curl -c jar http://localhost:5000/api/csrf # nhận token + cookie
curl -b jar -X POST /api/tai-len -H "X-CSRF-TOKEN: <token>" -F tep=@anh.png
```

- **API xác thực bằng token trong header** **Authorization: Bearer** (JWT — Chương 20) **không** bị CSRF (trình duyệt không tự gắn header đó) nên tắt antiforgery được (`.DisableAntiforgery()`); nguy cơ chỉ có khi xác thực bằng **cookie**.
- Bổ sung: cookie `SameSite=Lax/Strict` (mặc định hiện đại), POST cho mọi thao tác đổi dữ liệu, không bao giờ để GET thay đổi trạng thái.

Over-posting (mass assignment)

Nếu action nhận **entity** (`SanPham` có `GiaVon`, `IsAdmin` ...), kẻ tấn công thêm `GiaVon=1` vào request và nó được gán. Quy tắc: **bind vào kiểu Input/ViewModel/DTO** chỉ chứa trường cho phép; hoặc dùng `[Bind("Ten,Gia")]` (whitelist, kém khuyến khích vì dễ sót). Khi sửa: tải entity thật và **chỉ gán các trường cho phép** từ Input.

Mật khẩu

Không bao giờ lưu mật khẩu dạng gốc, cũng không dùng SHA/MD5 thuần. Dùng hàm băm **chậm, có muối**: PBKDF2/bcrypt/scrypt/Argon2. [ASP.NET](#) Core có sẵn `PasswordHasher<T>` (PBKDF2 + salt ngẫu nhiên + tham số lặp, tự nâng cấp về sau):

```
var bam = new PasswordHasher<object>();
string luu = bam.HashPassword(new object(), matKhai); // 1
var kq = bam.VerifyHashedPassword(new object(), luu, matKhaiNhap); // Su
```

Trong thực tế bạn dùng **ASP.NET Core Identity** (Chương 20) đã gói sẵn việc này. Ngoài ra: tránh dán lại mật khẩu vào form khi lỗi (asp-for với type=password không tự điền lại), đặt autocomplete="new-password" , luôn dùng HTTPS, giới hạn số lần thử.

Tải file lên (upload)

```
app.MapPost("/api/tai-len", async ([FromForm] IFormFile? tep, IWebHostEnvironment env,
{
    if (tep is null || tep.Length == 0) return Results.BadRequest("Chua chon tep");
    if (tep.Length > 1_000_000) return Results.BadRequest("Tep toi da 1 MB");

    string duoi = Path.GetExtension(tep.FileName).ToLowerInvariant();
    if (duoi is not (".png" or ".jpg" or ".jpeg")) return Results.BadRequest("Chi nhan
    if (!KiemTraDauTep.LaAnh(tep)) return Results.BadRequest("Noi dung tep khong phai

    string tenLuu = $"{Guid.NewGuid():N}{duoi}"; // tên MỚI do
    await using var fs = File.Create(Path.Combine(thuMuc, tenLuu));
    await tep.CopyToAsync(fs, ct);
    return Results.Ok(new { tenGoc = Path.GetFileName(tep.FileName), tenLuu });
});
```

Danh sách kiểm tra bảo mật (mỗi mục ứng với một kiểu tấn công thật):

Rủi ro	Phòng
Tệp khổng lồ (DoS)	giới hạn MaxRequestBodySize của Kestrel + tep.Length
Tệp thực thi giả dạng ảnh (x.exe, .php, .html)	whitelist đuôi tệp
Đổi đuôi .png cho tệp độc hại	kiểm tra nội dung (magic bytes PNG 89 50 4E 47, JPEG FF D8 FF), tốt nhất xử lý lại ảnh bằng thư viện
Path traversal (../../evil)	không dùng tên client gửi ; đặt tên mới (GUID) và Path.GetFileName nếu phải hiển thị
Ghi đè tệp có sẵn	tên ngẫu nhiên
Chạy tệp đã tải	lưu ngoài thư mục web (wwwroot) hoặc ở blob storage; phục vụ qua endpoint có kiểm soát, đặt Content-Type cố định + X-Content-Type-Options: nosniff
Không tin Content-Type/tên tệp do client khai báo	như trên
Quét virus	với ứng dụng nhận tệp của người lạ

Từ form HTML: `<form method="post" enctype="multipart/form-data"><input type="file" name="tep"></form>`; với nhiều tệp `IFormFileCollection / List<IFormFile>`. Tệp lớn: stream thẳng xuống đĩa/kho thay vì đệm bộ nhớ.

Lỗi thường gặp

- Chỉ validate ở client.
- Trả trang HTML sau `POST` thành công (không PRG).
- Form tự viết thiếu token → `400`.
- Bind thẳng entity (over-posting).
- Lưu mật khẩu plaintext/hash thô; điền lại mật khẩu vào form khi có lỗi.
- Dùng `ModelState.AddModelError("Email", ...)` khi trường tên `Input.Email` — lỗi không hiện cạnh ô.
- Dùng tên tệp client gửi để lưu (path traversal); lưu tệp tải lên trong `wwwroot` cho phép chạy.
- Tiết lộ thông tin nhạy cảm qua thông báo lỗi ("email không tồn tại").
- Quên PRG khi validation thành công nhưng trả `Page()`.

Bài tập

1. Thêm ô "Số điện thoại" chỉ chấp nhận số điện thoại Việt Nam bằng attribute tự viết `[SoDienThoaiVn]`.
2. Cho phép tải ảnh đại diện ngay trong form đăng ký (`IFormFile` trong `DangKyInput`, `enctype="multipart/form-data"`), áp dụng đầy đủ danh sách kiểm tra ở trên.
3. Viết test tích hợp (Chương 10) cho `DangKy`: lấy token từ HTML bằng regex, gửi form sai/đúng, kiểm tra `302` tới `ThanhCong` và `400` khi thiếu token.
4. Cố tình tắt `UseAntiforgery`/đổi form không có token, quan sát và giải thích thông báo.
5. Thêm trang "Đăng nhập" dùng `PasswordHasher.VerifyHashedPassword`, và thông báo lỗi chung "Email hoặc mật khẩu không đúng".

Chương 18 — Blazor

Mục tiêu học

Sau chương này, bạn sẽ:

- Hiểu **Blazor** là gì và các **chế độ render** (Static SSR, Interactive Server, WebAssembly, Auto).
- Viết **component Razor**: `@code`, sự kiện `@onclick`, **data binding** `@bind`, tham số `[Parameter]`, `EventCallback`.
- Dùng **vòng đời** component, `@inject` dịch vụ, tải dữ liệu **bất đồng bộ**.
- Xây form với `EditForm` + validation (`DataAnnotations`), component con và danh sách với `@key`.

Code mẫu: [code/ch18-blazor/](#) — bộ đếm (sự kiện, binding, vòng đời) và trang quản lý sản phẩm.

Blazor là gì?

Với MVC/Razor Pages, mỗi thao tác (bấm nút, lọc) thường là **một request** tải lại trang; muốn giao diện mượt phải viết **JavaScript**. **Blazor** cho phép viết giao diện **tương tác** bằng **C# + Razor**: cùng ngôn ngữ, cùng kiểu dữ liệu, cùng thư viện (validation, DTO, dịch vụ) cho cả server và giao diện — không cần framework JavaScript riêng cho hầu hết nhu cầu.

Đơn vị xây dựng là **component** — một file `.razor` gồm HTML/Razor và khối `@code` (C#). Component có thể lồng nhau, nhận tham số, phát sự kiện, dùng lại. Cả trang cũng là component (có `@page`).

Chế độ render

Blazor Web App (từ .NET 8) cho **chọn từng trang/component** chạy theo cách nào:

Chế độ	Chạy ở đâu	Tương tác?	Đặc điểm
Static SSR (mặc định)	server, sinh HTML một lần	Không (như Razor Pages)	nhanh, nhẹ, SEO tốt; form gửi bằng POST
Interactive Server	server ; sự kiện qua kết nối SignalR (WebSocket)	Có	khởi động nhanh, code chạy trên server (truy cập CSDL trực tiếp); cần kết nối liên tục, mỗi người dùng tốn bộ nhớ server

Chế độ	Chạy ở đâu	Tương tác?	Đặc điểm
Interactive WebAssembly	trình duyệt (.NET chạy bằng WebAssembly)	Có	chạy offline, giảm tải server; tải lần đầu nặng hơn; gọi server qua API
Interactive Auto	server lúc đầu, chuyển sang WebAssembly khi đã tải xong	Có	cân bằng cả hai

Chọn bằng attribute: `@rendermode InteractiveServer` (trên component/trang) hoặc `AddInteractiveServerRenderMode()` khi map. Sách dùng **Interactive Server** vì đơn giản nhất (không cần dự án API riêng, truy cập dịch vụ thẳng).

```
builder.Services.AddRazorComponents().AddInteractiveServerComponents();
...
app.UseAntiforgery();
app.MapRazorComponents<App>().AddInteractiveServerRenderMode();
```

Cấu trúc: `App.razor` (khung HTML, nạp `blazor.web.js`) → `Routes.razor` (bộ định tuyến) → `MainLayout.razor` (bố cục) → các trang trong `Components/Pages`. Trang tương tác được **prerender** (HTML dựng sẵn từ server, hiện ngay) rồi "gắn" kết nối để trở nên tương tác.

Kết quả prerender kiểm chứng bằng `curl : GET /dem` trả HTML đã có `<h1>Bộ đếm</h1>` và Giá trị hiện tại: 0. Tính tương tác cần trình duyệt chạy `blazor.web.js`.

Component đầu tiên: bộ đếm

```
@page "/dem"
@rendermode InteractiveServer

<p>Giá trị hiện tại: <strong>@soDem</strong></p>

<button class="btn" @onclick="Tang">Tăng</button>
<button class="btn" @onclick="() => soDem -= buoc">Giảm</button>
<button class="btn" @onclick="Dat0" disabled="@ (soDem == 0)">Về 0</button>

@code {
    private int soDem;
    private int buoc = 1;

    private void Tang() => soDem += buoc;
```

```
private void Dat0() => soDem = 0;
}
```

- `@page "/dem"` biến component thành trang có URL.
- `@soDem` in giá trị biến/biểu thức; `@code { ... }` chứa trường, thuộc tính, phương thức C#.
- `@onclick="Tang"` gắn xử lý sự kiện (phương thức hoặc lambda). Sau khi xử lý, Blazor **tự render lại** component và chỉ gửi **phần khác biệt** của giao diện (diff) tới trình duyệt.
- Thuộc tính HTML điều kiện: `disabled="@ (soDem == 0)"` — `true` thì bật, `false` thì bỏ thuộc tính.
- Điều kiện/lặp trong Razor: `@if (...)` { }, `@foreach (...)` { }, `@switch` .

Nếu thay đổi dữ liệu **ngoài** luồng sự kiện của Blazor (timer, callback từ dịch vụ), gọi `StateHasChanged()` (và `await InvokeAsync(StateHasChanged)` từ luồng khác) để yêu cầu vẽ lại.

Data binding: @bind

Liên kết **hai chiều** giữa ô nhập và biến:

```
<input type="number" @bind="buoc" min="1" max="100" />           @* cập
<input @bind="chu" @bind:event="oninput" />                     @* cập
<p>Bạn đã gõ <strong>@chu.Length</strong> ký tự: <em>@chu.ToUpperInvariant()</em></p>

<input @bind="tuKhoa" @bind:event="oninput" @bind:after="TaiAsync" /> @* chạ
```

Kiểu dữ liệu tự chuyển đổi (`int` , `decimal` , `DateTime` , `bool` ...). `@bind:after` (mới) gọn hơn viết `@bind-Value:set` ; dùng cho "gõ đến đâu tìm đến đó". Với tìm kiếm gọi CSDL nên thêm **debounce** (đợi người dùng ngừng gõ ~300 ms) để tránh gọi liên tục.

Vòng đời component

Chạy theo thứ tự (chỉ những hàm bạn `override`):

Phương thức	Khi nào	Dùng để
<code>SetParametersAsync</code>	nhận tham số	hiếm khi ghi đề
<code>OnInitialized[Async]</code>	một lần lúc tạo	khởi tạo, tải dữ liệu ban đầu
<code>OnParametersSet[Async]</code>	mỗi khi tham số từ cha đổi	phản ứng theo tham số mới
<code>OnAfterRender[Async](bool firstRender)</code>	sau mỗi lần vẽ	gọi JavaScript, thao tác DOM (chỉ ở đây DOM đã tồn tại)

Phương thức	Khi nào	Dùng để
Dispose / IAsyncDisposable	khi bị gỡ	hủy đăng ký sự kiện, timer

```
protected override async Task OnInitializedAsync() => await TaiAsync(); // tải dữ
```

Lưu ý với **prerender**: `OnInitialized` chạy **hai lần** (một lần lúc prerender trên server, một lần khi kết nối tương tác) — tránh tác dụng phụ không lặp được (gửi email, trừ kho) trong đó.

Tải dữ liệu bất đồng bộ và trạng thái "đang tải"

```
@inject SanPhamService DichVu

@if (dangTai) { <p><em>Đang tải...</em></p> }
else if (danhSach.Count == 0) { <p><em>Không có sản phẩm nào.</em></p> }
else { ...bảng... }

@code {
    private bool dangTai = true;
    private List<SanPham> danhSach = [];

    private async Task TaiAsync()
    {
        dangTai = true;
        danhSach = await DichVu.TimAsync(tuKhoa); // trong lúc chờ, giao diện hi
        dangTai = false;
    }
}
```

@inject Kiểu Tên lấy dịch vụ từ DI (Chương 5) — giống tiêm constructor. Dịch vụ Scoped trong Blazor Server có vòng đời là **một mạch kết nối (circuit)** = một tab trình duyệt, không phải một HTTP request. `DbContext` nên lấy qua `IDbContextFactory<T>` (tạo ngắn hạn cho mỗi thao tác) thay vì Scoped dài hạn.

Component con: [Parameter] và EventCallback

Dữ liệu chảy **xuống** qua tham số, sự kiện chảy **lên** qua callback:

```
@* SanPhamRow.razor *@
<tr>
    <td>@SanPham.Ma</td><td>@SanPham.Ten</td>
    <td><button class="btn nguy-hiem" @onclick="()" => OnXoa.InvokeAsync(SanPham.Id)">X
</td></tr>
```

```
@code {
    [Parameter, EditorRequired] public SanPham SanPham { get; set; } = default!;
    [Parameter] public EventCallback<int> OnXoa { get; set; }
}
```

```
@* trang cha *@
@foreach (var sp in danhSach)
{
    <SanPhamRow @key="sp.Id" SanPham="sp" OnXoa="XoaAsync" />
}
```

- `[Parameter]` phải là property `public` ; **đừng gán** vào tham số bên trong component con (cha sẽ ghi đè khi vẽ lại).
- `EventCallback<T>` là kiểu callback chuẩn: tự vẽ lại component cha sau khi gọi (khác `Action<T>`).
- `@key` giúp Blazor nhận đúng dòng nào là dòng nào khi danh sách đổi thứ tự/xoá — tránh giữ nhầm trạng thái của dòng khác.
- Ngoài ra: `RenderFragment ( ChildContent )` để nhận nội dung bên trong thẻ; `CascadingParameter` truyền dữ liệu xuống mọi con cháu (theme, người dùng); component **generic** (`@typeparam T`).

Form và validation: EditForm

```
<EditForm Model="form" OnValidSubmit="ThemAsync" FormName="them-san-pham">
    <DataAnnotationsValidator />
    <ValidationSummary />

    <label>Mã <InputText @bind-Value="form.Ma" /></label>
    <ValidationMessage For="() => form.Ma" />

    <label>Giá <InputNumber @bind-Value="form.Gia" /></label>
    <ValidationMessage For="() => form.Gia" />

    <button type="submit">Thêm</button>
</EditForm>
```

- `Model` là lớp có **DataAnnotations** — **chính các attribute** đã dùng ở Chương 8/17 (`[Required]` , `[Range]` , `[RegularExpression]`). Cùng một lớp validation dùng lại cho API, Razor Pages, Blazor.
- `<DataAnnotationsValidator />` chạy validation; `OnValidSubmit` chỉ gọi khi hợp lệ, `OnInvalidSubmit` khi không.

- Component `Input*`: `InputText`, `InputNumber`, `InputDate`, `InputCheckbox`, `InputSelect`, `InputTextArea`, `InputFile` (tải file). Chúng tự thêm CSS `valid / invalid`.
- Lỗi nghiệp vụ từ dịch vụ (mã trùng): bắt exception, hiện cảnh form (như `loi` trong mẫu) hoặc dùng `ValidationMessageStore` để gắn vào một trường.
- Với Static SSR, form cần `FormName` và `[SupplyParameterFromForm]`; với chế độ tương tác thì không.

Điều hướng và tham số route

```
@page "/san-pham/{Id:int}"
@inject NavigationManager Nav

@code {
    [Parameter] public int Id { get; set; }
    void VeDanhSach() => Nav.NavigateTo("/san-pham");
}
```

Dùng `<NavLink href="...">` cho menu (tự thêm class `active`), `Nav.NavigateTo(...)` để chuyển trang bằng code, `[SupplyParameterFromQuery]` để lấy tham số query.

Gọi JavaScript khi cần (JS interop)

Blazor không loại bỏ JavaScript hoàn toàn: cần API trình duyệt (clipboard, localStorage, thư viện biểu đồ) thì dùng `IJSRuntime`:

```
@inject IJSRuntime JS
await JS.InvokeVoidAsync("localStorage.setItem", "khoa", "gia_tri");
var w = await JS.InvokeAsync<int>("eval", "window.innerWidth"); // ví dụ minh họa
```

Chỉ gọi JS sau khi component đã vẽ (`OnAfterRenderAsync`). Với Blazor Server, mỗi lệnh gọi là một vòng qua mạng.

Blazor Server hay WebAssembly? Blazor hay React/Angular?

- **Server**: ứng dụng nội bộ/quản trị, cần truy cập dữ liệu thẳng, muốn khởi động nhanh, ít cần scale hàng chục nghìn kết nối đồng thời (mỗi kết nối giữ trạng thái trên server).
- **WebAssembly**: ứng dụng cần chạy offline, giảm tải server, đã có API; chấp nhận bản tải xuống lớn hơn ban đầu. Code chạy **trên máy người dùng** — không được nhét bí mật vào đó.
- **Auto**: kết hợp.
- So với SPA JavaScript (React/Angular/Vue): Blazor hợp khi đội mạnh C#, muốn chia sẻ code, không muốn duy trì hai hệ sinh thái; hệ sinh thái thư viện giao diện của JS lớn hơn

nhiều, vì vậy nhiều đội vẫn chọn JS cho giao diện phức tạp, dùng [ASP.NET](#) Core làm API (Phần 2–3).

Lỗi thường gặp

- Component không tương tác: quên `@rendermode` (mặc định Static SSR) hoặc quên `AddInteractiveServerComponents/RenderMode`.
- Giao diện không cập nhật vì đổi dữ liệu ngoài luồng sự kiện mà không gọi `StateHasChanged / InvokeAsync`.
- Gọi JS trong `OnInitialized` (chưa có DOM) — dùng `OnAfterRenderAsync`.
- Tác dụng phụ trong `OnInitialized` bị chạy 2 lần do prerender.
- Gán vào `[Parameter]` bên trong component con.
- Quên `@key` cho danh sách động → trạng thái nhập lẫn dòng.
- Giữ dữ liệu người dùng trong dịch vụ **Singleton** (lẫn giữa mọi người dùng) — dùng `Scoped` (theo circuit).
- `DbContext` `Scoped` sống suốt circuit → lỗi đồng thời/dữ liệu cũ; dùng `IDbContextFactory`.
- Mất kết nối: người dùng thấy giao diện "đơ" — cấu hình UI kết nối lại và chấp nhận nó là đặc tính của Blazor Server.
- Bỏ qua bảo mật: code chạy trên server vẫn phải kiểm tra quyền cho mọi thao tác (Chương 20).

Bài tập

1. Thêm nút "Sao chép mã" dùng JS interop (`navigator.clipboard.writeText`) trong `SanPhamRow`.
2. Thêm debounce 300 ms cho ô tìm kiếm bằng `System.Timers.Timer` hoặc `CancellationTokenSource`.
3. Thêm trang `/san-pham/{id:int}` hiển thị chi tiết và nút quay lại bằng `NavigationManager`.
4. Viết component `TheThongBao` nhận `ChildContent` và tham số `Loai` ("thanh-cong"/"loi") để hiển thị thông báo.
5. Thay `SanPhamService` bằng dịch vụ dùng EF Core + `IDbContextFactory<T>`; giải thích vì sao không dùng `DbContext` `Scoped` ở đây.

Chương 19 — Gọi API ngoài với HttpClient

Mục tiêu học

Sau chương này, bạn sẽ:

- Gọi API bên ngoài đúng cách bằng `IHttpClientFactory` và **typed client** — tránh cạm bẫy cổ điển của `new HttpClient()`.
- Thêm hành vi xuyên suốt bằng `DelegatingHandler` (API key, log).
- Làm chương trình **chịu lỗi** với **resilience**: retry, timeout, circuit breaker.
- Xử lý `HttpRequestException`, timeout, huỷ (`CancellationToken`) và ánh xạ sang mã HTTP của bạn (`502` , `504`).

Code mẫu: [code/ch19-httpclient/](#) — ứng dụng tự chứa một "dịch vụ thời tiết bên ngoài" hay lỗi để bạn thấy retry/timeout hoạt động.

Vì sao phải gọi API khác?

Ứng dụng thật hiếm khi đứng một mình: cổng thanh toán, gửi email/SMS, bản đồ, dịch vụ nội bộ khác (microservices). **Mạng không đáng tin**: chậm, mất gói, dịch vụ bên kia tạm sập, quá tải. Mã gọi ngoài phải giả định **lỗi là chuyện bình thường** ("*fallacies of distributed computing*").

Cạm bẫy của `new HttpClient()`

```
// X SAI khi làm trong mỗi request
using var http = new HttpClient();
var s = await http.GetStringAsync(url);
```

- Mỗi `HttpClient` mới mở **socket mới**; `Dispose` không trả socket ngay (giữ ở trạng thái `TIME_WAIT`) → dưới tải cao dẫn tới **cạn kiệt cổng (socket exhaustion)**.
- Ngược lại, giữ **một `HttpClient` static mãi mãi** thì **không cập nhật DNS** (khi dịch vụ đổi địa chỉ IP, ứng dụng vẫn nói chuyện với IP cũ).

Giải pháp chuẩn: `IHttpClientFactory` — quản lý vòng đời và **gộp/luân phiên các kết nối** (pool `HttpMessageHandler`, làm mới định kỳ), đồng thời cho phép cấu hình tập trung.

Ba cách dùng factory

1. Typed client (khuyến nghị)

```

builder.Services.AddHttpClient<ThoiTietClient>(c =>
{
    c.BaseAddress = new Uri(baseUrl);
    c.Timeout = TimeSpan.FromSeconds(10);
    c.DefaultRequestHeaders.Add("User-Agent", "CuaHangApp/1.0");
});

public class ThoiTietClient(HttpClient http) // HttpClient được ti
{
    public async Task<ThoiTiet?> LayAsync(string thanhPho, CancellationToken ct)
    {
        var res = await http.GetAsync($"ngoai/thoi-tiet/{Uri.EscapeDataString(thanhPh
        res.EnsureSuccessStatusCode(); // không phải 2xx → H
        return await res.Content.ReadFromJsonAsync<ThoiTiet>(ct);
    }
}

app.MapGet("/thoi-tiet/{tp}", async (string tp, ThoiTietClient client, CancellationTok

```

Gói toàn bộ chi tiết HTTP (URL, tên header, JSON) vào **một lớp có tên nghiệp vụ** (`ThoiTietClient`) — phần còn lại của ứng dụng chỉ thấy `LayAsync(...)`. Để test (mock lớp này), dễ đổi nhà cung cấp.

2. Named client

```

builder.Services.AddHttpClient("tran", c => c.BaseAddress = new Uri(baseUrl));
var http = factory.CreateClient("tran"); // IHttpClientFactory

```

3. Basic: `factory.CreateClient()` không cấu hình

Dùng khi chỉ cần client mặc định. Tốt hơn `new HttpClient()`.

Gửi request và đọc kết quả

```

await http.GetFromJsonAsync<ThoiTiet>("/ngoai/thoi-tiet/hanoi", ct); // GET
await http.PostAsJsonAsync("/api/don-hang", new { kháchHangId = 1 }, ct); // POS
await http.PutAsJsonAsync(...); await http.DeleteAsync(...);

using var req = new HttpRequestMessage(HttpMethod.Post, "/x") { Content = JsonContent.
req.Headers.Authorization = new("Bearer", token); // head
var res = await http.SendAsync(req, ct);

```

```
res.StatusCode; res.IsSuccessStatusCode; res.Headers; res.Content.Headers
await res.Content.ReadAsStringAsync(ct); await res.Content.ReadFromJsonAsync<T>(ct);
```

Lưu ý:

- **Kiểm tra mã trạng thái:** `HttpClient` **không** ném lỗi khi server trả 404/500 — chỉ ném khi không kết nối được. Dùng `EnsureSuccessStatusCode()` hoặc tự đọc `StatusCode`.
- **Mã hoá tham số** đưa vào URL: `Uri.EscapeDataString(x)` (hoặc `QueryHelpers.AddQueryString`). Đừng nối chuỗi thô từ dữ liệu người dùng (chèn tham số, SSRF).
- **Giải phóng** `HttpResponseMessage` bằng `using` khi bạn dùng `SendAsync` trực tiếp; các helper `GetFromJsonAsync` đã lo.
- Với body lớn dùng `HttpCompletionOption.ResponseHeadersRead` và đọc stream.
- Luôn truyền `CancellationToken` (từ request) — khi client huỷ, cuộc gọi ra ngoài cũng dừng.

DelegatingHandler — "middleware" của HttpClient

Cũng như pipeline của [ASP.NET Core](#) (Chương 3), request đi ra qua chuỗi handler:

```
public class ApiKeyHandler(IConfiguration cfg, ILogger<ApiKeyHandler> log) : DelegatingHandler
{
    protected override async Task<HttpResponseMessage> SendAsync(HttpRequestMessage request)
    {
        request.Headers.Add("X-API-Key", cfg["ThoiTiet:ApiKey"] ?? "khoa-thu-nghiem");
        var res = await base.SendAsync(request, ct); // chuyển tiếp
        log.LogInformation("Goi {Method} {Uri} -> {Status}", request.Method, request.RequestUri, res.StatusCode);
        return res;
    }
}

builder.Services.AddTransient<ApiKeyHandler>();
builder.Services.AddHttpClient<ThoiTietClient>().AddHttpClientHandler<ApiKeyHandler>()
```

Dùng cho: gắn token xác thực, correlation id, log, đo thời gian, làm mới token khi 401. Bí mật (API key) lấy từ cấu hình/kho bí mật (Chương 6), không viết cứng.

Resilience: sống chung với lỗi

Lỗi mạng chia hai loại:

- **Tạm thời (transient):** 503, 429, 408, mất kết nối, timeout — *thử lại là có thể thành công*.
- **Vĩnh viễn:** 400, 401, 404, 500 do bug — thử lại vô ích.

Gói `Microsoft.Extensions.Http.Resilience` (xây trên **Polly**) cung cấp bộ chiến lược dựng sẵn:

```
builder.Services.AddHttpClient<ThoiTietClient>(...)
    .AddStandardResilienceHandler(o =>
    {
        o.Retry.MaxRetryAttempts = 3;
        o.Retry.Delay = TimeSpan.FromMilliseconds(100);
        o.AttemptTimeout.Timeout = TimeSpan.FromSeconds(1);           // mỗi LẦN THỬ
        o.TotalRequestTimeout.Timeout = TimeSpan.FromSeconds(6);      // cả chuỗi thử
        o.CircuitBreaker.SamplingDuration = TimeSpan.FromSeconds(10); // ≥ 2 × Attempt
    });
```

`AddStandardResilienceHandler` xếp sẵn (từ ngoài vào trong): **giới hạn đồng thời (rate limiter)** → **timeout tổng** → **retry** → **circuit breaker** → **timeout mỗi lần thử**.

- **Retry** (thử lại) với **backoff lũy thừa + jitter** (giãn dần và ngẫu nhiên hoá để tránh mọi client thử lại cùng lúc — "thundering herd").
- **Timeout**: không bao giờ chờ vô hạn. Mặc định của `HttpClient` là 100 giây — quá dài với API người dùng đang chờ.
- **Circuit breaker** (cầu dao): khi tỉ lệ lỗi cao vượt ngưỡng, **ngắt mạch** — các lệnh gọi sau *thất bại ngay* (`BrokenCircuitException`) trong một khoảng thời gian, cho dịch vụ kia hồi phục thay vì dồn thêm tải vào nó, và giúp ứng dụng của bạn *nhANH chóng báo lỗi* thay vì treo.

Kết quả chạy code mẫu: dịch vụ giả trả `503` cho hai lần gọi đầu:

```
GET /thoi-tiet/hanoi          → 200 sau ~0.9 s   (log: OnRetry, OnRetry, rồi thành công)
GET /thoi-tiet-tran/hanoi     → 503          (client không có resilience: lỗi ngay)
GET /cham (dịch vụ chậm 3 s)  → 504 sau ~4.6 s   (mỗi lần thử bị cắt ở 1 s; thử 4 lần;)
```

Lưu ý quan trọng về retry:

- Chỉ retry thao tác **idempotent** (`GET`, `PUT`, `DELETE`). Retry `POST` tạo đơn hàng/thanh toán có thể tạo **hai lần** — cần **khoá idempotency** (header `Idempotency-Key` do client sinh; server lưu và trả lại kết quả cũ cho khoá trùng — Tập 3).
`AddStandardResilienceHandler` mặc định retry cả `POST`; với API không idempotent hãy tắt retry (`o.Retry.DisableForUnsafeHttpMethods()`) hoặc dùng `AddStandardHedgingHandler` cho `GET`.
- Retry nhân đôi tải lên dịch vụ đang lỗi — cần đi cùng circuit breaker và giới hạn số lần.
- Tổng thời gian chờ nhân lên ($1\text{ s} \times 4\text{ lần} + \text{trễ giữa các lần} \approx 4,6\text{ s}$): đặt `TotalRequestTimeout` để có trần cứng.

Chuyển lỗi ngoài thành lỗi của bạn

Client gọi API bên kia → bên kia lỗi → **bạn** là "cổng" (gateway). Chuẩn HTTP có mã riêng:

Tình huống	Mã bạn trả
Không kết nối được / bên kia trả lỗi 5xx	502 Bad Gateway
Bên kia phản hồi quá chậm (timeout)	504 Gateway Timeout
Cầu dao đang mở / quá tải	503 Service Unavailable (kèm Retry-After)

```
try { return Results.Ok(await client.LayAsync(tp, ct)); }
catch (HttpRequestException e)
{ return Results.Problem($"Dịch vụ tạm thời lỗi: {e.StatusCode}", statusCode: 502); }
catch (OperationCanceledException) when (!ct.IsCancellationRequested) // hết
{ return Results.Problem("Dịch vụ tạm thời phản hồi quá chậm", statusCode: 504); }
```

Phân biệt **hết giờ** với **client tự hủy** bằng `ct.IsCancellationRequested`. Không lộ chi tiết nội bộ (URL, khoá) trong thông báo; log đầy đủ ở server (Chương 9). Nên gom xử lý này vào `ExceptionHandler` chung thay vì lặp ở từng endpoint.

Phương án dự phòng (fallback) và bộ nhớ đệm

Khi dịch vụ ngoài lỗi, đôi khi trả **dữ liệu cũ đã cache** hoặc giá trị mặc định tốt hơn là báo lỗi (thời tiết cũ 10 phút vẫn hữu ích). `IMemoryCache` / `HybridCache` (Tập 3) + `resilience Fallback`. Quyết định theo nghiệp vụ: thanh toán không được "đoán", còn hiển thị thời tiết thì được.

Kiểm thử code gọi HTTP

Đừng gọi dịch vụ thật trong test. Thay `HttpMessageHandler` bằng bản giả trả kết quả định sẵn:

```
class HandlerGia(HttpStatusCode ma, string body) : HttpMessageHandler
{
    protected override Task<HttpResponseMessage> SendAsync(HttpRequestMessage r, Cance
        => Task.FromResult(new HttpResponseMessage(ma) { Content = new StringContent(b
    }
    var client = new ThoiTietClient(new HttpClient(new HandlerGia(HttpStatusCode.OK, "{...
```

Hoặc trong `WebApplicationFactory` (Chương 10) thay đăng ký typed client bằng `ConfigurePrimaryHttpMessageHandler`.

Lỗi thường gặp

- `new HttpClient()` trong mỗi request (cạn socket) hoặc static vĩnh viễn (DNS cũ).
- Quên `EnsureSuccessStatusCode()` → xử lý body lỗi như dữ liệu hợp lệ.

- Không đặt timeout/không dùng `CancellationToken` → yêu cầu treo, tích tụ luồng.
- Retry thao tác không idempotent (`POST` thanh toán) → thanh toán hai lần.
- Retry không giới hạn/không backoff → tự "DDoS" dịch vụ đang yếu.
- Nối chuỗi dữ liệu người dùng vào URL không mã hoá; cho người dùng nhập URL tùy ý để server gọi (**SSRF** — kẻ tấn công bắt server truy cập mạng nội bộ; dùng whitelist host).
- Log/ trả ra ngoài header xác thực, token.
- Cấu hình `AttemptTimeout / SamplingDuration` mâu thuẫn → ném lỗi validation khi khởi động.
- Quên đăng ký `DelegatingHandler` là `Transient` .

Bài tập

1. Thêm `Task<IReadOnlyList<ThoiTiet>> LayNhiềuAsync(IEnumerable<string> thanhPho, ...)` gọi song song bằng `Task.WhenAll` có giới hạn đồng thời (`SemaphoreSlim`).
2. Đặt `Retry.DisableForUnsafeHttpMethods()` và thử với endpoint `POST` — giải thích lý do.
3. Thêm `IMemoryCache` để nếu dịch vụ lỗi thì trả kết quả gần nhất (đánh dấu `"cu": true`).
4. Viết unit test cho `ThoiTietClient` bằng `HandlerGia` : trường hợp `200` , `404` , JSON sai.
5. Gọi `/ngoai/loi-vinh-vien` nhiều lần liên tiếp và quan sát circuit breaker mở (`BrokenCircuitException`) — ghi lại số lần gọi tới khi mạch mở.

Chương 20 — Xác thực và phân quyền

Mục tiêu học

Sau chương này, bạn sẽ:

- Phân biệt **xác thực (authentication)** và **phân quyền (authorization)**, và **401 vs 403**.
- Cài xác thực bằng **cookie** (ứng dụng web) và **JWT Bearer** (API/mobile/SPA).
- Dùng **claims, roles, policy**, và **resource-based authorization**.
- Lưu mật khẩu đúng cách; biết vai trò của [ASP.NET Core Identity](#) và **OpenID Connect**.

Code mẫu: [code/ch20-xac-thuc/](#) — một ứng dụng chạy cả hai cơ chế, đã kiểm tra bằng `curl` (mọi kết quả trong chương lấy từ lần chạy thật).

Hai câu hỏi khác nhau

	Xác thực (Authentication)	Phân quyền (Authorization)
Câu hỏi	Bạn là ai?	Bạn được làm gì?
Kết quả	danh tính (<code>ClaimsPrincipal</code>)	cho phép / từ chối
Thất bại →	401 Unauthorized (chưa biết bạn là ai)	403 Forbidden (biết rồi nhưng không đủ quyền)
Middleware	<code>UseAuthentication()</code>	<code>UseAuthorization()</code>

```
app.UseAuthentication();    // 1) xác định danh tính → HttpContext.User
app.UseAuthorization();     // 2) kiểm tra quyền theo policy của endpoint    (thứ tự)
```

Danh tính, claim, role

Danh tính trong .NET là `ClaimsPrincipal` chứa một hay nhiều `ClaimsIdentity`, mỗi identity là tập **claim** (khẳng định về người dùng: "tên là an", "vai trò là Admin", "tuổi là 20", "email...").

```
var claims = new List<Claim>
{
    new(ClaimTypes.NameIdentifier, "2"),
    new(ClaimTypes.Name, "an"),
    new(ClaimTypes.Role, "User"),
    new("tuoi", "20"),
};
var identity = new ClaimsIdentity(claims, authenticationType: "Cookies", ClaimTypes.Na
```

Trong endpoint: `ClaimsPrincipal user` (tham số handler) → `user.Identity.Name`, `user.IsInRole("Admin")`, `user.FindFirstValue("tuoi")`. **Role** chỉ là một loại claim đặc biệt (`ClaimTypes.Role`).

Mật khẩu

Không lưu mật khẩu gốc; dùng băm chậm có muối (Chương 17):

```
var bam = new PasswordHasher<object>();
string luu = bam.HashPassword(new object(), "Admin@123");
bam.VerifyHashedPassword(new object(), luu, matKhuNhap) != PasswordVerificationResult
```

Hai chi tiết bảo mật đã có trong `TaiKhoanStore.XacThuc`:

- **Thông báo lỗi chung** "Tên đăng nhập hoặc mật khẩu không đúng" cho cả hai trường hợp (không tiết lộ tài khoản nào tồn tại).
- Khi tên đăng nhập **không tồn tại**, vẫn thực hiện một phép băm giả để **thời gian phản hồi giống nhau** (chống dò tài khoản qua *timing attack*).

Xác thực bằng cookie (ứng dụng web có trình duyệt)

```
builder.Services.AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme)
    .AddCookie(o =>
    {
        o.Cookie.HttpOnly = true; // JavaScript không đọc được
        o.Cookie.SameSite = SameSiteMode.Lax; // giảm CSRF
        o.Cookie.SecurePolicy = CookieSecurePolicy.Always; // production: chỉ gửi qu
        o.ExpireTimeSpan = TimeSpan.FromMinutes(30);
        o.SlidingExpiration = true; // hoạt động thì gia hạn
        o.Events.OnRedirectToLogin = ctx => { ctx.Response.StatusCode = 401; return Ta
    });

// Đăng nhập
await ctx.SignInAsync(CookieAuthenticationDefaults.AuthenticationScheme, new ClaimsPri
// Đăng xuất
await ctx.SignOutAsync(CookieAuthenticationDefaults.AuthenticationScheme);
```

Cách hoạt động: sau khi xác thực thành công, server **mã hoá và ký** danh tính vào một **cookie**; mỗi request sau trình duyệt tự gửi cookie, middleware giải mã dựng lại `User`. Không có "session id tra cứu trên server" (stateless, chỉ cần khoá bảo vệ dữ liệu — Data Protection, Chương 21).

Kết quả thử thực tế:

GET /toi (chưa đăng nhập)	→ 401
POST /dang-nhap (sai mật khẩu)	→ 401
POST /dang-nhap (an)	→ 200, Set-Cookie
GET /toi	→ {"ten":"an","vaiTro":["User"],"tuoi":"20",...}
GET /quan-tri (an, role User)	→ 403
POST /dang-xuat rồi GET /toi	→ 401

Với cookie, form đăng nhập phải có **chống CSRF** (Chương 17) — mẫu tắt bằng `.DisableAntiforgery()` chỉ để thử bằng `curl`.

Xác thực bằng JWT (API, mobile, SPA)

Khi client không phải trình duyệt dùng cookie (app di động, SPA gọi API khác miền, dịch vụ gọi dịch vụ), dùng **token** gửi trong header:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
```

JWT (JSON Web Token) là chuỗi 3 phần `header.payload.signature` (mỗi phần Base64URL): `payload` chứa claim (`sub`, `name`, `role`, `exp` ...), chữ ký chứng minh **không bị sửa**. Server không lưu phiên: chỉ cần kiểm tra chữ ký và hạn.

```
builder.Services.AddAuthentication()
    .AddJwtBearer(o => o.TokenValidationParameters = new TokenValidationParameters
    {
        ValidateIssuer = true, ValidIssuer = "cuahang-api",
        ValidateAudience = true, ValidAudience = "cuahang-client",
        ValidateLifetime = true, ClockSkew = TimeSpan.FromSeconds(30),
        ValidateIssuerSigningKey = true, IssuerSigningKey = khoaKy,
    });

// Cấp token sau khi xác thực thông tin đăng nhập
var token = new JwtSecurityToken(issuer: "cuahang-api", audience: "cuahang-client", cl
    expires: DateTime.UtcNow.AddMinutes(15), // NGẮN hạn
    signingCredentials: new SigningCredentials(khoaKy, SecurityAlgorithms.HmacSha256))
return new { accessToken = new JwtSecurityTokenHandler().WriteToken(token) };
```

GET /api/toi (không token)	→ 401
GET /api/toi (token bị sửa 1 ký tự)	→ 401
GET /api/toi (token hợp lệ)	→ {"ten":"an","vaiTro":["User"]}
GET /api/admin/bao-cao (token an)	→ 403
GET /api/admin/bao-cao (token admin)	→ 200

Payload giải mã (Base64) của token thật:

```
{"/nameidentifier":"2","/name":"an","tuoi":"20","/role":"User","nbf":1789895649,"ex
```

Payload không được mã hoá, chỉ được ký — ai cũng đọc được. Không đặt thông tin bí mật vào JWT.

Nguyên tắc JWT an toàn:

1. **Khoá ký** ≥ 256 bit, ngẫu nhiên, **lưu trong kho bí mật/biến môi trường** (Chương 6), không commit; đổi định kỳ. Với hệ thống nhiều dịch vụ dùng **chữ ký bất đối xứng** (RS256/ES256: dịch vụ cấp token giữ khoá riêng, các dịch vụ khác chỉ cần khoá công khai).
2. **Access token ngắn hạn** (5–15 phút) + **refresh token** dài hạn, lưu phía server (có thể thu hồi), xoay vòng mỗi lần dùng; hoặc dùng nhà cung cấp danh tính (bên dưới).
3. Luôn **kiểm tra iss, aud, exp**, ký hiệu thuật toán cố định (chống tấn công alg: none).
4. Nơi lưu token phía trình duyệt: `localStorage` dễ bị XSS đánh cắp; cookie `HttpOnly` an toàn hơn cho trình duyệt. Với web app thuần, **cookie thường là lựa chọn tốt hơn JWT**.
5. Token đã phát **không thu hồi được** trước khi hết hạn nếu chỉ dựa vào chữ ký — nên hạn ngắn, hoặc có danh sách thu hồi/ `jt`.

Cookie hay JWT?

	Cookie	JWT Bearer
Client	trình duyệt (tự động gửi)	bất kỳ (JS/mobile/dịch vụ) gắn header
Rủi ro chính	CSRF (đã có antiforgery/SameSite)	XSS đánh cắp token nếu lưu ở JS
Thu hồi	dễ (xoá phiên/đổi khoá)	khó (chờ hết hạn/danh sách đen)
Cross-domain	khó hơn	dễ
Hợp với	ứng dụng web server-render, Blazor	API công khai, mobile, microservice

Có thể **dùng cả hai** trong một ứng dụng, mỗi endpoint chọn scheme:

```
RequireAuthorization(new AuthorizeAttribute { AuthenticationSchemes = "Bearer" })
```

(nhóm `/api` trong mẫu).

Phân quyền

`[Authorize]` / `.RequireAuthorization(...)` yêu cầu đăng nhập; thêm điều kiện:

```
app.MapGet("/toi", ...).RequireAuthorization(); // chỉ cần đăng n
app.MapGet("/quan-tri", ...).RequireAuthorization("QuanTri"); // theo policy
```

```
[Authorize(Roles = "Admin,Manager")] // controller/Raz
[AllowAnonymous] // ngoại lệ, cho
```

Policy — quy tắc có tên

```
builder.Services.AddAuthorizationBuilder()
    .AddPolicy("QuanTri", p => p.RequireRole("Admin"))
    .AddPolicy("TuoiTu18", p => p.RequireAssertion(ctx =>
        int.TryParse(ctx.User.FindFirstValue("tuoi"), out int tuoi) && tuoi >= 18))
    .AddPolicy("BearerAdmin", p => p.AddAuthenticationSchemes("Bearer").RequireRole("A
```

Policy dựa trên **role**, **claim** (`RequireClaim("department", "kho")`), điều kiện tùy ý (`RequireAssertion`) hoặc **requirement + handler** (bên dưới). Ưu tiên **policy có tên nghiệp vụ** thay vì rải `Roles = "Admin"` khắp nơi: đối quy tắc một chỗ. Có thể đặt **chính sách dự phòng** cho mọi endpoint: `o.FallbackPolicy = new AuthorizationPolicyBuilder().RequireAuthenticatedUser().Build();` — mặc định *chặn hết*, phải `AllowAnonymous` tường minh (an toàn theo mặc định).

Resource-based authorization — quyền phụ thuộc đối tượng

"Ai được xóa bài viết này?" — tùy **bài viết cụ thể** (tác giả hoặc admin), `[Authorize]` tĩnh không diễn tả được:

```
public class CungTacGiaRequirement : IAuthorizationRequirement;

public class CungTacGiaHandler : AuthorizationHandler<CungTacGiaRequirement, BaiViet>
{
    protected override Task HandleRequirementAsync(AuthorizationHandlerContext ctx, Cu
    {
        bool laTacGia = ctx.User.FindFirstValue(ClaimTypes.NameIdentifier) == bv.TacGi
        if (laTacGia || ctx.User.IsInRole("Admin")) ctx.Succeed(req);
        return Task.CompletedTask;
    }
}

// Trong endpoint: tải tài nguyên rồi hỏi
var kq = await auth.AuthorizeAsync(user, bv, new CungTacGiaRequirement());
if (!kq.Succeeded) return Results.Forbid();
```

Thử: An xóa bài của Admin → `403` ; An xóa bài của An → `204` ; Admin xóa bài của Admin → `204` . Đây là phòng thủ chính chống **IDOR** (Chương 21): *đừng chỉ kiểm tra "đã đăng nhập", phải kiểm tra "được truy cập đúng đối tượng này"*.

ASP.NET Core Identity

Tự viết đăng ký/đăng nhập như mẫu giúp hiểu cơ chế; **sản phẩm thật nên dùng ASP.NET Core Identity** — hệ thống thành viên đầy đủ:

- Lưu người dùng, vai trò, claim trong CSDL (qua EF Core): bảng `AspNetUsers`, `AspNetRoles` ...
- Bấm mật khẩu, chính sách mật khẩu, **khoá tài khoản** sau nhiều lần sai, xác nhận email, đổi/đặt lại mật khẩu, **xác thực hai yếu tố (2FA/TOTP)**, đăng nhập mạng xã hội (Google, Microsoft, GitHub).
- Giao diện có sẵn (scaffold) cho Razor Pages/MVC/Blazor; API endpoint có sẵn (`AddIdentityApiEndpoints<TUser>()`, `MapIdentityApi<TUser>()`) cho token/cookie.

```
builder.Services.AddDbContext<AppDbContext>(...);
builder.Services.AddIdentityApiEndpoints<IdentityUser>().AddEntityFrameworkStores<AppDbContext>().AddDefaultTokenProviders();
app.MapIdentityApi<IdentityUser>(); // /register, /login, /refresh, /manage/2fa,
```

Dịch vụ danh tính bên ngoài: OpenID Connect / OAuth 2.0

Quản lý mật khẩu, MFA, khoá tài khoản, tuân thủ... là bài toán khó. Nhiều tổ chức giao cho **nhà cung cấp danh tính (IdP)** và dùng chuẩn **OpenID Connect** (lớp danh tính trên **OAuth 2.0**): **Microsoft Entra ID, Auth0, Keycloak, Duende IdentityServer**, Google... Ứng dụng của bạn chuyển hướng người dùng tới IdP, nhận **ID token/access token** về, chỉ cần

`AddOpenIdConnect(...)` hoặc `AddJwtBearer(o => o.Authority = "https://idp...")`. Tập 3 trình bày chi tiết (flow Authorization Code + PKCE, refresh, scopes).

Bảo vệ nhiều lớp

- **HTTPS** cho mọi thứ (Chương 21).
- Giới hạn tốc độ đăng nhập (Chương 21).
- Ghi log sự kiện bảo mật (đăng nhập thất bại, đổi quyền) — không ghi mật khẩu/token.
- Nguyên tắc **đặc quyền tối thiểu**; mặc định từ chối (fallback policy).
- Kiểm tra quyền ở **server** cho mọi thao tác (ấn nút "Xóa" trên giao diện không phải là bảo mật).
- Blazor Server: `AuthorizeView`, `[Authorize]` trên component, nhưng vẫn kiểm tra ở tầng dịch vụ.

Lỗi thường gặp

- Đặt `UseAuthorization` trước `UseAuthentication` → luôn 401/403 (Chương 3).
- Quên `UseAuthentication()` → `User` luôn ẩn danh.

- Nhầm 401 với 403 ; redirect trang đăng nhập thay vì 401 cho API (cookie mặc định redirect).
- Khoá ký JWT yếu/viết cứng trong code/commit lên Git.
- Token sống quá lâu; không kiểm tra `aud/iss` .
- Đặt dữ liệu nhạy cảm vào JWT (không mã hoá).
- Chỉ kiểm tra "đã đăng nhập" mà không kiểm tra quyền sở hữu (IDOR).
- Lưu JWT ở `localStorage` trong ứng dụng có nguy cơ XSS.
- Thông báo đăng nhập tiết lộ "user không tồn tại".
- Tự viết mật mã/quản lý mật khẩu thay vì Identity/IIdP.
- Cookie không `Secure` / `HttpOnly` / `SameSite` ở production.

Bài tập

1. Thêm role `Manager` và policy `"QuanLyHoacAdmin"` (`RequireRole("Admin","Manager")`); gán cho một tài khoản mới và thử.
2. Đặt `FallbackPolicy` bắt buộc đăng nhập; thêm `.AllowAnonymous()` cho `/` và các endpoint đăng nhập.
3. Viết endpoint `POST /api/token/lam-moi` với refresh token lưu trong bộ nhớ (xoay vòng: mỗi refresh token dùng một lần).
4. Viết requirement `TuoiToiThieuRequirement(int tuoi)` + handler tái sử dụng cho nhiều policy.
5. Thêm `AddIdentityApiEndpoints` vào một dự án mới với EF Core SQLite và thử `/register` , `/login` bằng `curl` .

Chương 21 — Bảo mật ứng dụng web

Mục tiêu học

Sau chương này, bạn sẽ:

- Nắm các lỗ hổng phổ biến nhất (**OWASP Top 10**) và cách [ASP.NET](#) Core giúp phòng: **XSS, CSRF, SQL injection, IDOR, open redirect, SSRF**.
- Cấu hình **HTTPS/HSTS, header bảo mật, CSP, rate limiting**.
- Dùng **Data Protection** để ký/mã hoá dữ liệu ngắn hạn; xử lý bí mật, log và lỗi an toàn.
- Có **danh sách kiểm tra bảo mật** trước khi triển khai.

Code mẫu: [code/ch21-bao-mat-web/](#) — mỗi lỗ hổng có một endpoint "sai" và "đúng" để bạn thử.

Bảo mật không phải một tính năng thêm vào cuối, mà là **thói quen ở từng dòng code**. Nguyên tắc nền: *đừng tin dữ liệu từ bên ngoài, giảm tối thiểu quyền, phòng thủ nhiều lớp, thất bại một cách an toàn*.

OWASP Top 10 — bản đồ rủi ro

OWASP là tổ chức công bố danh sách 10 rủi ro bảo mật web phổ biến nhất (cập nhật định kỳ).

Ánh xạ với nội dung sách:

Rủi ro (rút gọn)	Ví dụ	Phòng ở đâu
Broken Access Control (IDOR, thiếu kiểm tra quyền)	đổi /don-hang/2 để xem đơn người khác	Ch. 20, mục IDOR dưới đây
Cryptographic Failures	lưu mật khẩu thô, dùng HTTP, khóa yếu	Ch. 17, 20; HTTPS/Data Protection
Injection (SQL, lệnh, XSS)	nối chuỗi vào SQL/HTML	Ch. 11, 13; XSS dưới đây
Insecure Design	thiếu giới hạn tốc độ, luồng nghiệp vụ hở	rate limiting, thiết kế
Security Misconfiguration	trang lỗi lộ stack trace, header thiếu, CORS quá rộng	Ch. 9; header
Vulnerable Components	thư viện NuGet có lỗ hổng	dotnet list package -vulnerable
Authentication Failures	brute-force, phiên yếu	Ch. 20; rate limit

Rủi ro (rút gọn)	Ví dụ	Phòng ở đâu
Software & Data Integrity	deserialize dữ liệu lạ, CI/CD không an toàn	Tập 1 Ch. 37; Tập 3
Logging & Monitoring Failures	không ghi/không theo dõi sự kiện bảo mật	Ch. 6, Tập 3
SSRF	ép server gọi địa chỉ nội bộ	Ch. 19, dưới đây

HTTPS và HSTS

HTTPS mã hoá đường truyền (TLS): chặn nghe lén, sửa đổi giữa đường, xác thực máy chủ. **Bắt buộc** cho mọi ứng dụng có đăng nhập/dữ liệu người dùng (trình duyệt còn chặn nhiều tính năng nếu không có HTTPS).

```
if (!app.Environment.IsDevelopment())
{
    app.UseHsts();                // Strict-Transport-Security: trình duyệt CHỈ dùng
    app.UseHttpsRedirection();    // 80 → 443
}
```

Chúng chỉ thật lấy từ CA (Let's Encrypt miễn phí) — thường cấu hình ở reverse proxy/load balancer (Tập 3). Đặt cookie `Secure`. Khi ứng dụng chạy **sau proxy** (Nginx, load balancer) hãy bật `UseForwardedHeaders` với **KnownProxies / KnownNetworks** rõ ràng, để biết IP thật/giao thức gốc — và *không* tin `X-Forwarded-For` từ nguồn lạ (ai cũng giả mạo được):

```
builder.Services.Configure<ForwardedHeadersOptions>(o =>
{
    o.ForwardedHeaders = ForwardedHeaders.XForwardedFor | ForwardedHeaders.XForwardedP
    o.KnownProxies.Add(IPAddress.Loopback);
});
app.UseForwardedHeaders();    // đặt SỚM
```

Header bảo mật

```
app.Use(async (ctx, next) =>
{
    var h = ctx.Response.Headers;
    h["X-Content-Type-Options"] = "nosniff";
    h["X-Frame-Options"] = "DENY";
    h["Referrer-Policy"] = "strict-origin-when-cross-origin";
    h["Permissions-Policy"] = "camera=(), microphone=(), geolocation=()";
    h["Content-Security-Policy"] = "default-src 'self'; script-src 'self'; style-src "
```

```
await next();
});
```

Header	Chống
X-Content-Type-Options: nosniff	trình duyệt "đoán" MIME (file tải lên bị thực thi như script)
X-Frame-Options: DENY / CSP frame-ancestors	clickjacking (trang khác nhúng trang bạn vào iframe ẩn để lừa bấm)
Referrer-Policy	rò rỉ URL nội bộ qua header Referer
Permissions-Policy	tính năng trình duyệt không cần (camera, mic)
Content-Security-Policy	XSS : chỉ cho phép tải script/style/ảnh từ nguồn liệt kê; script nội tuyến (<script>alert(1)</script>) bị chặn dù kẻ tấn công chèn được vào HTML
Strict-Transport-Security	hạ cấp về HTTP

CSP là lớp phòng thủ mạnh nhưng cần chỉnh cẩn thận với ứng dụng dùng script/CDN (dùng nonce cho script nội tuyến; Blazor cần nối tối thiểu). Công cụ đánh giá: securityheaders.com, Mozilla Observatory. Trong thực tế đóng gói thành middleware/thư viện (như `NetEscapades.AspNetCore.SecurityHeaders`).

XSS — Cross-Site Scripting

Kẻ tấn công cài `<script>` vào nội dung (bình luận, tên, tham số URL) → trình duyệt nạn nhân chạy nó dưới danh nghĩa trang của bạn → đánh cắp phiên, thực hiện hành động thay người dùng.

```
// ✗ Nối chuỗi thẳng vào HTML
app.MapGet("/xss-sai", (string ten) => Results.Content($"<h1>Xin chào {ten}</h1>", "text/html"))
// ?ten=<script>alert(1)</script> → <h1>Xin chào <script>alert(1)</script></h1> ←

// ✓ Mã hoá đầu ra
app.MapGet("/xss-dung", (string ten) => Results.Content($"<h1>Xin chào {HtmlEncoder.Encode(ten)}</h1>"))
// → <h1>Xin chào &lt;script&gt;alert(1)&lt;/script&gt;</h1>
```

Cả hai kết quả trên lấy từ lần chạy thật. Nguyên tắc: **mã hoá theo ngữ cảnh khi xuất** (HTML, thuộc tính, JavaScript, URL). Razor `@giaTri`, Blazor và tag helper **tự mã hoá HTML**, nên XSS xảy ra khi bạn:

- Dùng `@Html.Raw(...)` / `MarkupString` với dữ liệu người dùng.

- Tự nối chuỗi HTML như trên (Minimal API trả `text/html`).
- Nhúng dữ liệu vào `<script>` hoặc thuộc tính `onclick / href="javascript:..."` .
- Nhận HTML "giàu định dạng" (editor) — phải **lọc bằng thư viện** (HtmlSanitizer) theo whitelist thẻ.

Kèm **CSP** và cookie `HttpOnly` làm lớp phòng thủ thứ hai.

CSRF

Đã học ở Chương 17: antiforgery token cho form/cookie, `SameSite` , `POST` cho thay đổi. API dùng Bearer token không cần antiforgery.

SQL injection

Chương 11 và 13: **luôn tham số hoá**; EF Core LINQ và `FromSqlInterpolated` an toàn; **không** `FromSqlRaw` với chuỗi nối; tên cột/bảng động → whitelist.

IDOR — Insecure Direct Object Reference

Nếu API nhận `id` từ URL và chỉ kiểm tra người dùng *đã đăng nhập*, ai cũng đổi `id` để xem dữ liệu người khác. Đây là lỗi **số 1** OWASP (Broken Access Control):

```
app.MapGet("/don-hang/{id:int}", (int id, HttpContext ctx, DonHangKho kho) =>
{
    string nguoiDung = ...; // danh tính hiện tại
    var don = kho.Lay(id);
    if (don is null) return Results.NotFound();
    if (don.Chu != nguoiDung) return Results.NotFound(); // KHÔNG PHẢI CHỦ → 404 (
    return Results.Ok(don);
});
```

- **Luôn lọc theo chủ sở hữu ngay trong truy vấn** (`Where(d => d.KhachHangId == userId)`) hoặc dùng resource-based authorization (Chương 20).
- Trả **404** thay vì `403` để không tiết lộ đối tượng có tồn tại.
- ID khó đoán (GUID) **giảm** nhưng **không thay thế** kiểm tra quyền.

Thử: An xem `/don-hang/1` → 200; An xem `/don-hang/2` (của Bình) → 404.

Open redirect

`/dang-nhap?returnUrl=https://evil.com` — sau đăng nhập, chuyển người dùng sang trang lừa đảo trông như xuất phát từ trang uy tín. Chỉ chuyển hướng tới **đường dẫn nội bộ**:

```
static bool LaUrlNoiBo(string url)
    => !string.IsNullOrEmpty(url) && url[0] == '/' && (url.Length == 1 || (url[1] != '

Results.LocalRedirect(url);          // framework cũng ném lỗi nếu không phải URL cục bộ;
```

Chú ý các biến thể `//evil.com` và `/\evil.com` (trình duyệt hiểu là tên miền khác). Kết quả thử: `/don-hang/1` → 302; `https://evil.com`, `//evil.com`, `/\evil.com` → 400.

SSRF — Server-Side Request Forgery

Nếu server nhận **URL do người dùng cung cấp** rồi tự gọi (xem trước liên kết, webhook, tải ảnh từ URL), kẻ tấn công trở tới `http://169.254.169.254/` (metadata đám mây → lấy khoá truy cập), `http://localhost:8080/admin`, mạng nội bộ. Phòng: **whitelist host** cho phép, chặn dải IP nội bộ/loopback/link-local sau khi phân giải DNS, không theo redirect tự do, đặt timeout/giới hạn kích thước, chạy ở mạng cách ly. Đừng tin kiểm tra chỉ trên chuỗi URL.

Rate limiting

Chống brute-force mật khẩu, cào dữ liệu, làm dụng API, DoS nhẹ. Từ .NET 7 có sẵn middleware:

```
builder.Services.AddRateLimiter(o =>
{
    o.RejectionStatusCode = 429;
    o.GlobalLimiter = PartitionedRateLimiter.Create<HttpContext, string>(ctx =>
        RateLimitPartition.GetFixedWindowLimiter(ctx.Connection.RemoteIpAddress?.ToString
            _ => new FixedWindowRateLimiterOptions { PermitLimit = 20, Window = TimeSpan.Fr

    o.AddPolicy("dang-nhap", ctx => RateLimitPartition.GetFixedWindowLimiter(ip,
        _ => new FixedWindowRateLimiterOptions { PermitLimit = 3, Window = TimeSpan.Fr
    });
    app.UseRateLimiter();
    app.MapPost("/dang-nhap", ...).RequireRateLimiting("dang-nhap");
```

Kết quả thử:

```
POST /dang-nhap x5 → 401 401 401 429 429          (chính sách "dang-nhap": 3 lần / 3
GET / x25 liên tiếp → 200 x20 rồi 429 x5          (giới hạn chung 20 / 10 giây / IP
```

Bốn thuật toán: **Fixed window**, **Sliding window**, **Token bucket** (cho phép bùng nổ ngắn), **Concurrency** (giới hạn đồng thời). Chia theo **IP**, **người dùng**, **API key**. Khi sau proxy phải có `UseForwardedHeaders` đúng, nếu không mọi request có cùng IP proxy. Với hệ thống nhiều

instance dùng bộ đếm phân tán (Redis) hoặc rate limit ở API gateway/WAF. Phản hồi 429 kèm `Retry-After` .

Data Protection — ký và mã hoá dữ liệu ngắn hạn

API `IDataProtector` dùng để bảo vệ dữ liệu bạn cần gửi ra ngoài rồi nhận lại: token kích hoạt/đặt lại mật khẩu, tham số trong URL, nội dung cookie (chính cookie xác thực dùng nó):

```
var bv = dp.CreateProtector("kich-hoat-tai-khoan").ToTimeLimitedDataProtector();
string token = bv.Protect("user:42", lifetime: TimeSpan.FromMinutes(10)); // mã hoá
string goc = bv.Unprotect(token); // ném Cr
```

Thử: token hợp lệ → `{"noiDung":"user:42"}` ; sửa thêm 1 ký tự → "Token không hợp lệ hoặc hết hạn". Lưu ý: **purpose string** ("kich-hoat-tai-khoan") tách biệt miền sử dụng (token kích hoạt không dùng được làm token đặt lại mật khẩu). **Khoá** lưu ở đâu: mặc định thư mục hồ sơ người dùng — với nhiều instance/container phải dùng kho chung (`PersistKeysToFileSystem` , Redis, Azure Blob + `ProtectKeysWithAzureKeyVault`), nếu không mỗi instance có khoá riêng và token của instance này không giải mã được ở instance kia.

Bí mật, cấu hình, log, lỗi

- **Bí mật** (mật khẩu CSDL, khoá API, khoá ký): kho bí mật/biến môi trường, **không** commit (Chương 6). Đổi ngay nếu lỡ lộ.
- **Trang lỗi**: production không hiển thị stack trace (Chương 9); `UseDeveloperExceptionPage` chỉ ở Development.
- **Log**: không ghi mật khẩu/token/số thẻ; dùng tham số đặt tên để **chống log forging** (`log.LogInformation("Nguoi dung {Ten}", ten)` — giá trị chứa `\n` không tạo ra dòng log giả).
- **Thông báo lỗi** không lộ chi tiết nội bộ (tên bảng, đường dẫn).
- **Thư viện**: `dotnet list package --vulnerable --include-transitive` ; bật Dependabot/GitHub security alerts; cập nhật .NET (bản vá bảo mật hằng tháng).
- **CORS**: liệt kê chính xác origin (Chương 9).
- **Giới hạn kích thước request** (`MaxRequestBodySize`), số lượng, độ sâu JSON.
- **Chạy với quyền tối thiểu**: tài khoản CSDL chỉ có quyền cần dùng; container không chạy `root` .

Danh sách kiểm tra trước khi triển khai

- [] HTTPS + HSTS; cookie `Secure/HttpOnly/SameSite` .
- [] Xác thực đúng: mật khẩu băm (hoặc Identity/IdP), MFA nếu có thể; giới hạn tốc độ đăng nhập.

- [] Mọi endpoint nhạy cảm có `[Authorize]` /policy; `FallbackPolicy` chặn mặc định; kiểm tra **quyền sở hữu**.
- [] Validate mọi đầu vào ở server; tham số hoá SQL; mã hoá đầu ra; header bảo mật + CSP.
- [] Antiforgery cho form/cookie; CORS tối thiểu.
- [] Không có bí mật trong Git; kho bí mật ở production.
- [] Trang lỗi an toàn; log không chứa dữ liệu nhạy cảm; theo dõi cảnh báo.
- [] Thư viện được quét lỗ hổng; đã cập nhật .NET.
- [] Sao lưu dữ liệu, kế hoạch phản ứng sự cố.

Lỗi thường gặp

- Tin tưởng kiểm tra ở client; ấn nút thay cho kiểm tra quyền.
- `@Html.Raw / MarkupString` /nối HTML với dữ liệu người dùng.
- Kiểm tra "đã đăng nhập" nhưng bỏ sót quyền sở hữu (IDOR).
- Open redirect qua `returnUrl` không kiểm tra; SSRF qua URL người dùng.
- `AllowAnyOrigin + AllowCredentials` ; CORS coi như cơ chế bảo mật API.
- Cấu hình rate limit trước proxy mà không `UseForwardedHeaders` → giới hạn theo IP proxy.
- Khoá Data Protection không dùng chung giữa các instance → lỗi giải mã lạ sau khi scale/redeploy.
- Không cập nhật thư viện/.NET; lộ header `Server` /phiên bản.

Bài tập

1. Dùng `curl` gửi `?ten=<img src=x onerror=alert(1)>` tới `/xss-sai` và `/xss-dung` ; giải thích khác biệt và thử CSP chặn script nội tuyến trong trình duyệt.
2. Thêm `GET /don-hang` chỉ trả đơn của **người dùng hiện tại** (lọc trong truy vấn, không lọc trong bộ nhớ).
3. Đổi chính sách "dang-nhap" sang **sliding window** và **token bucket**; đo hành vi khác nhau bằng vòng lặp `curl` .
4. Viết token đặt lại mật khẩu hết hạn sau 15 phút dùng Data Protection và chỉ dùng được **một lần** (lưu `jti` đã dùng).
5. Chạy `dotnet list package --vulnerable --include-transitive` cho một dự án của chương này và ghi nhận kết quả.

Chương 22 — Dự án tổng hợp Tập 2: Web quản lý kho

Mục tiêu học

Sau chương này, bạn sẽ:

- Ghép mọi kiến thức Tập 2 thành **một hệ thống web hoàn chỉnh**: API + CSDL + xác thực + giao diện + kiểm thử.
- Thấy cách **đưa quy tắc nghiệp vụ của Tập 1 lên web** và xử lý đúng bài toán **đồng thời**.
- Có bộ khung dự án để mở rộng và tái cấu trúc sang **kiến trúc sạch** ở Tập 3.

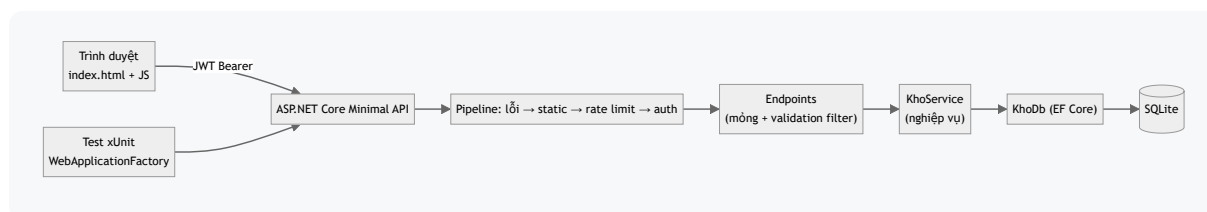
Code đầy đủ: [code/ch22-du-an-tong-hop/](#) — QuanLyKhoWeb (ứng dụng) + QuanLyKhoWeb.Tests (14 test tích hợp đều đạt).

Bài toán

Web hoá ứng dụng console *Quản lý kho* của Tập 1 (Chương 43) — nhiều nhân viên cùng thao tác:

#	Yêu cầu
1	Đăng nhập; vai trò Admin (thêm sản phẩm) và NhanVien (nhập/xuất kho)
2	Xem/tìm kiếm/phân trang sản phẩm; báo cáo theo nhóm; danh sách sắp hết hàng ; lịch sử giao dịch
3	Nhập kho và xuất kho — tồn kho không bao giờ âm kể cả khi nhiều người xuất cùng lúc
4	Ghi lại ai thực hiện giao dịch nào, khi nào
5	Lỗi trả đúng chuẩn (ProblemDetails); tài liệu OpenAPI; health check; giới hạn tốc độ đăng nhập
6	Một trang web đơn giản gọi API

Kiến trúc



QuanLyKhoWeb/

└─ Program.cs

└─ Data/KhoDb.cs

cấu hình: options, EF, JWT, policy, rate limit, lỗi,

entity SanPham, GiaoDich + DbContext + ràng buộc CHECK

```

├─ Services/
│   ├── KhoService.cs          # nghiệp vụ + DTO + lỗi nghiệp vụ
│   └─ XacThuc.cs             # tài khoản, phát hành JWT
├─ Endpoints/KhoEndpoints.cs  # route + phân quyền + validation
├─ Migrations/                # từ `dotnet ef migrations add`
└─ wwwroot/index.html         # giao diện đơn giản

```

Kiến trúc của các chương được dùng ở đâu:

Chương	Nơi dùng
3 Middleware	thứ tự <code>ExceptionHandler</code> → <code>UseStaticFiles</code> → <code>UseRateLimiter</code> → <code>UseAuthentication</code> → <code>UseAuthorization</code>
4 Minimal API	<code>MapGroup("/api")</code> , <code>Results<...></code> , endpoint filter
5 DI	<code>KhoService</code> Scoped, <code>TaiKhoanService/PhatHanhToken</code> Singleton, <code>TimeProvider</code>
6 Cấu hình	<code>JwtOptions</code> + <code>ValidateOnStart</code> ; <code>MigrateOnStartup</code> ; bí mật ngoài Git
8, 9	DTO, validation, <code>ExceptionHandler</code> , <code>ProblemDetails</code> , OpenAPI
10, 14	<code>WebApplicationFactory</code> , CSDL thật trong test
11–14 EF Core	ràng buộc, <code>ExecuteUpdate</code> , projection, transaction, migration
20, 21	JWT, policy theo vai trò, <code>FallbackPolicy</code> , rate limiting, mã hoá đầu ra
Tập 1	record, LINQ, async, exception nghiệp vụ, <code>TimeProvider</code> , SOLID

Những quyết định thiết kế đáng chú ý

1. An toàn theo mặc định

```

builder.Services.AddAuthorizationBuilder()
    .SetFallbackPolicy(new AuthorizationPolicyBuilder().RequireAuthenticatedUser().Build())
    .AddPolicy("GhiKho", p => p.RequireRole(VaiTro.NhanVien, VaiTro.Admin))
    .AddPolicy("QuanTri", p => p.RequireRole(VaiTro.Admin));

```

Mọi endpoint **mới** tự động bị bảo vệ; muốn công khai phải `AllowAnonymous()` tường minh (đăng nhập, health, OpenAPI). Quên gắn `[Authorize]` không còn là lỗi hổng. Test `KhongDangNhap_MoiEndpointTra401` khoá hành vi này.

2. Cấu hình phải hợp lệ mới cho chạy

Jwt:Khoa ngắn hơn 32 ký tự → ứng dụng **từ chối khởi động** (`ValidateOnStart`). Khoá phát triển chỉ được điền tự động ở `Development` ; production phải cấp bằng biến môi trường `Jwt__Khoa` . Tránh triển khai nhầm khoá mặc định.

3. Bài toán đồng thời — điểm quan trọng nhất

Ở Tập 1, `KhoService` giữ dữ liệu trong bộ nhớ của **một** tiến trình, một luồng. Trên web: nhiều request song song, có thể nhiều instance. Kiểu "đọc `TonKho` → kiểm tra đủ → trừ → ghi lại" có **race condition** (hai người cùng đọc 5, cùng trừ 4 → âm kho). Giải pháp của dự án: đưa điều kiện **vào chính câu lệnh cập nhật**:

```
await using var tx = await db.Database.BeginTransactionAsync(ct);
int n = await db.SanPhams.Where(s => s.Id == id && s.TonKho >= soLuong)
    .ExecuteUpdateAsync(s => s.SetProperty(x => x.TonKho, x => x.TonKho - soLuong), ct);

if (n == 0)
{
    var sp = await db.SanPhams.AsNoTracking().FirstOrDefaultAsync(s => s.Id == id, ct)
        ?? throw new KhongTimThayException(...);
    throw new KhongDuHangException(sp.Ma, soLuong, sp.TonKho);
}
db.GiaoDichs.Add(new GiaoDich { ... });
await db.SaveChangesAsync(ct);
await tx.CommitAsync(ct);
```

`UPDATE san_pham SET TonKho = TonKho - @n WHERE Id = @id AND TonKho >= @n` là **nguyên tử** ở CSDL: không cần khoá ứng dụng, không cần token đồng thời, đúng ngay cả khi có nhiều instance. Ràng buộc `CHECK (TonKho >= 0)` là lưới an toàn thứ hai. Thử bằng test:

```
// 20 yêu cầu xuất 1 đơn vị đồng thời cho sản phẩm chỉ còn 5
var ketQua = await Task.WhenAll(Enumerable.Range(0, 20).Select(_ => admin.PostAsJsonAs
Assert.Equal(5, ketQua.Count(r => r.StatusCode == HttpStatusCode.OK)); // đúng
Assert.Equal(15, ketQua.Count(r => r.StatusCode == HttpStatusCode.Conflict)); // 15 l
// tồn cuối = 0, không bao giờ âm
```

4. Bài học từ khi viết test (những lỗi thật đã gặp)

- **SQLite in-memory dùng chung một kết nối** (như Chương 14) không phản ánh đúng tình huống đồng thời: các transaction của nhiều request đâm lên nhau trên cùng một kết nối và test cho kết quả sai (chỉ 3 thay vì 5 lần thành công). Với test đồng thời hãy dùng **file SQLite tạm** (mỗi request một kết nối như production) và `Default Timeout` để chờ khoá ghi thay vì lỗi "database is locked". Nguyên tắc: *môi trường test phải giữ đúng tính chất mà bạn đang kiểm thử*.

- **SQLite không SUM được biểu thức decimal** (`TonKho * DonGia`): endpoint báo cáo trả 500 cho đến khi tính bằng `double` trong SQL rồi đổi về `decimal`. Trên SQL Server/PostgreSQL viết thẳng. Lỗi này **chỉ lộ nhờ test tích hợp**, không phải unit test nghiệp vụ — minh hoạ giá trị của chương 10/14.

5. Ghi nhận người thực hiện

Tên người dùng lấy từ **claim của token** (`user.Identity.Name`) — *không* nhận từ body request (client có thể giả mạo). Lịch sử giao dịch (`GiaoDich.NgườiThựcHien`) nhờ vậy đáng tin cậy để kiểm toán.

6. Giao diện đơn giản

`wwwroot/index.html` (HTML + JS thuần) đăng nhập, lấy JWT, gọi API bằng `fetch`, hiển thị danh sách, nhập/xuất kho. Cùng origin nên không cần CORS. Lưu ý ba điểm: dữ liệu chèn vào HTML đi qua hàm `esc()` (mã hoá, Chương 21); token giữ trong bộ nhớ trang (mất khi tải lại — đơn giản cho demo); ứng dụng thật dùng cookie `HttpOnly` hoặc OpenID Connect. Bạn hoàn toàn có thể thay bằng Blazor/Razor Pages/React gọi cùng API — đó là lợi thế của việc tách **API** khỏi **giao diện**.

Chạy và thử

```
cd QuanLyKhoWeb
dotnet run # tự migrate + nạp dữ liệu mẫu; mở http://localhost
```

Tài khoản mẫu: `admin / Admin@123` (Admin), `nhanvien / NhanVien@123` (NhanVien). Thử bằng `curl`:

```
TOKEN=$(curl -s -X POST http://localhost:5000/api/dang-nhap -H "Content-Type: applicat
-d '{"tenDangNhap":"nhanvien","matKhai":"NhanVien@123"}' | sed 's/.*"accessTok
curl -H "Authorization: Bearer $TOKEN" "http://localhost:5000/api/san-pham?tim=laptop"
curl -X POST -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
-d '{"soLuong":99}' http://localhost:5000/api/san-pham/1/xuat # 409: k
dotnet test ../QuanLyKhoWeb.Tests
```

Mở rộng (bài tập lớn)

1. **Xoá mềm** sản phẩm (Chương 13) và khôi phục; chỉ Admin.
2. **Nhiều kho** (`MaKho`): tồn kho theo từng kho, chuyển kho (một giao dịch trừ ở kho A, cộng ở kho B).
3. **Phiếu xuất/nhập nhiều dòng** (một phiếu gồm nhiều sản phẩm) — dùng transaction và xử lý thứ tự khoá tránh deadlock.

4. **Xuất báo cáo CSV/Excel** endpoint `GET /api/bao-cao/nhom.csv` (Tập 1, Chương 36).
5. Thay `TaiKhoanService` bằng **ASP.NET Core Identity** (+ refresh token) hoặc **OpenID Connect** (Chương 20).
6. Chuyển giao diện sang **Blazor** hoặc **Razor Pages**, giữ nguyên API (hoặc gọi thẳng dịch vụ).
7. **Phân trang keyset** cho lịch sử giao dịch; index (`SanPhamId`, `ThoiGian`) đã có sẵn.
8. Đổi provider sang **PostgreSQL** và ghi lại mọi thay đổi bạn phải làm (decimal, `Like`, migration).
9. **Idempotency-Key** cho `POST` `xuat` để gửi lại không trừ hai lần (Chương 19; Tập 3).

Hướng tới Tập 3

Dự án này là **kiến trúc "một khối" đơn giản**: endpoint gọi thẳng `KhoService`, `KhoService` dùng thẳng `DbContext`. Với hệ thống lớn, Tập 3 sẽ **tái cấu trúc** theo **Clean Architecture** (Domain / Application / Infrastructure / Web), tách **CQRS** (lệnh và truy vấn), dùng **Result pattern**, sự kiện miền, hàng đợi/nền, cache, quan sát (OpenTelemetry), **Docker**, **CI/CD** và triển khai lên đám mây — trên chính bài toán này.

Chúc mừng bạn đã hoàn thành Tập 2! Bạn đã xây được ứng dụng web thực thụ với [ASP.NET Core](#); sẵn sàng cho các vấn đề của hệ thống lớn ở Tập 3.

Phụ lục A — Lỗi thường gặp [ASP.NET Core](#)

Tra cứu theo **triệu chứng**. Cột "Chương" trỏ tới nội dung giải thích.

A.1 Khởi động và cấu hình

Triệu chứng / thông báo	Nguyên nhân	Cách xử lý	Ch.
Address already in use	cổng đang bị dùng	--urls http://localhost:5xxx; tắt tiến trình cũ	2
Trình duyệt báo chứng chỉ không tin cậy	chưa tin dev certificate	dotnet dev-certs https --trust	2
Cannot modify ServiceCollection after application started	gọi builder.Services.Add... sau Build()	đăng ký trước Build()	2, 5
OptionsValidationException khi khởi động	cấu hình sai (ValidateOnStart)	sửa giá trị theo thông báo	6
Cấu hình đọc ra null/mặc định	sai khoá, sai cấp, sai môi trường	dùng Options + validate; kiểm tra ASPNETCORE_ENVIRONMENT	6
FileNotFoundException: appsettings.json	không copy ra thư mục output	CopyToOutputDirectory (SDK Web đã làm sẵn)	6
Biến môi trường không ghi đè	dùng : thay __	CuaHang__Ten	6

A.2 Pipeline, routing, DI

Triệu chứng	Nguyên nhân	Cách xử lý	Ch.
Mọi URL rơi vào cùng một handler, endpoint không chạy	app.Run(...) terminal đặt trước endpoint	dùng MapFallback	3
Luôn 401/403 dù đã đăng nhập	UseAuthorization trước UseAuthentication, hoặc thiếu UseAuthentication	đúng thứ tự	3, 20
Headers are read-only, response has already started	sửa header sau khi body bắt đầu gửi	Response.OnStarting	3

Triệu chứng	Nguyên nhân	Cách xử lý	Ch.
Request treo	quên <code>await next(ctx) / .Result</code>	<code>await</code>	3
404 cho endpoint tồn tại	thiếu <code>MapControllers/MapRazorPages</code> , sai constraint (<code>{id:int}</code> với abc)	kiểm tra map và route	4, 7
<code>AmbiguousMatchException</code>	hai endpoint cùng route	đổi route / thêm constraint	4, 7
Body was inferred but the method does not allow inferred body parameters	tham số lớp không đăng ký DI bị hiểu là body	đăng ký dịch vụ, hoặc <code>[FromServices]/[FromBody]</code>	4
Unable to resolve service for type 'X'	quên đăng ký DI	<code>AddScoped<...></code>	5
Cannot consume scoped service from singleton	captive dependency	hạ vòng đời / <code>IServiceScopeFactory</code>	5
A circular dependency was detected	A cần B, B cần A	thiết kế lại	5
415 Unsupported Media Type	thiếu <code>Content-Type: application/json</code>	thêm header	7
400 bind lỗi kiểu	chuỗi vào số, JSON sai	xem errors trong <code>ProblemDetails</code>	4, 8

A.3 EF Core và dữ liệu

Triệu chứng	Nguyên nhân	Cách xử lý	Ch.
Dữ liệu không được lưu	quên <code>SaveChanges</code>	gọi (và <code>await</code>)	12
A second operation was started on this context	<code>DbContext</code> dùng chung nhiều luồng/quên <code>await</code>	mỗi request một context; <code>await</code> mọi lời gọi	12
no such table	chưa migrate/tạo DB	<code>Migrate()/dotnet ef database update</code>	12
Truy vấn chậm, hàng trăm câu SQL/request	N+1	<code>Include/projection</code> ; đếm câu SQL	13

Triệu chứng	Nguyên nhân	Cách xử lý	Ch.
NullReferenceException trên navigation	quên Include	Include hoặc projection	13
JsonException: possible object cycle	serialize entity có vòng tham chiếu	trả DTO	8, 13
DbUpdateException UNIQUE/FK/CHECK	vi phạm ràng buộc CSDL	dịch sang 409/400	12, 14
DbUpdateConcurrencyException	hai người sửa cùng dòng	tải lại, thử lại hoặc 409; hoặc UPDATE có điều kiện	13, 22
The LINQ expression could not be translated	dùng hàm C# EF không dịch được	viết lại, tách phần chạy trong bộ nhớ có chủ ý	12
SQLite does not support expressions of type 'decimal'	SUM/ORDER BY trên decimal ở SQLite	converter double; hoặc dùng CSDL khác	12, 22
Tìm kiếm phân biệt hoa/thường bất ngờ	Contains → instr trên SQLite	EF.Functions.Like	14
database is locked	nhiều writer SQLite	Default Timeout, WAL, hoặc CSDL server	22
Migration lệch schema	sửa migration đã áp dụng / trộn EnsureCreated	thêm migration mới	12

A.4 API, validation, lỗi

Triệu chứng	Nguyên nhân	Cách xử lý	Ch.
Trả 200 kèm nội dung lỗi	không dùng mã HTTP đúng	404/400/409; ProblemDetails	1, 9
Lộ stack trace ra client	trang lỗi dev ở production	UseExceptionHandler	9
[Required] không bắt được số vắng	kiểu giá trị mặc định 0	decimal? hoặc required	8
InvalidOperationException không chạy	còn lỗi validation thuộc tính	sửa lỗi thuộc tính trước (chủ ý)	8
Lỗi CORS chỉ trong trình duyệt	thiếu origin trong policy / sai thứ tự UseCors	liệt kê origin; UseCors trước auth	9

Triệu chứng	Nguyên nhân	Cách xử lý	Ch.
Test báo The type 'Program' is inaccessible	thiếu public partial class Program;	thêm	10
Test "xanh" nhưng không kiểm tra gì	quên await	await	10
Test lúc đạt lúc hỏng	trạng thái chung/thứ tự	dữ liệu riêng mỗi test	10

A.5 Giao diện web

Triệu chứng	Nguyên nhân	Cách xử lý	Ch.
The view 'X' was not found	sai tên/thư mục view	Views/<Controller>/X.cshtml	15
Model type mismatch (ViewDataDictionary)	truyền sai kiểu model	khớp @model	15
POST form trả 400	thiếu antiforgery token	dùng tag helper form / gửi header	17
F5 gửi lại form	không PRG	redirect sau POST thành công	17
Razor Page 404	thiếu @page	thêm @page	16
[BindProperty] rỗng khi GET	mặc định chỉ POST	SupportsGet = true	16
Lỗi validation không hiện cạnh ô	tên trường sai tiền tố (Input.Ma)	AddModelError("Input.Ma", ...)	16
Dropdown trống sau lỗi	quên nạp lại SelectList	nạp lại trước return Page()	16
Blazor: không tương tác	thiếu @rendermode	@rendermode InteractiveServer	18
Blazor: UI không cập nhật	đổi dữ liệu ngoài luồng sự kiện	InvokeAsync(StateHasChanged)	18
Tiếng Việt thành ộ trong HTML	mã hoá mặc định	WebEncoderOptions với UnicodeRanges.All	15

A.6 HttpClient và bảo mật

Triệu chứng	Nguyên nhân	Cách xử lý	Ch.
Cạn socket / SocketException	new HttpClient() mỗi request	IHttpClientFactory	19
Xử lý body lỗi như dữ liệu	quên EnsureSuccessStatusCode	kiểm tra mã	19
Timeout lâu, treo	không đặt timeout/CancellationToken	resilience + token	19
Thanh toán bị trừ hai lần	retry POST không idempotent	idempotency key; tắt retry cho POST	19
Token JWT bị 401 dù đúng	sai iss/aud, hết hạn, sai khoá	kiểm tra TokenValidationParameters, đồng hồ	20
Token giải mã không được ở instance khác	Data Protection khoá không dùng chung	lưu khoá dùng chung	21
Rate limit chặn cả nhóm người dùng	giới hạn theo IP proxy	UseForwardedHeaders đúng	21
429 khi test	rate limit bật	nới trong cấu hình test	21, 22

A.7 Cách tìm lỗi trong [ASP.NET Core](#)

1. Đọc **log** (mức `Information / Debug`): dòng `Request starting / Executing endpoint` cho biết request đã tới đâu.
2. Ở `Development` trang lỗi hiển thị stack trace; production dùng `traceId` trong `ProblemDetails` để tra log.
3. Bật log SQL: `Microsoft.EntityFrameworkCore.Database.Command = Information`.
4. Gọi bằng `curl -i / .http` để thấy header và body thật; DevTools → Network với trình duyệt.
5. Tái hiện bằng **integration test** rồi sửa — test ở lại làm hàng rào.

Phụ lục B — Tài liệu tham khảo (Tập 2)

Ưu tiên nguồn chính thức, miễn phí, cập nhật thường xuyên. Địa chỉ có thể đổi — nếu không mở được, tìm theo tên.

B.1 Tài liệu chính thức

- **ASP.NET Core:** <https://learn.microsoft.com/aspnet/core/> — Fundamentals, Minimal APIs, MVC, Razor Pages, Blazor, Security, Performance.
- **Entity Framework Core:** <https://learn.microsoft.com/ef/core/> — quan hệ, migration, hiệu năng, provider.
- **Bảo mật ASP.NET Core:** <https://learn.microsoft.com/aspnet/core/security/> — Identity, JWT, Data Protection, CORS, rate limiting.
- **HttpClient & resilience:** <https://learn.microsoft.com/dotnet/core/resilience/> — `Microsoft.Extensions.Http.Resilience`, `Polly`.
- **OpenAPI/Scalar:** <https://learn.microsoft.com/aspnet/core/fundamentals/openapi/>, <https://github.com/scalar/scalar>.
- **Blazor:** <https://learn.microsoft.com/aspnet/core/blazor/>.
- **RFC 9110** (HTTP semantics), **RFC 9457** (Problem Details), **RFC 7519** (JWT): tra cứu chuẩn.
- **OWASP:** <https://owasp.org/www-project-top-ten/>, **Cheat Sheet Series** <https://cheatsheetseries.owasp.org/>.
- **Mã nguồn ASP.NET Core:** <https://github.com/dotnet/aspnetcore>; **EF Core:** <https://github.com/dotnet/efcore>.

B.2 Sách

Sách	Tác giả	Nội dung
<i>ASP.NET Core in Action</i> (bản mới)	Andrew Lock	toàn diện, giải thích "vì sao"
<i>Pro ASP.NET Core</i>	Adam Freeman	tham khảo rộng
<i>Entity Framework Core in Action</i>	Jon P Smith	EF Core chuyên sâu
<i>Designing Web APIs</i>	Brenda Jin et al.	thiết kế API
<i>RESTful Web API Patterns and Practices Cookbook</i>	Mike Amundsen	mẫu thiết kế API
<i>Web Application Security</i>	Andrew Hoffman	bảo mật ứng dụng web
<i>Release It!</i>	Michael Nygard	chịu lỗi, retry/circuit breaker

Sách	Tác giả	Nội dung
<i>Designing Data-Intensive Applications</i>	Martin Kleppmann	nền tảng hệ thống dữ liệu
<i>SQL Antipatterns</i>	Bill Karwin	tránh lỗi thiết kế CSDL

B.3 Blog, video, cộng đồng

- **Andrew Lock** (andrewlock.net), **Khalid Abuhakmeh**, **Steve Gordon** — [ASP.NET](https://asp.net) Core chuyên sâu.
- **Milan Jovanović**, **Nick Chapsas**, **Tim Corey**, **Raw Coding** — video thực hành.
- **.NET Blog**, [ASP.NET](https://asp.net) Community Standup, **.NET Conf**.
- **Stack Overflow** (`asp.net-core` , `entity-framework-core`), **r/dotnet**, **Discord .NET**.

B.4 Công cụ

Nhu cầu	Công cụ
Thử API	file .http (VS/VS Code), Postman, Insomnia, Bruno , curl, Scalar/Swagger UI
CSDL	DB Browser for SQLite, Azure Data Studio /SSMS, pgAdmin /DBeaver
Xem SQL EF	<code>ToQueryString()</code> , <code>log Database.Command</code> , MiniProfiler
Kiểm thử	xUnit, WebApplicationFactory, Testcontainers (CSDL thật trong Docker), Respawn
Tải/hiệu năng	k6 , NBomber , <code>dotnet-counters</code> , BenchmarkDotNet
Bảo mật	<code>dotnet list package --vulnerable</code> , Dependabot, OWASP ZAP, securityheaders.com
Gỡ lỗi mạng	DevTools, Fiddler, mitmproxy, Wireshark

B.5 Luyện tập

1. Mở rộng dự án Chương 22 theo danh sách bài tập lớn.
2. Viết lại một API bạn từng dùng (thư viện sách, blog, đặt lịch) từ đầu đến khi có test, JWT, Docker.
3. Đọc mã nguồn mẫu chính thức: **eShop** (`dotnet/eShop`), **Contoso University**.
4. Thử thách bảo mật: **OWASP Juice Shop**, **PortSwigger Web Security Academy** (miễn phí).
5. Đóng góp/đọc issue của một dự án mã nguồn mở .NET web.

B.6 Lộ trình tiếp theo

Tập 3 của bộ sách: kiến trúc sạch, CQRS, mẫu thiết kế nâng cao, quan sát, bảo mật nâng cao (OIDC), Docker, CI/CD, triển khai đám mây, microservices tổng quan.

Phụ lục C — Cheat sheet [ASP.NET](#) Core

C.1 Lệnh

```
dotnet new web -n App      dotnet new webapi      dotnet new mvc      dotnet new razor
dotnet run [--urls http://localhost:5000]    dotnet watch run      dotnet publish -c Re
dotnet dev-certs https --trust
dotnet user-secrets set "K" "V"
dotnet ef migrations add Ten      dotnet ef database update      dotnet ef migrations
dotnet list package --vulnerable --include-transitive      dotnet list package --outda
```

C.2 Program.cs khung

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers(); // hoặc AddControllersWithViews / Add
builder.Services.AddDbContext<AppDb>(o => o.UseSqlite(builder.Configuration.GetConnect
builder.Services.AddScoped<IDichVu, DichVu>());
builder.Services.AddOpenApi(); builder.Services.AddProblemDetails(); builder.Service
builder.Services.AddAuthentication().AddJwtBearer(); builder.Services.AddAuthorizatio
builder.Services.AddRateLimiter(...); builder.Services.AddCors(...); builder.Service

var app = builder.Build();

app.UseExceptionHandler(); app.UseHsts(); app.UseHttpsRedirection();
app.UseStaticFiles(); app.UseRouting(); app.UseRateLimiter(); app.UseCors();
app.UseAuthentication(); app.UseAuthorization(); app.UseAntiforgery();

app.MapGet("/", () => "ok"); app.MapControllers(); app.MapRazorPages(); app.MapHeal
app.Run();
public partial class Program;
```

C.3 Minimal API

```
var g = app.MapGroup("/api/todos").WithTags("Todos").RequireAuthorization();
g.MapGet("/", (AppDb db, string? tim, int trang = 1) => ...);
g.MapGet("/{id:int}", Results<Ok<Todo>, NotFound> (int id, AppDb db) => ...);
g.MapPost("/", (TaoTodo req, AppDb db) => TypedResults.Created($"api/todos/{id}", tod
g.MapPut("/{id:int}", ...); g.MapDelete("/{id:int}", ...);
.AddEndpointFilter<MyFilter>() .RequireAuthorization("Policy") .AllowAnonymous()
```

Binding: route {id}, query (kiểu đơn giản), body (kiểu phức tạp), [FromHeader], [FromForm], [FromServices], [AsParameters], CancellationToken, HttpContext, ClaimsPrincipal. Kết quả: TypedResults.Ok / Created / NoContent / NotFound / BadRequest / Conflict / ValidationProblem / Problem / File / Redirect.

C.4 Controller

```
[ApiController] [Route("api/[controller]")]
public class XController(IDichVu dv) : ControllerBase
{
    [HttpGet("{id:int}")] public async Task<ActionResult<Dto>> Get(int id, Cancellatio
    [HttpPost] public async Task<ActionResult<Dto>> Post(TaoDto req) => CreatedAtActio
}
```

C.5 Razor Pages / MVC / Blazor

```
@page "{id:int}" @model XModel @* Razor Page *@
<form method="post"> <input asp-for="Input.Ten" /> <span asp-validation-for="Input.Ten
<a asp-page="Details" asp-route-id="@x.Id">...</a> <a asp-controller="X" asp-actio
```

```
[BindProperty] public Input Input { get; set; } public async Task<IActionResult
```

```
@page "/" @rendermode InteractiveServer @inject IDichVu Dv
<button @onclick="Tang">@so</button> <input @bind="ten" @bind:event="oninput" />
<EditForm Model="m" OnValidSubmit="Luu"><DataAnnotationsValidator /><InputText @bind-V
@code { int so; void Tang() => so++; [Parameter] public int Id { get; set; } [Paramete
```

C.6 DI

```
AddTransient<I, C>() AddScoped<I, C>() AddSingleton<I, C>() AddKeyedScoped<I, C>
AddOptions<T>().Bind(cfg.GetSection("X")).ValidateDataAnnotations().ValidateOnStart()
IOptions<T> (cố định) · IOptionsSnapshot<T> (mỗi request) · IOptionsMonitor<T> (theo d
```

Quy tắc: dịch vụ sống lâu **không** phụ thuộc dịch vụ sống ngắn (Singleton →/Scoped).

DbContext luôn Scoped.

C.7 EF Core

```

db.Set.Add(x); await db.SaveChangesAsync(); db.Set.Remove(x);
await db.Set.AsNoTracking().Where(...).OrderBy(...).Select(x => new Dto(...)).ToListAsync
await db.Set.Where(...).ExecuteUpdateAsync(s => s.SetProperty(x => x.Ton, x => x.Ton -
.Include(x => x.Con).ThenInclude(...) .AsSplitQuery() .IgnoreQueryFilters() EF.F
await using var tx = await db.Database.BeginTransactionAsync(); ... await tx.CommitAsy
e.HasIndex(x => x.Ma).IsUnique(); e.HasOne(x => x.A).WithMany(a => a.Bs).HasForeignKey

```

C.8 Xác thực / phân quyền

```

app.UseAuthentication(); app.UseAuthorization(); // đúng thứ tự
AddAuthorizationBuilder().SetFallbackPolicy(...RequireAuthenticatedUser()).AddPolicy("
.RequireAuthorization("P") [Authorize(Roles="A,B")] [AllowAnonymous]
await auth.AuthorizeAsync(user, resource, new MyRequirement()); // resource-based
ctx.SignInAsync(scheme, principal) ctx.SignOutAsync(scheme)

```

C.9 Mã trạng thái nên dùng

Tình huống	Mã
Đọc thành công	200
Tạo mới	201 + Location
Thành công không body	204
Dữ liệu gửi sai	400 (ValidationProblem)
Chưa xác thực / token sai	401
Không đủ quyền	403
Không tồn tại (hoặc không được biết là tồn tại)	404
Xung đột (trùng mã, hết hàng, phiên bản)	409
Dữ liệu hợp lệ về cú pháp nhưng sai nghiệp vụ	422
Quá giới hạn tốc độ	429 + Retry-After
Lỗi máy chủ	500
Dịch vụ ngoài lỗi / chậm	502 / 504
Quá tải/bảo trì	503

C.10 Danh sách kiểm tra nhanh

- [] Pipeline đúng thứ tự (lỗi → HTTPS → static → routing → CORS → auth → endpoint).

- [] DTO vào $\neq$ ra $\neq$ entity; validation ở server; PRG với form; antiforgery.
- [] SQL luôn tham số hoá; không N+1; có phân trang và giới hạn kích thước.
- [] Fallback policy bắt đăng nhập; kiểm tra quyền sở hữu; rate limit đăng nhập.
- [] Bí mật ngoài Git; HTTPS; header bảo mật; log không lộ dữ liệu nhạy cảm.
- [] Test tích hợp cho luồng chính và nhánh lỗi; `dotnet list package --vulnerable`.

